nuclear-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `nuclear_engineering_ai999x.about()` for the same answer.

Every capability nuclear-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import nuclear_engineering_ai999x

result = nuclear_engineering_ai999x.solve('nuc_specific_activity', nuclide='Co-60', m='0.001 kg')
result.show()
# A = 4.18689e+13 1/s, A_Ci = 1131.59, a = 4.18689e+16 1/(kg*s), a_Ci_per_g = 1131.59, N =
1.0048e+22
```

By topic

topic	capabilities
Decay and activity / decay chains	Three-member decay chain with initial activities (Bateman)
Decay and activity / decay chains and equilibrium	Time of maximum daughter activity, Transient and secular equilibrium: activity ratio and approach
Decay and activity / decay chains of real nuclides	Activity of a radionuclide and its progeny after a decay time
Decay and activity / generators	Radionuclide generator: daughter activity after elution (with branching)
Decay and activity / neutron activation	Neutron activation: activity after irradiation and decay, Activation with target depletion and product burn-up, Irradiation time to reach a fraction of saturation activity
Decay and activity / specific activity	Activity and specific activity of a mass of a radionuclide, Mass and number of atoms of a radionuclide with a given activity
Detectors and counting / activation foils	Neutron flux from a measured foil activation
Detectors and counting / counting statistics	Optimal division of counting time between sample and background, Counting time for a target precision of the net count rate
Detectors and counting / dead time	Dead-time correction, non-paralyzable detector, Dead-time correction, paralyzable detector
Detectors and counting / decay	Counts recorded from a decaying source (decay during counting)

topic	capabilities
corrections	
Detectors and counting / decay-curve analysis	Half-life from a measured decay curve (weighted log-linear fit), Two-component decay curve: amplitudes of two known half-lives
Detectors and counting / detection limits	Currie critical level, detection limit and minimum detectable activity
Detectors and counting / efficiency	Total and intrinsic efficiency of a slab detector for a point source on axis, Expected count rate from a point source on a detector's axis
Detectors and counting / gamma spectra	Compton edge and backscatter peak of a gamma line
Detectors and counting / gamma spectroscopy	Single and double escape peaks of a gamma ray
Detectors and counting / geometry	Solid angle and geometric efficiency of a disk detector (source on axis), Solid angle of a disk detector for an off-axis point source
Dosimetry / effective dose	Effective dose from organ equivalent doses and tissue weighting factors
Dosimetry / internal dose	Committed effective dose from a dose coefficient
Dosimetry / internal emitters	Effective half-life (physical and biological removal), Committed equivalent dose to an organ from an intake (retention compartments)
Dosimetry / units	Radiation unit conversion (Bq and Ci, Gy and rad, Sv and rem)
Fuel cycle / enrichment	Uranium enrichment: feed, tails and separative work (SWU)
Neutron physics / cross sections	Maxwellian-averaged 1/v cross section, Number densities and macroscopic cross sections of UO2 fuel
Neutron physics / diffusion	Diffusion length from an exponential pile measurement, Flux from a plane neutron source at the centre of a slab, Flux from a point neutron source at the centre of a sphere, Thermal diffusion length and migration length
Neutron physics / moderation	Neutron energy and lab angle after an elastic collision, Average logarithmic energy decrement and mean scattering cosine, Average number of collisions to slow a neutron down, Moderating power and ratio of a compound moderator
Neutron physics / thermal neutrons	Speeds and mean energy of thermal neutrons (Maxwell distribution)
Reactor kinetics / control rods	Integral and differential control-rod worth (sine-squared weighting)
Reactor kinetics / inhour equation	Stable period and prompt root of the inhour equation (six delayed groups), Reactivity from a measured stable period or doubling time (inhour)
Reactor kinetics / point kinetics	Stable reactor period after a reactivity step (one delayed group), Power after a reactivity step (one delayed group, exact), Power after a reactivity step (six delayed groups, exact)
Reactor physics / approach to criticality	Approach to criticality: inverse multiplication (1/M) extrapolation

topic	capabilities
Reactor physics / criticality of bare reactors	Critical thickness of a bare slab reactor (one group), Critical radius of a bare spherical reactor (one group), Critical radius of a bare cylindrical reactor of given height (one group), Minimum-volume critical bare cylinder (one group), Critical third side of a bare rectangular reactor (one group), K-effective and geometric buckling of a bare reactor (one group)
Reactor physics / decay heat	Decay heat after shutdown (Way-Wigner), Decay heat energy released between two times after shutdown
Reactor physics / fission-product poisons	Equilibrium iodine-135 and xenon-135 concentrations, Xenon-135 after shutdown: concentration and peak, Samarium-149 poisoning: equilibrium and after shutdown
Reactor physics / flux shapes	Fundamental-mode flux shape of a bare reactor at a point
Reactor physics / fuel depletion and burnup	U-235 depletion and energy released at constant flux, U-235 depletion at constant power
Reactor physics / neutron life cycle	Multiplication factor by the four- and six-factor formulas
Reactor physics / power and flux	Power density and power from the fission reaction rate
Reactor physics / power and flux shapes	Reactor power from the peak flux (or peak flux from power) of a bare core
Reactor physics / reflected reactors	Critical thickness of a reflected slab reactor and reflector savings (one group)
Reactor physics / two-group theory	K-effective of a bare reactor in two-group diffusion theory
Shielding / attenuation	Half-value and tenth-value layers
Shielding and dose / attenuation	Narrow-beam transmission through a multilayer shield, Linear attenuation coefficient from narrow-beam transmission measurements
Shielding and dose / dose over time	Dose received passing a point source, Dose accumulated near a decaying source, Stay time to a dose limit near a decaying source
Shielding and dose / extended sources	Uncollided flux from a shielded line source (Sievert integral), Uncollided flux on the axis of a shielded disk source
Shielding and dose / photon interactions	Klein-Nishina Compton cross sections per electron
Shielding and dose / point sources	Dose rate from a shielded point gamma source with buildup
Shielding and dose / shield design	Shield thickness for a target dose rate (point source, buildup)
Thermal hydraulics / coolant channels	Coolant temperature along a channel with a chopped-cosine power shape, Location and value of the maximum cladding surface temperature
Thermal hydraulics / fuel rods	Fuel-rod radial temperatures: centreline, pellet surface and cladding, Fuel centreline temperature with temperature-dependent conductivity

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
nuc_specific_activity	Activity and specific activity of a mass of a radionuclide	nuclide, m	A, A_Ci, a, a_Ci_per_g, N
nuc_mass_for_activity	Mass and number of atoms of a radionuclide with a given activity	nuclide, A	m, m_ug, N
nuc_chain_activity	Activity of a radionuclide and its progeny after a decay time	nuclide, A0, t	A_parent, A_daughter, A_total, ratio
nuc_three_member_chain	Three-member decay chain with initial activities (Bateman)	T1, T2, T3, A10, A20, A30, t	A1, A2, A3, A_total
nuc_daughter_max_activity_time	Time of maximum daughter activity	T_P, T_D, A_P0	t_max, ratio_max, t_max_half_lives, A_D_max
nuc_equilibrium_ratio	Transient and secular equilibrium: activity ratio and approach	T_P, T_D	ratio_A, ratio_N, t_99
nuc_generator_daughter_activity	Radionuclide generator: daughter activity after elution (with branching)	A_P0, T_P, T_D, b, t	A_D, A_P, ratio, t_max, A_D_max
nuc_counts_during_decay	Counts recorded from a decaying source (decay during counting)	A0, T_half, tc, eff, y, td	N, A_mean, F
nuc_effective_half_life	Effective half-life (physical and biological removal)	T_p, T_b, A0	T_eff, lambda_eff, N_decays
nuc_activation_activity	Neutron activation: activity after irradiation and decay	m, M, theta, sigma, phi, T_half, t_irr, t_d	A_eoi, A_d, A_sat, S
nuc_activation_with_burnup	Activation with target depletion and product burn-up	N_T0, sigma_T, sigma_P, phi, T_half, t	A_P, f_T, t_max, A_max
nuc_irradiation_time_for_saturation	Irradiation time to reach a fraction of saturation activity	T_half, f	t_irr, n_half
nuc_decay_curve_fit	Half-life from a measured decay curve (weighted log-linear fit)	t, C, t_unit	lambda_, T_half, C0, sigma_T
nuc_two_component_decay_fit	Two-component decay curve: amplitudes of two known half-lives	t, C, T1, T2, t_unit	C1, C2, frac1
nuc_decay_heat_power	Decay heat after shutdown (Way-Wigner)	P0, T_op, t	P_d, ratio

id	operation	inputs	outputs
nuc_decay_heat_energy	Decay heat energy released between two times after shutdown	P0, T_op, t1, t2	E, E_MWh, P_mean
nuc_committed_dose_retention	Committed equivalent dose to an organ from an intake (retention compartments)	I, f, Tb, t_unit, T_p, E, w_R, m, tau	U, H, H_mSv
nuc_point_source_dose_rate	Dose rate from a shielded point gamma source with buildup	Gamma, A, r, mu, x, buildup, A1, alpha1, alpha2	H_dot, H_uSv_h, B, T
nuc_shield_thickness_for_dose_rate	Shield thickness for a target dose rate (point source, buildup)	Gamma, A, r, mu, H_target, buildup	x, n_mfp, B
nuc_multilayer_transmission	Narrow-beam transmission through a multilayer shield	mu, x, H0	T, tau, n_hvl, attenuation, H
nuc_line_source_flux	Uncollided flux from a shielded line source (Sievert integral)	S_L, h, y1, y2, mu, t	phi, phi0, T
nuc_disk_source_flux	Uncollided flux on the axis of a shielded disk source	S_A, R, h, mu, t	phi, phi0, T
nuc_transit_dose	Dose received passing a point source	Gamma, A, d, v, L	D, D_uSv, frac
nuc_accumulated_dose_decaying_source	Dose accumulated near a decaying source	H0, T_half, t, td	D, D_mSv, D_inf, H_mean
nuc_stay_time_decaying_source	Stay time to a dose limit near a decaying source	H0, T_half, D_lim	t_stay, frac
nuc_klein_nishina	Klein-Nishina Compton cross sections per electron	E	sigma, sigma_b, sigma_tr, T_mean
nuc_compton_edge	Compton edge and backscatter peak of a gamma line	E	T_max, E_bs
nuc_dead_time_nonparalyzable	Dead-time correction, non-paralyzable detector	m, tau	n, loss, corr
nuc_dead_time_paralyzable	Dead-time correction, paralyzable detector	m, tau, branch	n, loss, m_max
nuc_count_time_split	Optimal division of counting time between sample and background	T, Rg, Rb	tg, tb, sigma, rel
nuc_count_time_for_precision	Counting time for a target precision of the net count rate	Rg, Rb, r	T, tg, tb
nuc_solid_angle_on_axis	Solid angle and geometric efficiency of	d, a	Omega, g

id	operation	inputs	outputs
	a disk detector (source on axis)		
nuc_solid_angle_off_axis	Solid angle of a disk detector for an off-axis point source	d, a, rho0	Omega, g
nuc_detector_efficiency_point_source	Total and intrinsic efficiency of a slab detector for a point source on axis	d, a, t, mu	eff, g, intrinsic
nuc_inverse_multiplication	Approach to criticality: inverse multiplication (1/M) extrapolation	x, C	x_crit, slope, x_crit_last2, inv_M_last
nuc_elastic_scattering	Neutron energy and lab angle after an elastic collision	A, E, theta	E_out, ratio, psi, du
nuc_logarithmic_energy_decrement	Average logarithmic energy decrement and mean scattering cosine	A	alpha, xi, mu_bar
nuc_collisions_to_thermalize	Average number of collisions to slow a neutron down	A, E0, Ef	n, xi
nuc_moderator_mixture	Moderating power and ratio of a compound moderator	n, A, sigma_s, sigma_a, rho, M	xi, MP, MR
nuc_maxwellian_average_cross_section	Maxwellian-averaged 1/v cross section	sigma0, T	sigma_avg, sigma_avg_b, g
nuc_fuel_cross_sections	Number densities and macroscopic cross sections of UO2 fuel	rho, w, sigma_a5, sigma_f5, sigma_a8, sigma_aO, nu	N5, N8, NO, Sigma_a, Sigma_f, eta, mfp_a
nuc_critical_slab	Critical thickness of a bare slab reactor (one group)	D, Sigma_a, nuSigma_f, extrapolation	a, a_ext
nuc_critical_sphere	Critical radius of a bare spherical reactor (one group)	D, Sigma_a, nuSigma_f, extrapolation	R, R_ext, V
nuc_critical_cylinder	Critical radius of a bare cylindrical reactor of given height (one group)	D, Sigma_a, nuSigma_f, extrapolation, H	R, R_ext, V
nuc_critical_cylinder_minimum_volume	Minimum-volume critical bare cylinder (one group)	D, Sigma_a, nuSigma_f, extrapolation	R, H, V_ext, HD
nuc_critical_cuboid	Critical third side of a bare rectangular reactor (one group)	D, Sigma_a, nuSigma_f, extrapolation, a, b	c, c_ext, V
nuc_bare_reactor_keff	k-effective and geometric buckling of a bare reactor (one group)	D, Sigma_a, nuSigma_f, extrapolation, geometry, a, b, c, R, H	B2, k_eff, P_NL, rho

id	operation	inputs	outputs
nuc_two_group_bare_keff	k-effective of a bare reactor in two-group diffusion theory	geometry, D1, D2, Sigma_a1, Sigma_12, Sigma_a2, nuSigma_f1, nuSigma_f2, a, b, c, R, H	B2, k_eff, k_inf, flux_ratio
nuc_reflected_slab_critical	Critical thickness of a reflected slab reactor and reflector savings (one group)	D_c, Sigma_ac, nuSigma_fc, D_r, Sigma_ar, T_r, extrapolation	a, delta
nuc_exponential_pile	Diffusion length from an exponential pile measurement	z, phi, a, b	gamma, L, L2
nuc_plane_source_slab_flux	Flux from a plane neutron source at the centre of a slab	S, D, Sigma_a, a, x	phi, phi0, leak
nuc_point_source_sphere_flux	Flux from a point neutron source at the centre of a sphere	S, D, Sigma_a, R, r	phi, leak, phi_inf
nuc_flux_shape	Fundamental-mode flux shape of a bare reactor at a point	geometry, a, R, H, x, r, z	ratio, B2
nuc_core_power_from_flux	Reactor power from the peak flux (or peak flux from power) of a bare core	geometry, Sigma_f, E_f, phi_max, P, R, H, a, b, c	P, phi_max, phi_avg, peaking
nuc_reactor_period_one_group	Stable reactor period after a reactivity step (one delayed group)	rho, beta, lam, Lambda	omega, T, T_d
nuc_power_after_step_one_group	Power after a reactivity step (one delayed group, exact)	rho, beta, lam, Lambda, t	P, A1, omega
nuc_power_after_step_six_group	Power after a reactivity step (six delayed groups, exact)	rho, beta, lam, Lambda, t	P, omega, T
nuc_stable_period_six_group	Stable period and prompt root of the inhour equation (six delayed groups)	rho, beta, lam, Lambda	omega, T, T_d, omega_p
nuc_reactivity_from_period	Reactivity from a measured stable period or doubling time (inhour)	beta, lam, Lambda, T, T_d	rho, rho_dollars, rho_pcm
nuc_control_rod_worth	Integral and differential control-rod worth (sine-squared weighting)	rho_tot, H, x, beta	rho, frac, drho_dx, rho_dollars
nuc_xenon_equilibrium	Equilibrium iodine-135 and xenon-135 concentrations	Sigma_f, phi, gamma_I, gamma_X, sigma_X	I_eq, X_eq, abs_per_fission

id	operation	inputs	outputs
nuc_xenon_after_shutdown	Xenon-135 after shutdown: concentration and peak	Sigma_f, phi, gamma_I, gamma_X, sigma_X, t	X, ratio, t_peak, peak_ratio, I
nuc_samarium	Samarium-149 poisoning: equilibrium and after shutdown	Sigma_f, phi, gamma_Pm, sigma_Sm, t	Pm_eq, Sm_eq, Sm_max, max_ratio, Sm_t
nuc_depletion_constant_flux	U-235 depletion and energy released at constant flux	m0, sigma_a, sigma_f, phi, t, E_f	m, burnt, E, E_MWd, P_t
nuc_enrichment_swu	Uranium enrichment: feed, tails and separative work (SWU)	x_p, x_f, x_w, P	F, W, SWU, F_per_P, SWU_per_P
nuc_fuel_rod_temperatures	Fuel-rod radial temperatures: centreline, pellet surface and cladding	q_lin, R_f, k_f, h_g, t_c, k_c, h, T_cool	T_0, T_fs, T_ci
nuc_centreline_temperature_variable_k	Fuel centreline temperature with temperature-dependent conductivity	q_lin, T_s, A, B	T_0, dT, k_s, k_0
nuc_channel_coolant_temperature	Coolant temperature along a channel with a chopped-cosine power shape	q0, H, H_e, m_dot, c_p, T_in, z	T, T_out, Q
nuc_max_clad_temperature	Location and value of the maximum cladding surface temperature	q0, H, H_e, m_dot, c_p, T_in, R_o, h	z_max, T_max, T_out
nuc_thermal_neutron_speeds	Speeds and mean energy of thermal neutrons (Maxwell distribution)	T	v_p, v_mean, v_rms, E_mean, E_mean_eV
nuc_attenuation_coefficient_fit	Linear attenuation coefficient from narrow-beam transmission measurements	x, I	mu, HVL, TVL, I0
nuc_count_rate_point_source	Expected count rate from a point source on a detector's axis	A, y, d, a, eps_int	R, g, R_cpm
nuc_flux_from_foil_activation	Neutron flux from a measured foil activation	A_d, m, M, theta, sigma, T_half, t_irr, t_d	phi, A_sat, R_atom
nuc_effective_dose	Effective dose from organ equivalent doses and tissue weighting factors	H, w	E_mSv, E, w_sum
nuc_committed_dose_coefficient	Committed effective dose from a dose	I, e	E, E_mSv, I_Bq

id	operation	inputs	outputs
	coefficient		
nuc_power_from_reaction_rate	Power density and power from the fission reaction rate	E_f, N, sigma, phi, V	R, q, q_MW_m3, P, P_MW
nuc_six_factor_formula	Multiplication factor by the four- and six-factor formulas	eta, f, p, eps, P_FNL, P_TNL	k_inf, k_eff, P_NL, rho
nuc_diffusion_length	Thermal diffusion length and migration length	D, Sigma_a, tau	L, L2, L_cm, M2, M
nuc_currie_detection_limits	Currie critical level, detection limit and minimum detectable activity	B, t, eff, y	L_C, L_D, MDA
nuc_escape_peaks	Single and double escape peaks of a gamma ray	E	single_escape_keV, double_escape_keV, annihilation_keV, E_keV
nuc_half_value_layer	Half-value and tenth-value layers	mu, x	HVL, TVL, mfp, HVL_cm, TVL_cm, transmission, n_HVL
nuc_radiation_unit_conversion	Radiation unit conversion (Bq and Ci, Gy and rad, Sv and rem)	A, D, H	SI_value, traditional_value, si_unit, traditional_unit, value_mCi, value_MBq, value_mSv_or_mGy, value_mrem_or_mrad
nuc_constant_power_depletion	U-235 depletion at constant power	m0, P, t, E_f, sigma_a, sigma_f	m, burnt, consumed, flux_ratio, fissions

Activity and specific activity of a mass of a radionuclide — `nuc_specific_activity`

The activity of a stated mass of a named radionuclide, its specific activity (activity per unit mass) and the number of atoms, from its tabulated half-life and atomic mass.

$$A = \lambda N = \frac{\ln 2}{T_{1/2}} \, \frac{m \, N_A}{M}$$

input	meaning	condition
`nuclide`	the radionuclide, written like $Co-60$, $Cs-137$, $Tc-99m$, $U-235$	
`m`	mass of the radionuclide (the pure nuclide, not the compound)	

output	meaning
`A`	activity (Bq)
`A_Ci`	activity in curies
`a`	specific activity (Bq/kg)
`a_Ci_per_g`	specific activity in Ci/g
`N`	number of atoms

- **Method:** $N = m$ N_A / M, $A = \lambda$ N with $\lambda = ln$ 2 / T_half

- **Assumes:** the nuclide is in the table (ICRP-107 half-lives, radioactivedecay 0.6.1); $m > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_specific_activity', nuclide='Co-60', m=0.001)
# -> A = 4.18689e+13 1/s, A_Ci = 1131.59, a = 4.18689e+16 1/(kg*s), a_Ci_per_g = 1131.59, N =
1.0048e+22   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_specific_activity', nuclide='Ra-226', m=0.001)
# -> A = 3.65767e+10 1/s, A_Ci = 0.988559, a = 3.65767e+13 1/(kg*s), a_Ci_per_g = 0.988559, N =
2.66436e+21   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_specific_activity', nuclide='Cs-137', m=1e-09)
# -> A = 3.20274e+06 1/s, A_Ci = 8.65606e-05, a = 3.20274e+15 1/(kg*s), a_Ci_per_g = 86.5606, N =
4.39871e+15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Specific activity: nuclide = Co-60, m = 1 g	SOLVED	A = 4.18689e+13 1/s, A_Ci = 1131.59, a = 4.18689e+16 1/(kg*s), a_Ci_per_g = 1131.59, N = 1.0048e+22
Specific activity: nuclide = Ra-226, m = 1 g	SOLVED	A = 3.65767e+10 1/s, A_Ci = 0.988559, a = 3.65767e+13 1/(kg*s), a_Ci_per_g = 0.988559, N = 2.66436e+21
Specific activity: nuclide = Cs-137, m = 1 ug	SOLVED	A = 3.20274e+06 1/s, A_Ci = 8.65606e-05, a = 3.20274e+15 1/(kg*s), a_Ci_per_g = 86.5606, N = 4.39871e+15
Specific activity: m = 1 g	NEEDS_INPUT	nuc_specific_activity needs nuclide (the radionuclide, written like Co-60, Cs-137, Tc-99m, U-235). Example: solve('nuc_s
Specific activity: nuclide = Co-60, m = 1 g, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Specific activity: nuclide = co60, m = 1000 mg	SOLVED	A = 4.18689e+13 1/s, A_Ci = 1131.59, a = 4.18689e+16 1/(kg*s), a_Ci_per_g = 1131.59, N = 1.0048e+22
Specific activity: nuclide = Xx-999, m = 1 g	INVALID_INPUT	nuclide: one of H-3, C-14, Na-22, P-32, K-40, Co-57, Co-60, Kr-85, Sr-89, Sr-90, Y-90, Tc-99m, Mo-99, I-125, I-131, I-13
Specific activity: nuclide = Co-60, m = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Mass and number of atoms of a radionuclide with a given activity — `nuc_mass_for_activity`

The mass and the number of atoms of a named radionuclide that has a stated activity (how much material a source of that strength contains).

$$m = \frac{A\,T_{1/2}}{\ln 2}\,\frac{M}{N_A}$$

input	meaning	condition
`nuclide`	the radionuclide, written like $Co-60$, $Cs-137$, $Tc-99m$, $U-235$	
`A`	activity (Bq, or Ci)	

output	meaning
`m`	mass of the nuclide
`m_ug`	mass in micrograms
`N`	number of atoms

- **Method:** $N = A$ T_half / ln 2, $m = N$ M / N_A
- **Assumes:** the nuclide is in the table; $A > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_mass_for_activity', nuclide='Tc-99m', A=370000000.0)
# -> m = 1.8984e-12 kg, m_ug = 0.0018984, N = 1.15588e+13   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_mass_for_activity', nuclide='Am-241', A=37000.0)
# -> m = 2.91423e-10 kg, m_ug = 0.291423, N = 7.28041e+14   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mass for activity: nuclide = Tc-99m, A = 10 mCi	SOLVED	m = 1.8984e-12 kg, m_ug = 0.0018984, N = 1.15588e+13
Mass for activity: nuclide = Am-241, A = 37 kBq	SOLVED	m = 2.91423e-10 kg, m_ug = 0.291423, N = 7.28041e+14
Mass for activity: A = 10 mCi	NEEDS_INPUT	nuc_mass_for_activity needs nuclide (the radionuclide, written like Co-60, Cs-137, Tc-99m, U-235). Example: solve('nuc_m
Mass for activity: nuclide = Tc-99m, A = 10 mCi, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Mass for activity: nuclide = Tc-99m, A = 370 MBq	SOLVED	m = 1.8984e-12 kg, m_ug = 0.0018984, N = 1.15588e+13
Mass for activity: nuclide = Tc-99m, A = -5 Bq	INVALID_INPUT	Mass and number of atoms of a radionuclide with a given activity: the activity A must be a positive number.

Activity of a radionuclide and its progeny after a decay time — `nuc_chain_activity`

Starting from a pure parent of stated activity, the activities of the parent, of its first daughter and of the whole chain (all radioactive progeny) after a time t, with the real branching fractions (Sr-90/Y-90, $Mo-99/Tc-99m$, $Cs-137/Ba-137m$, $Ge-68/Ga-68$, ...).

$$N_i(t) = \sum_j c_{ij}\, e^{-\lambda_j t}, \quad c_{ij} = \frac{\sum_p b_{pi}\lambda_p c_{pj}}{\lambda_i - \lambda_j}$$

input	meaning	condition
`nuclide`	the parent nuclide (Sr-90, $Cs-137$, $Mo-99$, $Ge-68$, $W-188$, $Ru-106$, $Ce-144$, $Ba-140$, $Te-132$, $Sn-113$, $Sr-82$, $Zr-95$, $Sb-125$, $Pb-212$, $Te-131m$, $I-133$, $Pd-103$, $Ce-134$, $Ba-131$, $Hg-195m$, Sr-91)	
`A0`	activity of the pure parent at $t = 0$	

input	meaning	condition
`t`	decay time	

output	meaning
`A_parent`	activity of the parent at t
`A_daughter`	activity of the first daughter at t (Y-90, $Ba-137m$, $Tc-99m$, $Ga-68$, $Re-188$, $Rh-106$, $Pr-144$, $La-140$, $I-132$, $In-113m$, $Rb-82$, $Nb-95$, $Te-125m$, $Bi-212$, $I-131$, $Xe-133$, $Rh-103m$, $La-134$, $Cs-131$, $Hg-195$, Y-91m)
`A_total`	total activity of the parent and all radioactive progeny
`ratio`	daughter-to-parent activity ratio

- **Method:** Bateman solution of the whole chain (every branch), evaluated with 40-digit arithmetic
- **Assumes:** pure parent at $t = 0$; $t \geq 0$; branching fractions and half-lives of radioactivedecay 0.6.1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_chain_activity', nuclide='Mo-99', A0=37000000000.0,
t=86400.0)
# -> A_parent = 2.87499e+10 1/s, A_daughter = 2.55061e+10 1/s, A_total = 5.4256e+10 1/s, ratio =
0.887172   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_chain_activity', nuclide='Sr-90', A0=1000000.0, t=864000.0)
# -> A_parent = 999341 1/s, A_daughter = 924949 1/s, A_total = 1.92429e+06 1/s, ratio = 0.925558
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_chain_activity', nuclide='Cs-137', A0=1000.0, t=3600.0)
# -> A_parent = 999.997 1/s, A_daughter = 943.988 1/s, A_total = 1943.98 1/s, ratio = 0.94399
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chain activity: nuclide = Mo-99, A0 = 1 Ci, t = 1 d	SOLVED	A_parent = 2.87499e+10 1/s, A_daughter = 2.55061e+10 1/s, A_total = 5.4256e+10 1/s, ratio = 0.887172
Chain activity: nuclide = Sr-90, A0 = 1 MBq, t = 10 d	SOLVED	A_parent = 999341 1/s, A_daughter = 924949 1/s, A_total = 1.92429e+06 1/s, ratio = 0.925558
Chain activity: nuclide = Cs-137, A0 = 1 kBq, t = 1 h	SOLVED	A_parent = 999.997 1/s, A_daughter = 943.988 1/s, A_total = 1943.98 1/s, ratio = 0.94399
Chain activity: A0 = 1 Ci, t = 1 d	NEEDS_INPUT	nuc_chain_activity needs nuclide (the parent nuclide (Sr-90, Cs-137, Mo-99, Ge-68, W-188, Ru-106, Ce-144, Ba-140, Te-132
Chain activity: nuclide = Mo-99, A0 = 1 Ci, t = 1 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Chain activity: nuclide = Mo-99, A0 = 37 GBq, t = 24 h	SOLVED	A_parent = 2.87499e+10 1/s, A_daughter = 2.55061e+10 1/s, A_total = 5.4256e+10 1/s, ratio = 0.887172
Chain activity: nuclide = Co-60, A0 = 1 Ci, t = 1 d	INVALID_INPUT	nuclide: one of Sr-90, Cs-137, Mo-99, Ge-68, W-188, Ru-106, Ce-144, Ba-140, Te-132, Sn-113, Sr-82, Zr-95, Sb-125, Pb-212

Three-member decay chain with initial activities (Bateman) — `nuc_three_member_chain`

Activities of the three radioactive members of a chain 1 -> 2 -> 3 after a time t, from their half-lives and the initial activity of every member (the daughters need not start at zero).

$$N_3 = N_{30}e^{-\lambda_3 t} + \frac{\lambda_2 N_{20}}{\lambda_3 - \lambda_2}(e^{-\lambda_2 t} - e^{-\lambda_3 t}) + \lambda_1\lambda_2 N_{10}\sum_i \frac{e^{-\lambda_i t}}{\prod_{j\neq i}(\lambda_j - \lambda_i)}$$

input	meaning	condition
`T1`	half-life of member 1 (the parent)	
`T2`	half-life of member 2	
`T3`	half-life of member 3	
`A10`	initial activity of member 1	
`A20`	initial activity of member 2 (0 if none)	
`A30`	initial activity of member 3 (0 if none)	
`t`	elapsed time	

output	meaning
`A1`	activity of member 1
`A2`	activity of member 2
`A3`	activity of member 3
`A_total`	total activity

- **Method:** Bateman solution with the initial numbers $N_i0 = A_i0/lambda_i$, in 40-digit arithmetic
- **Assumes:** every decay of 1 gives 2 and of 2 gives 3 (no branching); the half-lives differ; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_three_member_chain', T1=100.0, T2=10.0, T3=1.0, A10=1000.0,
A20=0.0, A30=0.0, t=50.0)
# -> A1 = 707.107 1/s, A2 = 750.952 1/s, A3 = 755.03 1/s, A_total = 2213.09 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_three_member_chain', T1=3600.0, T2=7200.0, T3=600.0,
A10=1000000.0, A20=500000.0, A30=0.0, t=1800.0)
# -> A1 = 707107 1/s, A2 = 554238 1/s, A3 = 472939 1/s, A_total = 1.73428e+06 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Three-member chain: T1 = 100.0 s, T2 = 10.0 s, T3 = 1.0 s, A10 = 1000.0 1/s, A20 = 0.0 1/s, A30 = 0.0 1/s, t = 50.0 s	SOLVED	A1 = 707.107 1/s, A2 = 750.952 1/s, A3 = 755.03 1/s, A_total = 2213.09 1/s
Three-member chain: T1 = 1 h, T2 = 2 h, T3 = 10 min, A10 = 1 MBq, A20 = 0.5 MBq, A30 = 0 Bq, t = 30 min	SOLVED	A1 = 707107 1/s, A2 = 554238 1/s, A3 = 472939 1/s, A_total = 1.73428e+06 1/s

question	outcome	result
Three-member chain: T2 = 10.0 s, T3 = 1.0 s, A10 = 1000.0 1/s, A20 = 0.0 1/s, A30 = 0.0 1/s, t = 50.0 s	NEEDS_INPUT	nuc_three_member_chain needs T1 (half-life of member 1 (the parent)). Example: solve('nuc_three_member_chain', T1='100 s
Three-member chain: T1 = 100.0 s, T2 = 10.0 s, T3 = 1.0 s, A10 = 1000.0 1/s, A20 = 0.0 1/s, A30 = 0.0 1/s, t = 50.0 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Three-member chain: T1 = 100 s, T2 = 100 s, T3 = 1 s, A10 = 1000 Bq, A20 = 0 Bq, A30 = 0 Bq, t = 50 s	INVALID_INPUT	Three-member decay chain with initial activities (Bateman): the three half-lives must differ (by more than 0.01 %).

Time of maximum daughter activity — `nuc_daughter_max_activity_time`

When the activity of a radioactive daughter, growing from a pure parent, reaches its maximum, and how large it is then (it equals the parent's activity at that moment).

$$t_{max} = \frac{\ln(\lambda_D/\lambda_P)}{\lambda_D - \lambda_P}$$

input	meaning	condition
`T_P`	half-life of the parent	
`T_D`	half-life of the daughter	
`A_P0`	initial parent activity (optional: gives the maximum daughter activity)	

output	meaning
`t_max`	time of maximum daughter activity
`ratio_max`	maximum daughter activity / initial parent activity
`t_max_half_lives`	t_max in daughter half-lives
`A_D_max`	maximum daughter activity (when A_P0 is given)

- **Method:** $dA_D/dt = 0$: t_max = ln(lambda_D/lambda_P)/(lambda_D - lambda_P) (1/lambda when equal); A_D(t_max) = A_P(t_max)
- **Assumes:** pure parent at $t = 0$; every parent decay gives the daughter
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_daughter_max_activity_time', T_P=237384.0, T_D=21654.0)
# -> t_max = 82312.6 s, ratio_max = 0.786354, t_max_half_lives = 3.80127   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_daughter_max_activity_time', T_P=10.0, T_D=10.0, A_P0=1000.0)
# -> t_max = 14.427 s, ratio_max = 0.367879, t_max_half_lives = 1.4427, A_D_max = 367.879 1/s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Daughter maximum activity time: T_P = 65.94 h, T_D = 6.015 h	SOLVED	t_max = 82312.6 s, ratio_max = 0.786354, t_max_half_lives = 3.80127
Daughter maximum activity time: T_P = 10.0 s, T_D = 10.0 s, A_P0 = 1000.0 1/s	SOLVED	t_max = 14.427 s, ratio_max = 0.367879, t_max_half_lives = 1.4427, A_D_max = 367.879 1/s
Daughter maximum activity time: T_D = 6.015 h	NEEDS_INPUT	nuc_daughter_max_activity_time needs T_P (half-life of the parent). Example: solve('nuc_daughter_max_activity_time', T_P
Daughter maximum activity time: T_P = 65.94 h, T_D = 6.015 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Transient and secular equilibrium: activity ratio and approach — `nuc_equilibrium_ratio`

The daughter-to-parent activity and atom ratios once equilibrium is reached (transient; close to 1 in secular equilibrium) and the time after which the activity ratio is within 1 % of it, starting from pure parent.

$$\frac{A_D}{A_P} = \frac{\lambda_D}{\lambda_D - \lambda_P}\left(1 - e^{-(\lambda_D - \lambda_P)t}\right)$$

input	meaning	condition
`T_P`	half-life of the parent	
`T_D`	half-life of the daughter (shorter than the parent's)	

output	meaning
`ratio_A`	equilibrium activity ratio $A_D/A_P = T_P/(T_P - T_D)$
`ratio_N`	equilibrium atom ratio N_D/N_P
`t_99`	time to reach 99 % of the equilibrium activity ratio

- **Method:** A_D/A_P -> lambda_D/(lambda_D - lambda_P); the ratio approaches it as $1 - exp$(-(lambda_D - lambda_P) t)
- **Assumes:** $T_P > T_D$; pure parent at $t = 0$; every parent decay gives the daughter
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_equilibrium_ratio', T_P=237384.0, T_D=21654.0)
# -> ratio_A = 1.10038, ratio_N = 0.100375, t_99 = 158307 s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_equilibrium_ratio', T_P=908523901.8432001, T_D=230760.0)
# -> ratio_A = 1.00025, ratio_N = 0.000254059, t_99 = 1.53353e+06 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equilibrium ratio: T_P = 237384.0 s, T_D = 21654.0 s	SOLVED	ratio_A = 1.10038, ratio_N = 0.100375, t_99 = 158307 s

question	outcome	result
Equilibrium ratio: T_P = 908523901.8432001 s, T_D = 230760.0 s	SOLVED	ratio_A = 1.00025, ratio_N = 0.000254059, t_99 = 1.53353e+06 s
Equilibrium ratio: T_D = 21654.0 s	NEEDS_INPUT	nuc_equilibrium_ratio needs T_P (half-life of the parent). Example: solve('nuc_equilibrium_ratio', T_P='237384 s', T_D='
Equilibrium ratio: T_P = 237384.0 s, T_D = 21654.0 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Transient equilibrium: T_P = 1 h, T_D = 2 h	INVALID_INPUT	Transient and secular equilibrium: activity ratio and approach: no equilibrium: the parent must live longer than the dau

Radionuclide generator: daughter activity after elution (with branching) — `nuc_generator_daughter_activity`

The daughter activity available in a generator (Mo-99/Tc-99m, $Ge-68/Ga-68$, $Sr-82/Rb-82$ ...) a time t after the last complete elution, with the branching fraction to the daughter, plus when the daughter activity peaks and its peak value.

$$A_D(t) = b\,A_{P0}\frac{\lambda_D}{\lambda_D-\lambda_P}\left(e^{-\lambda_P t}-e^{-\lambda_D t}\right)$$

input	meaning	condition
`A_P0`	parent activity at the elution	
`T_P`	half-life of the parent	
`T_D`	half-life of the daughter	
`b`	fraction of parent decays that produce the daughter (0 < b <= 1, e.g. 0.8773 for $Mo-99$ -> Tc-99m)	
`t`	time since the elution	

output	meaning
`A_D`	daughter activity at t
`A_P`	parent activity at t
`ratio`	A_D / A_P
`t_max`	time of maximum daughter activity
`A_D_max`	maximum daughter activity

- **Method:** A_D(t) = b A_P0 lambda_D/(lambda_D - lambda_P) (e^(-lambda_P $t) - e^{(- lambda_D}$ t)) (40 digits); t_max = ln(lambda_D/lambda_P)/(lambda_D - lambda_P)
- **Assumes:** daughter fully removed at the elution; $0 < b \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_generator_daughter_activity', A_P0=37000000000.0,
T_P=237384.0, T_D=21654.0, b=0.8773, t=86400.0)
# -> A_D = 2.55061e+10 1/s, A_P = 2.87499e+10 1/s, ratio = 0.887172, t_max = 82312.6 s, A_D_max =
2.55251e+10 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_generator_daughter_activity', A_P0=1000000000.0,
T_P=23410080.0, T_D=4062.6, b=1.0, t=3600.0)
# -> A_D = 4.5891e+08 1/s, A_P = 9.99893e+08 1/s, ratio = 0.458959, t_max = 50760.6 s, A_D_max =
9.98498e+08 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Generator daughter activity: A_P0 = 1 Ci, T_P = 65.94 h, T_D = 6.015 h, b = 0.8773, t = 24 h	SOLVED	A_D = 2.55061e+10 1/s, A_P = 2.87499e+10 1/s, ratio = 0.887172, t_max = 82312.6 s, A_D_max = 2.55251e+10 1/s
Generator daughter activity: A_P0 = 1 GBq, T_P = 23410080.0 s, T_D = 4062.6 s, b = 1.0, t = 1 h	SOLVED	A_D = 4.5891e+08 1/s, A_P = 9.99893e+08 1/s, ratio = 0.458959, t_max = 50760.6 s, A_D_max = 9.98498e+08 1/s
Generator daughter activity: T_P = 65.94 h, T_D = 6.015 h, b = 0.8773, t = 24 h	NEEDS_INPUT	nuc_generator_daughter_activity needs A_P0 (parent activity at the elution). Example: solve('nuc_generator_daughter_acti
Generator daughter activity: A_P0 = 1 Ci, T_P = 65.94 h, T_D = 6.015 h, b = 0.8773, t = 24 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Generator elution: A_P0 = 1 Ci, T_P = 65.94 h, T_D = 6.015 h, b = 87.73%, t = 24 h	SOLVED	A_D = 2.55061e+10 1/s, A_P = 2.87499e+10 1/s, ratio = 0.887172, t_max = 82312.6 s, A_D_max = 2.55251e+10 1/s
Generator elution: A_P0 = 1 Ci, T_P = 65.94 h, T_D = 6.015 h, b = 87.73, t = 24 h	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Counts recorded from a decaying source (decay during counting) — `nuc_counts_during_decay`

The expected number of counts recorded from a source that decays during the count, the mean activity during the count and the decay-correction factor that converts the mean activity to the activity at the start of counting.

$$N = \varepsilon\, y\, A_0 e^{-\lambda t_d} \frac{1 - e^{-\lambda t_c}}{\lambda}$$

input	meaning	condition
`A0`	activity at the reference time	
`T_half`	half-life	
`tc`	counting time	
`eff`	detection efficiency (fraction of emitted radiations counted)	
`y`	emission probability per decay (1 when every decay is counted)	
`td`	delay from the reference time to the start of counting (0 when omitted)	

output	meaning
`N`	expected counts
`A_mean`	mean activity during the count

output	meaning
`F`	decay-correction factor: activity at the start of counting / mean activity

- **Method:** N = eff y A0 e^(-lambda td) (1 - e^(-lambda tc))/lambda; $F = \lambda$ tc/(1 - e^(-lambda tc))
- **Assumes:** 0 < eff <= 1; $0 < y \leq 1$; no dead-time losses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_counts_during_decay', A0=100000.0, T_half=6586.2, tc=3600.0,
eff=0.2, y=0.967)
# -> N = 5.79538e+07, A_mean = 83238.3 1/s, F = 1.20137   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_counts_during_decay', A0=37000.0, T_half=600.0, tc=600.0,
eff=0.1, y=1.0, td=300.0)
# -> N = 1.13235e+06, A_mean = 18872.6 1/s, F = 1.38629   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Counts during decay: A0 = 100 kBq, T_half = 6586.2 s, tc = 1 h, eff = 20%, y = 96.7%	SOLVED	N = 5.79538e+07, A_mean = 83238.3 1/s, F = 1.20137
Counts during decay: A0 = 1 uCi, T_half = 10 min, tc = 10 min, eff = 0.1, y = 1.0, td = 5 min	SOLVED	N = 1.13235e+06, A_mean = 18872.6 1/s, F = 1.38629
Counts during decay: T_half = 6586.2 s, tc = 1 h, eff = 20%, y = 96.7%	NEEDS_INPUT	nuc_counts_during_decay needs A0 (activity at the reference time). Example: solve('nuc_counts_during_decay', A0='100000
Counts during decay: A0 = 100 kBq, T_half = 6586.2 s, tc = 1 h, eff = 20%, y = 96.7%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Effective half-life (physical and biological removal) — `nuc_effective_half_life`

The effective half-life of a radionuclide in the body or another system that removes it biologically while it decays, the effective decay constant and (with the initial activity) the total number of decays.

$$\frac{1}{T_{eff}} = \frac{1}{T_p} + \frac{1}{T_b}$$

input	meaning	condition
`T_p`	physical half-life	
`T_b`	biological half-life	
`A0`	initial activity in the body (optional)	

output	meaning
`T_eff`	effective half-life
`lambda_eff`	effective removal constant

output	meaning
`N_decays`	total number of decays in the body (cumulated activity, when A0 is given)

- **Method:** 1/T_eff = 1/T_p + 1/T_b; cumulated activity A0 T_eff/ln 2
- **Assumes:** single-exponential biological removal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_effective_half_life', T_p=692988.48, T_b=6912000.0)
# -> T_eff = 629841 s, lambda_eff = 1.10051e-06 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_effective_half_life', T_p=21654.0, T_b=86400.0,
A0=370000000.0)
# -> T_eff = 17314.5 s, lambda_eff = 4.00327e-05 1/s, N_decays = 9.24245e+12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective half-life: T_p = 8.0207 d, T_b = 80 d	SOLVED	T_eff = 629841 s, lambda_eff = 1.10051e-06 1/s
Effective half-life: T_p = 21654.0 s, T_b = 1 d, A0 = 10 mCi	SOLVED	T_eff = 17314.5 s, lambda_eff = 4.00327e-05 1/s, N_decays = 9.24245e+12
Effective half-life: T_b = 80 d	NEEDS_INPUT	nuc_effective_half_life needs T_p (physical half-life). Example: solve('nuc_effective_half_life', T_p='692988 s', T_b='6
Effective half-life: T_p = 8.0207 d, T_b = 80 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Neutron activation: activity after irradiation and decay — `nuc_activation_activity`

The activity induced in a target by a neutron flux: at the end of irradiation, after a decay time, at saturation, and the saturation fraction reached (target depletion neglected).

$$A = N\sigma\phi\left(1 - e^{-\lambda t_{irr}}\right)e^{-\lambda t_d}$$

input	meaning	condition
`m`	mass of the target element	
`M`	molar mass of the target element	
`theta`	isotopic abundance of the target nuclide	
`sigma`	activation cross section (barns)	
`phi`	neutron flux	
`T_half`	half-life of the product	
`t_irr`	irradiation time	
`t_d`	decay time after irradiation (0 when omitted)	

output	meaning
`A_eoi`	activity at the end of irradiation
`A_d`	activity after the decay time
`A_sat`	saturation activity N sigma phi
`S`	saturation fraction 1 - e^(-lambda t_irr)

- **Method:** A(t_irr) = N sigma phi (1 - e^(-lambda t_irr)), $N = m$ N_A theta / M; then A e^(-lambda t_d)
- **Assumes:** constant flux; target depletion and product burn-up neglected; $0 < \theta \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_activation_activity', m=0.001, M=0.058933195, theta=1.0,
sigma=3.718e-27, phi=1e+17, T_half=166346024.445504, t_irr=2592000.0)
# -> A_eoi = 4.08136e+10 1/s, A_d = 4.08136e+10 1/s, A_sat = 3.79927e+12 1/s, S = 0.0107425
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_activation_activity', m=0.0001, M=0.02698, theta=1.0,
sigma=2.31e-29, phi=1e+16, T_half=134.7, t_irr=600.0, t_d=60.0)
# -> A_eoi = 4.9209e+08 1/s, A_d = 3.61373e+08 1/s, A_sat = 5.1561e+08 1/s, S = 0.954385   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Neutron activation: m = 1 g, M = 58.933195 g/mol, theta = 1.0, sigma = 37.18 b, phi = 1e13 n/(cm^2*s), T_half = 166346024.445504 s, t_irr = 30 d	SOLVED	A_eoi = 4.08136e+10 1/s, A_d = 4.08136e+10 1/s, A_sat = 3.79927e+12 1/s, S = 0.0107425
Neutron activation: m = 0.1 g, M = 26.98 g/mol, theta = 1.0, sigma = 0.231 barn, phi = 1e12 1/(cm^2*s), T_half = 134.7 s, t_irr = 10 min, t_d = 1 min	SOLVED	A_eoi = 4.9209e+08 1/s, A_d = 3.61373e+08 1/s, A_sat = 5.1561e+08 1/s, S = 0.954385
Neutron activation: M = 58.933195 g/mol, theta = 1.0, sigma = 37.18 b, phi = 1e13 n/(cm^2*s), T_half = 166346024.445504 s, t_irr = 30 d	NEEDS_INPUT	nuc_activation_activity needs m (mass of the target element). Example: solve('nuc_activation_activity', m='0.001 kg', M=
Neutron activation: m = 1 g, M = 58.933195 g/mol, theta = 1.0, sigma = 37.18 b, phi = 1e13 n/(cm^2*s), T_half = 166346024.445504 s, t_irr = 30 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Activation with target depletion and product burn-up — `nuc_activation_with_burnup`

The product activity when a high flux both depletes the target nuclide and burns up the product by its own absorption, the fraction of target left, and the time and value of the maximum product activity.

$$N_P = \frac{N_{T0}\sigma_T\phi}{\Lambda_P - \Lambda_T}\left(e^{-\Lambda_T t} - e^{-\Lambda_P t}\right),\ \Lambda_T = \sigma_T\phi,\ \Lambda_P = \lambda + \sigma_P\phi$$

input	meaning	condition
`N_T0`	initial number of target atoms	
`sigma_T`	target capture cross section (barns)	
`sigma_P`	product absorption cross section (barns)	
`phi`	neutron flux	
`T_half`	half-life of the product	
`t`	irradiation time	

output	meaning
`A_P`	product activity at t
`f_T`	fraction of target atoms left
`t_max`	irradiation time of maximum product activity
`A_max`	maximum product activity

- **Method:** $dN_T/dt = -sigma_T$ phi N_T, $dN_P/dt = sigma_T$ phi N_T - (lambda + sigma_P phi) N_P: $N_P = N_T0$ sigma_T phi (e^(-a t) $- e^($ $-$ b t))/(b - a) (40 digits)
- **Assumes:** constant flux; removal rates sigma_T phi and lambda + sigma_P phi differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_activation_with_burnup', N_T0=1e+22, sigma_T=3.718e-27,
sigma_P=2e-28, phi=1e+18, T_half=166346024.445504, t=31557600.0)
# -> A_P = 4.30359e+12 1/s, f_T = 0.889291, t_max = 2.47908e+08 s, A_max = 1.4114e+13 1/s
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_activation_with_burnup', N_T0=1e+20, sigma_T=1e-25,
sigma_P=5e-25, phi=5e+18, T_half=3000000.0, t=1000000.0)
# -> A_P = 2.80328e+12 1/s, f_T = 0.606531, t_max = 761002 s, A_max = 2.89131e+12 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Activation with burnup: N_T0 = 1e+22, sigma_T = 37.18 b, sigma_P = 2 b, phi = 1e14 n/(cm^2*s), T_half = 166346024.445504 s, t = 1 y	SOLVED	A_P = 4.30359e+12 1/s, f_T = 0.889291, t_max = 2.47908e+08 s, A_max = 1.4114e+13 1/s
Activation with burnup: N_T0 = 1e+20, sigma_T = 1000 b, sigma_P = 5000 b, phi = 5e14 1/(cm^2*s), T_half = 3000000.0 s, t = 1000000.0 s	SOLVED	A_P = 2.80328e+12 1/s, f_T = 0.606531, t_max = 761002 s, A_max = 2.89131e+12 1/s
Activation with burnup: sigma_T = 37.18 b, sigma_P = 2 b, phi = 1e14 n/(cm^2*s), T_half = 166346024.445504 s, t = 1 y	NEEDS_INPUT	nuc_activation_with_burnup needs N_T0 (initial number of target atoms). Example: solve('nuc_activation_with_burnup', N_T
Activation with burnup: N_T0 = 1e+22, sigma_T = 37.18 b, sigma_P = 2 b, phi = 1e14 n/(cm^2*s), T_half = 166346024.445504 s, t = 1 y, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Irradiation time to reach a fraction of saturation activity — `nuc_irradiation_time_for_saturation`

How long a target must be irradiated (constant flux) for the induced activity to reach a stated fraction of its saturation value, in time and in product half-lives.

$$t_{irr} = -\frac{T_{1/2}}{\ln 2}\ln(1-f)$$

input	meaning	condition
`T_half`	half-life of the product	
`f`	fraction of the saturation activity $(0 < f < 1)$	

output	meaning
`t_irr`	irradiation time
`n_half`	irradiation time in half-lives

- **Method:** 1 - e^(-lambda t) = f -> $t = -ln$(1 - f)/lambda
- **Assumes:** constant flux; $0 < f < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_irradiation_time_for_saturation', T_half=54000.0, f=0.9)
# -> t_irr = 179384 s, n_half = 3.32193   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_irradiation_time_for_saturation', T_half=134.7, f=0.5)
# -> t_irr = 134.7 s, n_half = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Irradiation time for saturation: T_half = 15 h, f = 90%	SOLVED	t_irr = 179384 s, n_half = 3.32193
Irradiation time for saturation: T_half = 134.7 s, f = 0.5	SOLVED	t_irr = 134.7 s, n_half = 1
Irradiation time for saturation: f = 90%	NEEDS_INPUT	nuc_irradiation_time_for_saturation needs T_half (half-life of the product). Example: solve('nuc_irradiation_time_for_sa
Irradiation time for saturation: T_half = 15 h, f = 90%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Half-life from a measured decay curve (weighted log-linear fit) — `nuc_decay_curve_fit`

The decay constant, half-life, initial count and the standard error of the half-life from counts measured at several times (equal counting intervals), by a Poisson-weighted least-squares fit of ln C against t.

$$\min \sum_i C_i(\ln C_i - \ln C_0 + \lambda t_i)^2$$

input	meaning	condition
`t`	measurement times, in the unit t_unit	
`C`	counts recorded at those times (equal counting intervals)	
`t_unit`	unit of the times in the list: s, min, h, d or y	

output	meaning
`lambda_`	fitted decay constant
`T_half`	fitted half-life
`C0`	fitted count at $t = 0$
`sigma_T`	standard error of the half-life (Poisson weights)

- **Method:** weighted least squares of ln $C_i = ln\ C_0 - \lambda$ t_i with weights C_i; sigma_lambda^2 = S/(S Stt - St^2)
- **Assumes:** a single radionuclide; counts positive and decreasing overall; weights $w_i = C_i$ (Var ln C = 1/C)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_decay_curve_fit', t=[0.0, 1.0, 2.0, 3.0, 4.0], C=[10000.0,
7100.0, 5000.0, 3540.0, 2500.0], t_unit='h')
# -> lambda_ = 9.62845e-05 1/s, T_half = 7198.95 s, C0 = 10012.6, sigma_T = 94.4047 s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_decay_curve_fit', t=[0.0, 10.0, 20.0, 30.0], C=[5120.0,
2560.0, 1280.0, 640.0], t_unit='min')
# -> lambda_ = 0.00115525 1/s, T_half = 600 s, C0 = 5120, sigma_T = 9.51438 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay curve fit: t = [0.0, 1.0, 2.0, 3.0, 4.0], C = [10000.0, 7100.0, 5000.0, 3540.0, 2500.0], t_unit = h	SOLVED	lambda_ = 9.62845e-05 1/s, T_half = 7198.95 s, C0 = 10012.6, sigma_T = 94.4047 s
Decay curve fit: t = [0.0, 10.0, 20.0, 30.0], C = [5120.0, 2560.0, 1280.0, 640.0], t_unit = min	SOLVED	lambda_ = 0.00115525 1/s, T_half = 600 s, C0 = 5120, sigma_T = 9.51438 s
Decay curve fit: C = [10000.0, 7100.0, 5000.0, 3540.0, 2500.0], t_unit = h	NEEDS_INPUT	nuc_decay_curve_fit needs t (measurement times, in the unit t_unit). Example: solve('nuc_decay_curve_fit', t=[0.0, 1.0,
Decay curve fit: t = [0.0, 1.0, 2.0, 3.0, 4.0], C = [10000.0, 7100.0, 5000.0, 3540.0, 2500.0], t_unit = h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Decay curve fit: t = [0, 1, 2], C = [100, 200, 400], t_unit = h	INVALID_INPUT	Half-life from a measured decay curve (weighted log-linear fit): the counts do not decrease: no decay constant can be fi

question	outcome	result
Decay curve fit: t = [0, 1, 2, 3, 4] h, C = [10000, 7100, 5000, 3540, 2500]	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Two-component decay curve: amplitudes of two known half-lives —

`nuc_two_component_decay_fit`

Resolve a decay curve that is the sum of two radionuclides of known half-lives into the initial count of each component (Poisson-weighted linear least squares).

$$\min \sum_i \frac{\left(C_i - C_1 e^{-\lambda_1 t_i} - C_2 e^{-\lambda_2 t_i}\right)^2}{C_i}$$

input	meaning	condition
`t`	measurement times, in the unit t_unit	
`C`	counts recorded at those times (equal intervals)	
`T1`	half-life of component 1	
`T2`	half-life of component 2	
`t_unit`	unit of the times in the list: s, min, h, d or y	

output	meaning
`C1`	component 1 count at $t = 0$
`C2`	component 2 count at $t = 0$
`frac1`	fraction of the initial count due to component 1

- **Method:** weighted linear least squares of $C_i = C_1$ e^(-lambda1 t_i) + C2 e^(-lambda2 t_i) (2x2 normal equations)
- **Assumes:** two components with known, different half-lives; weights $1/C_i$ (Poisson)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_two_component_decay_fit', t=[0.0, 1.0, 2.0, 4.0, 8.0, 16.0],
C=[11000.0, 6000.0, 3500.0, 1625.0, 1062.0, 844.0], T1=3600.0, T2=36000.0, t_unit='h')
# -> C1 = 8973.7, C2 = 1696.92, frac1 = 0.840973   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_two_component_decay_fit', t=[0.0, 5.0, 10.0, 20.0, 40.0], C=
[2000.0, 1150.0, 800.0, 560.0, 330.0], T1=300.0, T2=3000.0, t_unit='min')
# -> C1 = 1364.46, C2 = 568.417, frac1 = 0.705922   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-component decay fit: t = [0.0, 1.0, 2.0, 4.0, 8.0, 16.0], C = [11000.0, 6000.0, 3500.0, 1625.0, 1062.0, 844.0], T1 = 1 h, T2 = 10 h, t_unit = h	SOLVED	C1 = 8973.7, C2 = 1696.92, frac1 = 0.840973

question	outcome	result
Two-component decay fit: t = [0.0, 5.0, 10.0, 20.0, 40.0], C = [2000.0, 1150.0, 800.0, 560.0, 330.0], T1 = 5 min, T2 = 50 min, t_unit = min	SOLVED	C1 = 1364.46, C2 = 568.417, frac1 = 0.705922
Two-component decay fit: C = [11000.0, 6000.0, 3500.0, 1625.0, 1062.0, 844.0], T1 = 1 h, T2 = 10 h, t_unit = h	NEEDS_INPUT	nuc_two_component_decay_fit needs t (measurement times, in the unit t_unit). Example: solve('nuc_two_component_decay_fit
Two-component decay fit: t = [0.0, 1.0, 2.0, 4.0, 8.0, 16.0], C = [11000.0, 6000.0, 3500.0, 1625.0, 1062.0, 844.0], T1 = 1 h, T2 = 10 h, t_unit = h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Decay heat after shutdown (Way-Wigner) — `nuc_decay_heat_power`

The fission-product decay power a time t after shutdown of a reactor that ran at constant power for an operating time T, by the Way-Wigner correlation.

$$\frac{P_d}{P_0} = 0.0622\left[t^{-0.2} - (t+T)^{-0.2}\right]$$

input	meaning	condition
`P0`	thermal power before shutdown	
`T_op`	operating time at that power	
`t`	time after shutdown (10 s to about 1e8 s)	

output	meaning
`P_d`	decay power
`ratio`	decay power / operating power

- **Method:** $P/P_0 = 0.0622\ [t^{-0.2} - (t+T)^{-0.2}]$, t and T in seconds (Way-Wigner)
- **Assumes:** constant power before shutdown; 10 $s \le t \le 1e8$ s
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_decay_heat_power', P0=3000000000.0, T_op=31557600.0,
t=3600.0)
# -> P_d = 3.03755e+07 W, ratio = 0.0101252   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_decay_heat_power', P0=100000000.0, T_op=2592000.0, t=86400.0)
# -> P_d = 318187 W, ratio = 0.00318187   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay heat: P0 = 3000 MW, T_op = 1 y, t = 1 h	SOLVED	P_d = 3.03755e+07 W, ratio = 0.0101252

question	outcome	result
Decay heat: P0 = 100 MW, T_op = 30 d, t = 1 d	SOLVED	P_d = 318187 W, ratio = 0.00318187
Decay heat: T_op = 1 y, t = 1 h	NEEDS_INPUT	nuc_decay_heat_power needs P0 (thermal power before shutdown). Example: solve('nuc_decay_heat_power', P0='3e+09 W', T_op
Decay heat: P0 = 3000 MW, T_op = 1 y, t = 1 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Decay heat: P0 = 3000 MW, T_op = 1 y, t = 1 s	INVALID_INPUT	Decay heat after shutdown (Way-Wigner): the Way-Wigner correlation holds from 10 s to about 1e8 s after shutdown.

Decay heat energy released between two times after shutdown — `nuc_decay_heat_energy`

The total fission-product decay energy released between two times after shutdown (the Way-Wigner power integrated), and the mean decay power over that interval.

$$E = 0.0622\,P_0\left[\frac{t^{0.8} - (t+T)^{0.8}}{0.8}\right]_{t_1}^{t_2}$$

input	meaning	condition
`P0`	thermal power before shutdown	
`T_op`	operating time at that power	
`t1`	start of the interval after shutdown	
`t2`	end of the interval after shutdown	

output	meaning
`E`	decay energy released
`E_MWh`	the same energy in MWh
`P_mean`	mean decay power over the interval

- **Method:** $E = 0.0622$ P0 [$(t^{0.8} - (t+T)^{0.8})/0.8$] from t1 to t2
- **Assumes:** 10 $s \le t_1 < t_2 \le 1e8$ s; constant power before shutdown
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_decay_heat_energy', P0=3000000000.0, T_op=31557600.0,
t1=10.0, t2=86400.0)
# -> E = 1.56376e+12 J, E_MWh = 434.377, P_mean = 1.81011e+07 W   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_decay_heat_energy', P0=100000000.0, T_op=2592000.0,
t1=3600.0, t2=604800.0)
# -> E = 1.31822e+11 J, E_MWh = 36.6174, P_mean = 219266 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay heat energy: P0 = 3000 MW, T_op = 1 y, t1 = 10.0 s, t2 = 1 d	SOLVED	E = 1.56376e+12 J, E_MWh = 434.377, P_mean = 1.81011e+07 W
Decay heat energy: P0 = 100 MW, T_op = 30 d, t1 = 1 h, t2 = 7 d	SOLVED	E = 1.31822e+11 J, E_MWh = 36.6174, P_mean = 219266 W
Decay heat energy: T_op = 1 y, t1 = 10.0 s, t2 = 1 d	NEEDS_INPUT	nuc_decay_heat_energy needs P0 (thermal power before shutdown). Example: solve('nuc_decay_heat_energy', P0='3e+09 W', T_
Decay heat energy: P0 = 3000 MW, T_op = 1 y, t1 = 10.0 s, t2 = 1 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Committed equivalent dose to an organ from an intake (retention compartments) — `nuc_committed_dose_retention`

The number of decays in an organ over the commitment period after an intake retained in exponential compartments, and the committed equivalent dose from the energy absorbed per decay, the radiation weighting factor and the organ mass.

$$H_T(\tau) = \frac{w_R E}{m}\, I \sum_i f_i \frac{1 - e^{-(\lambda_p + \lambda_{b,i})\tau}}{\lambda_p + \lambda_{b,i}}$$

input	meaning	condition
`I`	activity taken into the organ (intake reaching it)	
`f`	fractions of the intake retained in each compartment (sum <= 1)	
`Tb`	biological half-lives of the compartments, in the unit t_unit	
`t_unit`	unit of the times in the list: s, min, h, d or y	
`T_p`	physical half-life	
`E`	energy absorbed in the organ per decay (e.g. MeV)	
`w_R`	radiation weighting factor (1 for photons and electrons, 20 for alpha particles)	
`m`	organ mass	
`tau`	commitment period (50 y for adults)	

output	meaning
`U`	number of decays in the organ during tau
`H`	committed equivalent dose (Sv)
`H_mSv`	committed equivalent dose in mSv

- **Method:** $U = I$ sum_i f_i (1 - e^(-k_i tau))/k_i with $k_i = lambda_p + ln\,2/Tb_i$; $H = U$ E w_R / m
- **Assumes:** instantaneous intake; retention R(t) = sum f_i e^(-lambda_b,i t); all energy per decay absorbed in the organ as stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_committed_dose_retention', I=1000000.0, f=[0.3], Tb=[80.0],
t_unit='d', T_p=692988.48, E=2.8839179412000003e-14, w_R=1.0, m=0.02, tau=1577880000.0)
# -> U = 2.72601e+11, H = 0.393079 J/kg, H_mSv = 393.079   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_committed_dose_retention', I=1000.0, f=[0.6, 0.4], Tb=[10.0,
100.0], t_unit='y', T_p=760837487788.8, E=8.331318496800001e-13, w_R=20.0, m=5.0, tau=1577880000.0)
# -> U = 7.97567e+11, H = 2.65791 J/kg, H_mSv = 2657.91   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Committed dose: I = 1 MBq, f = [0.3], Tb = [80.0], t_unit = d, T_p = 8.0207 d, E = 0.18 MeV, w_R = 1.0, m = 20 g, tau = 50 y	SOLVED	U = 2.72601e+11, H = 0.393079 J/kg, H_mSv = 393.079
Committed dose: I = 1 kBq, f = [0.6, 0.4], Tb = [10.0, 100.0], t_unit = y, T_p = 760837487788.8 s, E = 5.2 MeV, w_R = 20.0, m = 5 kg, tau = 50 y	SOLVED	U = 7.97567e+11, H = 2.65791 J/kg, H_mSv = 2657.91
Committed dose: f = [0.3], Tb = [80.0], t_unit = d, T_p = 8.0207 d, E = 0.18 MeV, w_R = 1.0, m = 20 g, tau = 50 y	NEEDS_INPUT	nuc_committed_dose_retention needs I (activity taken into the organ (intake reaching it)). Example: solve('nuc_committed
Committed dose: I = 1 MBq, f = [0.3], Tb = [80.0], t_unit = d, T_p = 8.0207 d, E = 0.18 MeV, w_R = 1.0, m = 20 g, tau = 50 y, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Dose rate from a shielded point gamma source with buildup — `nuc_point_source_dose_rate`

The dose rate at a distance from a point gamma source of stated dose-rate constant, behind a slab shield, with the buildup factor in the stated form (none, linear or Taylor), plus the buildup factor and the transmission.

$$\dot{H} = \frac{\Gamma A}{r^2}\, B(\mu x)\, e^{-\mu x}$$

input	meaning	condition
`Gamma`	dose-rate constant of the nuclide (e.g. uSv*m^2/(MBq*h))	
`A`	source activity	
`r`	source-to-point distance	
`mu`	linear attenuation coefficient of the shield	
`x`	shield thickness (0 for none)	
`buildup`	buildup form: none $(B = 1)$, linear (B = 1 + mu x) or taylor	
`A1`	Taylor buildup parameter A1 (only with buildup = taylor)	
`alpha1`	Taylor buildup parameter alpha1 (only with buildup = taylor)	
`alpha2`	Taylor buildup parameter alpha2 (only with buildup = taylor)	

output	meaning
`H_dot`	dose rate (same quantity as Gamma: Sv/s, Gy/s)

output	meaning
`H_uSv_h`	dose rate in uSv/h (or uGy/h)
`B`	buildup factor
`T`	transmission B e^(-mu x)

- **Method:** $H = Gamma$ A B(mu x) e^(-mu x) / r^2; $B = 1, 1 + mu$ x, or A1 e^(-alpha1 mu x) + (1 - A1) e^(-alpha2 mu x)
- **Assumes:** point isotropic source; shield between source and point $(x < r)$; Taylor parameters only with the Taylor form
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_point_source_dose_rate', Gamma=9.750000000000001e-17,
A=370000000000.0, r=2.0, mu=6.56, x=0.1, buildup='linear')
# -> H_dot = 7.75014e-06 J/(kg*s), H_uSv_h = 27900.5, B = 1.656, T = 0.859336   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_point_source_dose_rate', Gamma=9.750000000000001e-17,
A=370000000000.0, r=2.0, mu=6.56, x=0.1, buildup='taylor', A1=10.0, alpha1=-0.07, alpha2=0.06)
# -> H_dot = 8.50487e-06 J/(kg*s), H_uSv_h = 30617.5, B = 1.81727, T = 0.943021   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_point_source_dose_rate', Gamma=2.575e-17, A=1000000000.0,
r=1.0, mu=0.0, x=0.0, buildup='none')
# -> H_dot = 2.575e-08 J/(kg*s), H_uSv_h = 92.7, B = 1, T = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Point source dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm, buildup = linear	SOLVED	H_dot = 7.75014e-06 J/(kg*s), H_uSv_h = 27900.5, B = 1.656, T = 0.859336
Point source dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm, buildup = taylor, A1 = 10.0, alpha1 = -0.07, alpha2 = 0.06	SOLVED	H_dot = 8.50487e-06 J/(kg*s), H_uSv_h = 30617.5, B = 1.81727, T = 0.943021
Point source dose rate: Gamma = 0.0927 uSv*m^2/(MBq*h), A = 1 GBq, r = 1 m, mu = 0 1/m, x = 0 m, buildup = none	SOLVED	H_dot = 2.575e-08 J/(kg*s), H_uSv_h = 92.7, B = 1, T = 1
Point source dose rate: A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm, buildup = linear	NEEDS_INPUT	nuc_point_source_dose_rate needs Gamma (dose-rate constant of the nuclide (e.g. uSv*m^2/(MBq*h))). Example: solve('nuc_p
Point source dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm, buildup = linear, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Point source dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm, buildup = linear, A1 = 10	INVALID_INPUT	Dose rate from a shielded point gamma source with buildup: Taylor parameters are given but the buildup form is not taylo

question	outcome	result
Point source dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.0656 1/cm, x = 10 cm	NEEDS_INPUT	nuc_point_source_dose_rate needs buildup (buildup form: none (B = 1), linear (B = 1 + mu x) or taylor). Example: solve('

Shield thickness for a target dose rate (point source, buildup) —

`nuc_shield_thickness_for_dose_rate`

The slab thickness that brings the dose rate from a point gamma source down to a target value, with no buildup or the linear buildup form $B = 1 + mu$ x (the root of the attenuation equation).

$$\frac{\Gamma A}{r^2} B(\mu x) e^{-\mu x} = \dot{H}_{target}$$

input	meaning	condition
`Gamma`	dose-rate constant of the nuclide	
`A`	source activity	
`r`	source-to-point distance	
`mu`	linear attenuation coefficient of the shield	
`H_target`	target dose rate (e.g. uSv/h)	
`buildup`	buildup form: none $(B = 1)$ or linear (B = 1 + mu x)	

output	meaning
`x`	required shield thickness
`n_mfp`	thickness in mean free paths
`B`	buildup factor at that thickness

- **Method:** solve B(mu x) e^(-mu x) = H_target r^2/(Gamma A): $x = ln$(H0/H)/mu (no buildup) or $(1 + b)\, e^{-b} = q$ by bisection
- **Assumes:** target below the unshielded dose rate; the shield fits between source and point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_shield_thickness_for_dose_rate', Gamma=9.750000000000001e-17,
A=370000000000.0, r=2.0, mu=65.6, H_target=6.944444444444445e-10, buildup='linear')
# -> x = 0.183532 m, n_mfp = 12.0397, B = 13.0397   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_shield_thickness_for_dose_rate', Gamma=9.750000000000001e-17,
A=370000000000.0, r=2.0, mu=65.6, H_target=6.944444444444445e-10, buildup='none')
# -> x = 0.144386 m, n_mfp = 9.4717, B = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shield thickness for dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2	SOLVED	x = 0.183532 m, n_mfp = 12.0397, B = 13.0397

question	outcome	result
m, mu = 0.656 1/cm, H_target = 2.5 uSv/h, buildup = linear		
Shield thickness for dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.656 1/cm, H_target = 2.5 uSv/h, buildup = none	SOLVED	x = 0.144386 m, n_mfp = 9.4717, B = 1
Shield thickness for dose rate: A = 10 Ci, r = 2 m, mu = 0.656 1/cm, H_target = 2.5 uSv/h, buildup = linear	NEEDS_INPUT	nuc_shield_thickness_for_dose_rate needs Gamma (dose-rate constant of the nuclide). Example: solve('nuc_shield_thickness
Shield thickness for dose rate: Gamma = 0.351 uSv*m^2/(MBq*h), A = 10 Ci, r = 2 m, mu = 0.656 1/cm, H_target = 2.5 uSv/h, buildup = linear, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Shield thickness: Gamma = 0.351 uSv*m^2/(MBq*h), A = 1 kBq, r = 2 m, mu = 0.656 1/cm, H_target = 2.5 uSv/h, buildup = none	INVALID_INPUT	Shield thickness for a target dose rate (point source, buildup): the unshielded dose rate is already below the target: n

Narrow-beam transmission through a multilayer shield — `nuc_multilayer_transmission`

The uncollided (narrow-beam) transmission through several shield layers, the total optical thickness, its value in half-value layers, the attenuation factor and (optionally) the dose rate behind the shield.

$$T = \exp\Big(-\sum_i \mu_i x_i\Big)$$

input	meaning	condition
`mu`	linear attenuation coefficient of each layer, in $1/cm$	
`x`	thickness of each layer, in cm	
`H0`	uncollided dose rate in front of the shield (optional)	

output	meaning
`T`	uncollided transmission
`tau`	total optical thickness sum mu_i x_i
`n_hvl`	thickness in half-value layers
`attenuation`	attenuation factor $1/T$
`H`	uncollided dose rate behind the shield (when H0 is given)

- **Method:** $T = exp$(-sum mu_i x_i)
- **Assumes:** narrow beam (no buildup); layers perpendicular to the beam
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_multilayer_transmission', mu=[0.656, 0.149], x=[5.0, 30.0])
# -> T = 0.000430743, tau = 7.75, n_hvl = 11.1809, attenuation = 2321.57   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_multilayer_transmission', mu=[1.2, 0.2, 0.07], x=[2.0, 10.0,
20.0], H0=2.7777777777777776e-07)
# -> T = 0.00302755, tau = 5.8, n_hvl = 8.36763, attenuation = 330.3, H = 8.40987e-10 J/(kg*s)
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multilayer transmission: mu = [0.656, 0.149], x = [5.0, 30.0]	SOLVED	T = 0.000430743, tau = 7.75, n_hvl = 11.1809, attenuation = 2321.57
Multilayer transmission: mu = [1.2, 0.2, 0.07], x = [2.0, 10.0, 20.0], H0 = 1 mSv/h	SOLVED	T = 0.00302755, tau = 5.8, n_hvl = 8.36763, attenuation = 330.3, H = 8.40987e-10 J/(kg*s)
Multilayer transmission: x = [5.0, 30.0]	NEEDS_INPUT	nuc_multilayer_transmission needs mu (linear attenuation coefficient of each layer, in 1/cm). Example: solve('nuc_multil
Multilayer transmission: mu = [0.656, 0.149], x = [5.0, 30.0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Multilayer transmission: mu = [0.656, 0.149] 1/cm, x = [5, 30] cm	SOLVED	T = 0.000430743, tau = 7.75, n_hvl = 11.1809, attenuation = 2321.57
Multilayer transmission: mu = [0.656, 0.149] 1/m, x = [5, 30] cm	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Multilayer transmission: mu = [0.656, 0.149], x = [5, 30, 4]	INVALID_INPUT	Narrow-beam transmission through a multilayer shield: give one attenuation coefficient per layer thickness (1 to 10 laye

Uncollided flux from a shielded line source (Sievert integral) — `nuc_line_source_flux`

The uncollided photon flux at a point at perpendicular distance h from a straight line source (a pipe, a wire) running from y1 to y2 along the line, behind a slab shield parallel to the line, with the unshielded flux and the transmission.

$$\phi = \frac{S_L}{4\pi h}[F(\theta_2, \mu t) - F(\theta_1, \mu t)],\ F(\theta, b) = \int_0^{\theta} e^{-b\sec u}\, du$$

input	meaning	condition
`S_L`	source strength per unit length (photons per second per metre)	
`h`	perpendicular distance from the line to the point	
`y1`	position of one end along the line, measured from the foot of the perpendicular (may be negative)	
`y2`	position of the other end $(y_2 > y_1)$	
`mu`	linear attenuation coefficient of the shield	
`t`	shield thickness (0 for none)	

output	meaning
`phi`	uncollided flux at the point
`phi0`	flux without the shield

output	meaning
`T`	transmission $\phi/phi0$

- **Method:** phi = S_L/(4 pi h) [F(theta2, mu t) - F(theta1, mu t)], F(theta, b) = int_0^theta e^(-b sec u) du (Sievert integral), theta_i = atan(y_i/h)
- **Assumes:** isotropic line source in vacuum; slab shield parallel to the line $(t < h)$; no buildup
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_line_source_flux', S_L=100000000.0, h=1.0, y1=-2.0, y2=2.0,
mu=10.0, t=0.1)
# -> phi = 5.01271e+06 1/(m^2*s), phi0 = 1.76208e+07 1/(m^2*s), T = 0.284476   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_line_source_flux', S_L=5000000000.0, h=2.0, y1=0.0, y2=3.0,
mu=0.0, t=0.0)
# -> phi = 1.95521e+08 1/(m^2*s), phi0 = 1.95521e+08 1/(m^2*s), T = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line source flux: S_L = 1e8 1/(m*s), h = 1 m, y1 = -2 m, y2 = 2 m, mu = 0.1 1/cm, t = 10 cm	SOLVED	phi = 5.01271e+06 1/(m^2*s), phi0 = 1.76208e+07 1/(m^2*s), T = 0.284476
Line source flux: S_L = 5e9 1/(m*s), h = 2 m, y1 = 0 m, y2 = 3 m, mu = 0 1/m, t = 0 m	SOLVED	phi = 1.95521e+08 1/(m^2*s), phi0 = 1.95521e+08 1/(m^2*s), T = 1
Line source flux: h = 1 m, y1 = -2 m, y2 = 2 m, mu = 0.1 1/cm, t = 10 cm	NEEDS_INPUT	nuc_line_source_flux needs S_L (source strength per unit length (photons per second per metre)). Example: solve('nuc_lin
Line source flux: S_L = 1e8 1/(m*s), h = 1 m, y1 = -2 m, y2 = 2 m, mu = 0.1 1/cm, t = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Uncollided flux on the axis of a shielded disk source — `nuc_disk_source_flux`

The uncollided flux on the axis of a uniform disk source (a contaminated floor patch, a tank bottom) at distance h, behind a slab shield parallel to the disk, with the unshielded flux and the transmission.

$$\phi = \frac{S_A}{2}[E_1(\mu t) - E_1(\mu t \sec\theta)]$$

input	meaning	condition
`S_A`	source strength per unit area (photons per second per square metre)	
`R`	disk radius	
`h`	distance from the disk on its axis	
`mu`	linear attenuation coefficient of the shield	
`t`	shield thickness (0 for none)	

output	meaning
`phi`	uncollided flux on the axis
`phi0`	flux without the shield
`T`	transmission $\phi/phi0$

- **Method:** $\phi = (S_A/2)$ [E1(b) - E1(b sec theta)], sec $\theta = \frac{\sqrt{h^2+R^2}}{h}$, $b = mu$ t; unshielded $(S_A/4)$ ln(1 + R^2/h^2)
- **Assumes:** uniform isotropic disk source; slab shield parallel to the disk $(t < h)$; no buildup
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_disk_source_flux', S_A=10000000.0, R=1.0, h=0.5, mu=10.0,
t=0.05)
# -> phi = 1.89574e+06 1/(m^2*s), phi0 = 4.02359e+06 1/(m^2*s), T = 0.471155   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_disk_source_flux', S_A=1000000.0, R=2.0, h=1.0, mu=0.0,
t=0.0)
# -> phi = 402359 1/(m^2*s), phi0 = 402359 1/(m^2*s), T = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Disk source flux: S_A = 1e7 1/(m^2*s), R = 1 m, h = 50 cm, mu = 0.1 1/cm, t = 5 cm	SOLVED	phi = 1.89574e+06 1/(m^2*s), phi0 = 4.02359e+06 1/(m^2*s), T = 0.471155
Disk source flux: S_A = 1e6 1/(m^2*s), R = 2 m, h = 1 m, mu = 0 1/m, t = 0 m	SOLVED	phi = 402359 1/(m^2*s), phi0 = 402359 1/(m^2*s), T = 1
Disk source flux: R = 1 m, h = 50 cm, mu = 0.1 1/cm, t = 5 cm	NEEDS_INPUT	nuc_disk_source_flux needs S_A (source strength per unit area (photons per second per square metre)). Example: solve('nu
Disk source flux: S_A = 1e7 1/(m^2*s), R = 1 m, h = 50 cm, mu = 0.1 1/cm, t = 5 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Dose received passing a point source — `nuc_transit_dose`

The dose received by a person or detector moving at constant speed along a straight path past an unshielded point source, from the closest distance, the speed and the path length (centred on the closest approach).

$$D = \frac{\Gamma A}{d\,v}\,2\arctan\frac{L}{2d}$$

input	meaning	condition
`Gamma`	dose-rate constant of the nuclide	
`A`	source activity	
`d`	closest distance to the source	
`v`	speed along the path	

input	meaning	condition
`L`	length of the path, centred on the point of closest approach	

output	meaning
`D`	dose received (Sv or Gy, as Gamma)
`D_uSv`	dose in uSv (or uGy)
`frac`	fraction of the dose of an infinitely long path

- **Method:** D = (Gamma A/(d v)) 2 atan(L/(2 d)); infinite path Gamma A pi/(d v)
- **Assumes:** unshielded point source; straight path at constant speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_transit_dose', Gamma=9.750000000000001e-17,
A=3700000000000.0, d=2.0, v=1.0, L=20.0)
# -> D = 0.000495454 J/kg, D_uSv = 495.454, frac = 0.874334   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_transit_dose', Gamma=2.575e-17, A=1000000000000.0, d=5.0,
v=10.0, L=1000.0)
# -> D = 1.60762e-06 J/kg, D_uSv = 1.60762, frac = 0.993634   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transit dose: Gamma = 0.351 uSv*m^2/(MBq*h), A = 100 Ci, d = 2 m, v = 1 m/s, L = 20 m	SOLVED	D = 0.000495454 J/kg, D_uSv = 495.454, frac = 0.874334
Transit dose: Gamma = 0.0927 uSv*m^2/(MBq*h), A = 1 TBq, d = 5 m, v = 10 m/s, L = 1 km	SOLVED	D = 1.60762e-06 J/kg, D_uSv = 1.60762, frac = 0.993634
Transit dose: A = 100 Ci, d = 2 m, v = 1 m/s, L = 20 m	NEEDS_INPUT	nuc_transit_dose needs Gamma (dose-rate constant of the nuclide). Example: solve('nuc_transit_dose', Gamma='9.75e-17 J*m
Transit dose: Gamma = 0.351 uSv*m^2/(MBq*h), A = 100 Ci, d = 2 m, v = 1 m/s, L = 20 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Dose accumulated near a decaying source — `nuc_accumulated_dose_decaying_source`

The dose accumulated over an exposure time when the dose rate falls with the source's half-life, the dose if the exposure lasted until complete decay, and the mean dose rate.

$$D = \dot{H}_0 \, e^{-\lambda t_d} \, \frac{1 - e^{-\lambda t}}{\lambda}$$

input	meaning	condition
`H0`	dose rate at the reference time	
`T_half`	half-life of the source	

input	meaning	condition
`t`	exposure time	
`td`	delay from the reference time to the start (0 when omitted)	

output	meaning
`D`	dose accumulated
`D_mSv`	dose in mSv (or mGy)
`D_inf`	dose until complete decay
`H_mean`	mean dose rate during the exposure

- **Method:** $D = H_0$ e^(-lambda td) (1 - e^(-lambda t))/lambda
- **Assumes:** the dose rate follows one half-life
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_accumulated_dose_decaying_source', H0=2.7777777777777776e-07,
T_half=6586.2, t=7200.0)
# -> D = 0.00140226 J/kg, D_mSv = 1.40226, D_inf = 0.00263941 J/kg, H_mean = 1.94758e-07 J/(kg*s)
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_accumulated_dose_decaying_source', H0=1.388888888888889e-08,
T_half=692988.48, t=604800.0, td=86400.0)
# -> D = 0.00578081 J/kg, D_mSv = 5.78081, D_inf = 0.0127361 J/kg, H_mean = 9.55822e-09 J/(kg*s)
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Accumulated dose from a decaying source: H0 = 1 mSv/h, T_half = 6586.2 s, t = 2 h	SOLVED	D = 0.00140226 J/kg, D_mSv = 1.40226, D_inf = 0.00263941 J/kg, H_mean = 1.94758e-07 J/(kg*s)
Accumulated dose from a decaying source: H0 = 50 uSv/h, T_half = 692988.48 s, t = 7 d, td = 1 d	SOLVED	D = 0.00578081 J/kg, D_mSv = 5.78081, D_inf = 0.0127361 J/kg, H_mean = 9.55822e-09 J/(kg*s)
Accumulated dose from a decaying source: T_half = 6586.2 s, t = 2 h	NEEDS_INPUT	nuc_accumulated_dose_decaying_source needs H0 (dose rate at the reference time). Example: solve('nuc_accumulated_dose_de
Accumulated dose from a decaying source: H0 = 1 mSv/h, T_half = 6586.2 s, t = 2 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Stay time to a dose limit near a decaying source — `nuc_stay_time_decaying_source`

How long one can stay at a place where the dose rate decays with the source's half-life before receiving a stated dose, compared with the stay time if the rate were constant.

$$t = -\frac{1}{\lambda}\ln\left(1 - \frac{\lambda D_{lim}}{\dot{H}_0}\right)$$

input	meaning	condition
`H0`	dose rate at the start of the stay	
`T_half`	half-life of the source	
`D_lim`	dose limit for the stay	

output	meaning
`t_stay`	allowed stay time
`frac`	dose limit as a fraction of the dose until complete decay

- **Method:** H0 (1 - e^(-lambda t))/lambda = D_lim -> $t = -ln(1 - \lambda$ D_lim/H0)/lambda
- **Assumes:** D_lim < H0 T_half/ln 2 (otherwise the limit is never reached)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_stay_time_decaying_source', H0=5.555555555555555e-07,
T_half=6586.2, D_lim=0.001)
# -> t_stay = 1995.63 s, frac = 0.189436   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_stay_time_decaying_source', H0=2.777777777777778e-08,
T_half=21654.0, D_lim=0.0002)
# -> t_stay = 8184.26 s, frac = 0.230473   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stay time: H0 = 2 mSv/h, T_half = 6586.2 s, D_lim = 1 mSv	SOLVED	t_stay = 1995.63 s, frac = 0.189436
Stay time: H0 = 100 uSv/h, T_half = 21654.0 s, D_lim = 200 uSv	SOLVED	t_stay = 8184.26 s, frac = 0.230473
Stay time: T_half = 6586.2 s, D_lim = 1 mSv	NEEDS_INPUT	nuc_stay_time_decaying_source needs H0 (dose rate at the start of the stay). Example: solve('nuc_stay_time_decaying_sour
Stay time: H0 = 2 mSv/h, T_half = 6586.2 s, D_lim = 1 mSv, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Stay time: H0 = 2 mSv/h, T_half = 6586.2 s, D_lim = 100 mSv	INVALID_INPUT	Stay time to a dose limit near a decaying source: the dose limit is never reached: the source decays first (D_lim >= H0

Klein-Nishina Compton cross sections per electron — `nuc_klein_nishina`

The total Compton (Klein-Nishina) cross section per electron for a photon energy, the energy-transfer cross section and the mean kinetic energy given to the recoil electron.

$$\sigma = 2\pi r_e^2 \left\{ \frac{1+k}{k^2} \left[\frac{2(1+k)}{1+2k} - \frac{\ln(1+2k)}{k} \right] + \frac{\ln(1+2k)}{2k} - \frac{1+3k}{(1+2k)^2} \right\}$$

input	meaning	condition
`E`	photon energy (10 keV to 100 MeV)	

output	meaning
`sigma`	Klein-Nishina cross section per electron
`sigma_b`	the same in barns per electron
`sigma_tr`	energy-transfer cross section per electron
`T_mean`	mean kinetic energy of the Compton electron (MeV)

- **Method:** closed Klein-Nishina integrals over the scattering angle, k = E/(m_e c^2); T_mean = E sigma_tr/sigma
- **Assumes:** free electrons at rest (no binding); 10 keV <= E <= 100 MeV
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_klein_nishina', E=1.0606409317080002e-13)
# -> sigma = 2.5614e-29 m^2, sigma_b = 0.25614, sigma_tr = 9.77494e-30 m^2, T_mean = 0.252635 MeV
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_klein_nishina', E=2.0027207925000002e-13)
# -> sigma = 1.88819e-29 m^2, sigma_b = 0.188819, sigma_tr = 8.88542e-30 m^2, T_mean = 0.588222 MeV
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Klein-Nishina cross section: E = 0.662 MeV	SOLVED	sigma = 2.5614e-29 m^2, sigma_b = 0.25614, sigma_tr = 9.77494e-30 m^2, T_mean = 0.252635 MeV
Klein-Nishina cross section: E = 1.25 MeV	SOLVED	sigma = 1.88819e-29 m^2, sigma_b = 0.188819, sigma_tr = 8.88542e-30 m^2, T_mean = 0.588222 MeV
Klein-Nishina cross section:	NEEDS_INPUT	nuc_klein_nishina needs E (photon energy (10 keV to 100 MeV)). Example: solve('nuc_klein_nishina', E='1.06064e-13 J').
Klein-Nishina cross section: E = 0.662 MeV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Klein-Nishina: E = 1 keV	INVALID_INPUT	Klein-Nishina Compton cross sections per electron: the photon energy must lie between 10 keV and 100 MeV.

Compton edge and backscatter peak of a gamma line — `nuc_compton_edge`

Where a gamma line's Compton edge (maximum electron energy) and backscatter peak (energy of a photon scattered through 180 degrees) appear in a detector's spectrum.

$$T_{max} = \frac{2E^2}{m_e c^2 + 2E}, \quad E_{bs} = \frac{E}{1 + 2E/m_e c^2}$$

input	meaning	condition
`E`	gamma-ray energy	

output	meaning
`T_max`	Compton edge: maximum electron recoil energy (MeV)
`E_bs`	backscatter peak: energy of the photon scattered through 180 degrees (MeV)

- **Method:** E_bs = E/(1 + 2E/(m_e c^2)), T_max = E - E_bs
- **Assumes:** free electron at rest
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_compton_edge', E=1.0606409317080002e-13)
# -> T_max = 0.47765 MeV, E_bs = 0.18435 MeV   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_compton_edge', E=2.1340992764880002e-13)
# -> T_max = 1.11762 MeV, E_bs = 0.214378 MeV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compton edge: E = 662 keV	SOLVED	T_max = 0.47765 MeV, E_bs = 0.18435 MeV
Compton edge: E = 1.332 MeV	SOLVED	T_max = 1.11762 MeV, E_bs = 0.214378 MeV
Compton edge:	NEEDS_INPUT	nuc_compton_edge needs E (gamma-ray energy). Example: solve('nuc_compton_edge', E='1.06064e-13 J').
Compton edge: E = 662 keV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Dead-time correction, non-paralyzable detector — `nuc_dead_time_nonparalyzable`

The true event rate from the observed count rate of a non-paralyzable (non-extendable) detector with a stated dead time, the fraction of events lost and the correction factor.

$$n = \frac{m}{1 - m\tau}$$

input	meaning	condition
`m`	observed count rate (cps, cpm)	
`tau`	dead time (e.g. us)	

output	meaning
`n`	true event rate

output	meaning
`loss`	fraction of events lost
`corr`	correction factor n/m

- **Method:** n = m / (1 - m tau)
- **Assumes:** non-paralyzable (non-extendable) dead time; m $\tau < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_dead_time_nonparalyzable', m=50000.0, tau=5e-06)
# -> n = 66666.7 1/s, loss = 0.25, corr = 1.33333   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_dead_time_nonparalyzable', m=16666.666666666668, tau=1e-05)
# -> n = 20000 1/s, loss = 0.166667, corr = 1.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Non-paralyzable dead time: m = 50000 cps, tau = 5 us	SOLVED	n = 66666.7 1/s, loss = 0.25, corr = 1.33333
Non-paralyzable dead time: m = 1e6 cpm, tau = 10 us	SOLVED	n = 20000 1/s, loss = 0.166667, corr = 1.2
Non-paralyzable dead time: tau = 5 us	NEEDS_INPUT	nuc_dead_time_nonparalyzable needs m (observed count rate (cps, cpm)). Example: solve('nuc_dead_time_nonparalyzable', m=
Non-paralyzable dead time: m = 50000 cps, tau = 5 us, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Non-paralyzable dead time: m = 300000 cps, tau = 5 us	INVALID_INPUT	Dead-time correction, non-paralyzable detector: m tau must be below 1 for a non-paralyzable detector.

Dead-time correction, paralyzable detector — `nuc_dead_time_paralyzable`

The true event rate from the observed count rate of a paralyzable (extendable) detector, on the stated branch of the rate curve (below or above the maximum), with the fraction lost and the largest observable count rate.

$$m = n\,e^{-n\tau}$$

input	meaning	condition
`m`	observed count rate	
`tau`	dead time	
`branch`	branch of the rate curve: low (true rate below 1/tau) or high (above it)	

output	meaning
`n`	true event rate

output	meaning
`loss`	fraction of events lost
`m_max`	largest observable count rate 1/(e tau)

- **Method:** $m = n$ e^(-n tau) solved for n on the stated branch (bisection in $x = n$ tau)
- **Assumes:** paralyzable (extendable) dead time; m $\tau \leq 1/e$; the branch is stated (two true rates give the same m)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_dead_time_paralyzable', m=50000.0, tau=5e-06, branch='low')
# -> n = 71480.6 1/s, loss = 0.300509, m_max = 73575.9 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_dead_time_paralyzable', m=50000.0, tau=5e-06, branch='high')
# -> n = 430658 1/s, loss = 0.883899, m_max = 73575.9 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Paralyzable dead time: m = 50000 cps, tau = 5 us, branch = low	SOLVED	n = 71480.6 1/s, loss = 0.300509, m_max = 73575.9 1/s
Paralyzable dead time: m = 50000 cps, tau = 5 us, branch = high	SOLVED	n = 430658 1/s, loss = 0.883899, m_max = 73575.9 1/s
Paralyzable dead time: tau = 5 us, branch = low	NEEDS_INPUT	nuc_dead_time_paralyzable needs m (observed count rate). Example: solve('nuc_dead_time_paralyzable', m='50000 1/s', tau=
Paralyzable dead time: m = 50000 cps, tau = 5 us, branch = low, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Paralyzable dead time: m = 100000 cps, tau = 5 us, branch = low	INVALID_INPUT	Dead-time correction, paralyzable detector: m tau above 1/e is impossible for a paralyzable detector.

Optimal division of counting time between sample and background — `nuc_count_time_split`

How to split a fixed total counting time between the sample (gross) count and the background count to minimise the standard deviation of the net count rate, and that minimum standard deviation.

$$\frac{t_g}{t_b} = \sqrt{\frac{R_g}{R_b}}, \quad \sigma_{net} = \frac{\sqrt{R_g} + \sqrt{R_b}}{\sqrt{T}}$$

input	meaning	condition
`T`	total counting time available	
`Rg`	gross count rate (sample plus background)	
`Rb`	background count rate	

output	meaning
`tg`	time for the gross count
`tb`	time for the background count
`sigma`	minimum standard deviation of the net count rate
`rel`	relative standard deviation of the net count rate

- **Method:** tg/$tb = \sqrt{\frac{Rg}{Rb}}$; $\sigma_{net} = \frac{\sqrt{Rg}+\sqrt{Rb}}{\sqrt{T}}$
- **Assumes:** Poisson counting statistics; $Rg > Rb > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_count_time_split', T=3600.0, Rg=100.0, Rb=25.0)
# -> tg = 2400 s, tb = 1200 s, sigma = 0.25 1/s, rel = 0.00333333   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_count_time_split', T=600.0, Rg=10.0, Rb=0.5)
# -> tg = 490.354 s, tb = 109.646 s, sigma = 0.157967 1/s, rel = 0.0166281   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Counting time split: T = 1 h, Rg = 100 cps, Rb = 25 cps	SOLVED	tg = 2400 s, tb = 1200 s, sigma = 0.25 1/s, rel = 0.00333333
Counting time split: T = 10 min, Rg = 600 cpm, Rb = 30 cpm	SOLVED	tg = 490.354 s, tb = 109.646 s, sigma = 0.157967 1/s, rel = 0.0166281
Counting time split: Rg = 100 cps, Rb = 25 cps	NEEDS_INPUT	nuc_count_time_split needs T (total counting time available). Example: solve('nuc_count_time_split', T='3600 s', Rg='100
Counting time split: T = 1 h, Rg = 100 cps, Rb = 25 cps, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Counting time for a target precision of the net count rate — `nuc_count_time_for_precision`

The total counting time (optimally split between sample and background) needed for the net count rate to have a stated relative standard deviation, and the two counting times.

$$T = \frac{(\sqrt{R_g} + \sqrt{R_b})^2}{(R_g - R_b)^2\, r^2}$$

input	meaning	condition
`Rg`	gross count rate	
`Rb`	background count rate	
`r`	target relative standard deviation of the net count rate (e.g. 1%)	

output	meaning
`T`	total counting time

output	meaning
`tg`	gross counting time
`tb`	background counting time

- **Method:** T = (sqrt(Rg) + sqrt(Rb))^2 / ($(Rg - Rb)^2$ r^2)
- **Assumes:** Poisson statistics; optimal split tg/$tb = \sqrt{\frac{Rg}{Rb}}$; $Rg > Rb$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_count_time_for_precision', Rg=100.0, Rb=25.0, r=0.01)
# -> T = 400 s, tg = 266.667 s, tb = 133.333 s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_count_time_for_precision', Rg=5.0, Rb=1.0, r=0.05)
# -> T = 261.803 s, tg = 180.902 s, tb = 80.9017 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Counting time for precision: Rg = 100 cps, Rb = 25 cps, r = 1%	SOLVED	T = 400 s, tg = 266.667 s, tb = 133.333 s
Counting time for precision: Rg = 300 cpm, Rb = 60 cpm, r = 5%	SOLVED	T = 261.803 s, tg = 180.902 s, tb = 80.9017 s
Counting time for precision: Rb = 25 cps, r = 1%	NEEDS_INPUT	nuc_count_time_for_precision needs Rg (gross count rate). Example: solve('nuc_count_time_for_precision', Rg='100 1/s', R
Counting time for precision: Rg = 100 cps, Rb = 25 cps, r = 1%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Solid angle and geometric efficiency of a disk detector (source on axis) — `nuc_solid_angle_on_axis`

The solid angle subtended by a circular detector face (or aperture) at a point source on its axis and the geometric efficiency (fraction of isotropic emissions that hit it).

$$\Omega = 2\pi\left(1 - \frac{d}{\sqrt{d^2 + a^2}}\right)$$

input	meaning	condition
`d`	distance from the source to the detector face	
`a`	radius of the detector face	

output	meaning
`Omega`	solid angle (sr)
`g`	geometric efficiency Omega/(4 pi)

- **Method:** $Omega = 2$ pi (1 - d/sqrt(d^2 + a^2))
- **Assumes:** point isotropic source on the axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_solid_angle_on_axis', d=0.1, a=0.0254)
# -> Omega = 0.193375, g = 0.0153883   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_solid_angle_on_axis', d=0.01, a=0.01)
# -> Omega = 1.8403, g = 0.146447   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solid angle on axis: d = 10 cm, a = 2.54 cm	SOLVED	Omega = 0.193375, g = 0.0153883
Solid angle on axis: d = 1 cm, a = 1 cm	SOLVED	Omega = 1.8403, g = 0.146447
Solid angle on axis: a = 2.54 cm	NEEDS_INPUT	nuc_solid_angle_on_axis needs d (distance from the source to the detector face). Example: solve('nuc_solid_angle_on_axis
Solid angle on axis: d = 10 cm, a = 2.54 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Solid angle of a disk detector for an off-axis point source — `nuc_solid_angle_off_axis`

The solid angle subtended by a circular detector face at a point source displaced sideways from its axis, and the geometric efficiency.

$$\Omega = \int_0^{2\pi} \left(1 - \frac{d}{\sqrt{d^2 + r_e(\varphi)^2}}\right) d\varphi$$

input	meaning	condition
`d`	distance from the source to the plane of the detector face	
`a`	radius of the detector face	
`rho0`	lateral offset of the source from the detector axis	

output	meaning
`Omega`	solid angle (sr)
`g`	geometric efficiency Omega/(4 pi)

- **Method:** Omega = int over azimuth of (1 - d/sqrt(d^2 + r_edge(phi)^2)) dphi (one-dimensional, mpmath tanh-sinh)
- **Assumes:** point isotropic source; flat circular face
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_solid_angle_off_axis', d=0.05, a=0.025, rho0=0.01)
# -> Omega = 0.637049, g = 0.0506947   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_solid_angle_off_axis', d=0.05, a=0.025, rho0=0.05)
# -> Omega = 0.282708, g = 0.0224972   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solid angle off axis: d = 5 cm, a = 2.5 cm, rho0 = 1 cm	SOLVED	Omega = 0.637049, g = 0.0506947
Solid angle off axis: d = 5 cm, a = 2.5 cm, rho0 = 5 cm	SOLVED	Omega = 0.282708, g = 0.0224972
Solid angle off axis: a = 2.5 cm, rho0 = 1 cm	NEEDS_INPUT	nuc_solid_angle_off_axis needs d (distance from the source to the plane of the detector face). Example: solve('nuc_solid
Solid angle off axis: d = 5 cm, a = 2.5 cm, rho0 = 1 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Total and intrinsic efficiency of a slab detector for a point source on axis —

`nuc_detector_efficiency_point_source`

The total detection efficiency of a flat detector of given thickness and attenuation coefficient for a point source on its axis (every photon entering the front face crosses the thickness along its slant path), the geometric efficiency and the intrinsic efficiency.

$$\varepsilon = \frac{1}{2}\int_{\cos\theta_{max}}^{1}\left(1 - e^{-\mu t/u}\right)du$$

input	meaning	condition
`d`	source-to-face distance	
`a`	radius of the detector face	
`t`	detector thickness	
`mu`	total linear attenuation coefficient of the detector material	

output	meaning
`eff`	total efficiency (interacting photons per emitted photon)
`g`	geometric efficiency
`intrinsic`	intrinsic efficiency eff/g

- **Method:** eff = (1/2) int_{cos theta_max}^1 (1 - e^(-mu t/u)) du, cos $\theta_{max} = \frac{d}{\sqrt{d^2+a^2}}$
- **Assumes:** point source on the axis; path length t/cos(theta) for every ray through the front face (side escape neglected)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_detector_efficiency_point_source', d=0.1, a=0.0381, t=0.0762,
mu=28.0)
# -> eff = 0.0291542, g = 0.0327635, intrinsic = 0.889836   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_detector_efficiency_point_source', d=0.02, a=0.025, t=0.005,
mu=50.0)
# -> eff = 0.0503627, g = 0.187652, intrinsic = 0.268383   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Detector efficiency for a point source: d = 10 cm, a = 3.81 cm, t = 7.62 cm, mu = 0.28 1/cm	SOLVED	eff = 0.0291542, g = 0.0327635, intrinsic = 0.889836
Detector efficiency for a point source: d = 2 cm, a = 2.5 cm, t = 5 mm, mu = 0.5 1/cm	SOLVED	eff = 0.0503627, g = 0.187652, intrinsic = 0.268383
Detector efficiency for a point source: a = 3.81 cm, t = 7.62 cm, mu = 0.28 1/cm	NEEDS_INPUT	nuc_detector_efficiency_point_source needs d (source-to-face distance). Example: solve('nuc_detector_efficiency_point_so
Detector efficiency for a point source: d = 10 cm, a = 3.81 cm, t = 7.62 cm, mu = 0.28 1/cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Approach to criticality: inverse multiplication (1/M) extrapolation — `nuc_inverse_multiplication`

Predict the critical loading (fuel elements, kg, rod position) from count rates measured as fuel is added, by extrapolating the inverse multiplication $1/M = C_0/C$ to zero: least-squares line through all points and the line through the last two.

$$\frac{1}{M} = \frac{C_0}{C}, \quad x_{crit} : \frac{1}{M}(x_{crit}) = 0$$

input	meaning	condition
`x`	loadings (fuel elements, kg or any consistent measure), first = reference	
`C`	count rates at those loadings (any consistent unit)	

output	meaning
`x_crit`	predicted critical loading (least-squares line through all points)
`slope`	slope of $1/M$ per unit loading
`x_crit_last2`	predicted critical loading from the last two points
`inv_M_last`	$1/M$ at the last loading

- **Method:** $1/M_i = C_1/C_i$; straight lines in x; x_crit where the line reaches $1/M = 0$
- **Assumes:** $1/M$ referred to the first count rate; $1/M$ decreasing with loading; subcritical measurements

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_inverse_multiplication', x=[0.0, 20.0, 40.0, 60.0, 80.0], C=
[100.0, 125.0, 167.0, 250.0, 500.0])
# -> x_crit = 99.976, slope = -0.01, x_crit_last2 = 100, inv_M_last = 0.2   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_inverse_multiplication', x=[10.0, 30.0, 50.0], C=[50.0, 80.0,
160.0])
# -> x_crit = 67.5758, slope = -0.0171875, x_crit_last2 = 70, inv_M_last = 0.3125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inverse multiplication: x = [0.0, 20.0, 40.0, 60.0, 80.0], C = [100.0, 125.0, 167.0, 250.0, 500.0]	SOLVED	x_crit = 99.976, slope = -0.01, x_crit_last2 = 100, inv_M_last = 0.2
Inverse multiplication: x = [10.0, 30.0, 50.0], C = [50.0, 80.0, 160.0]	SOLVED	x_crit = 67.5758, slope = -0.0171875, x_crit_last2 = 70, inv_M_last = 0.3125
Inverse multiplication: C = [100.0, 125.0, 167.0, 250.0, 500.0]	NEEDS_INPUT	nuc_inverse_multiplication needs x (loadings (fuel elements, kg or any consistent measure), first = reference). Example:
Inverse multiplication: x = [0.0, 20.0, 40.0, 60.0, 80.0], C = [100.0, 125.0, 167.0, 250.0, 500.0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Neutron energy and lab angle after an elastic collision — `nuc_elastic_scattering`

The energy of a neutron after elastic scattering from a nucleus of mass ratio A through a stated centre-of-mass angle, the energy ratio, the scattering angle in the laboratory and the lethargy gained.

$$\frac{E'}{E} = \frac{A^2 + 2A\cos\theta + 1}{(A+1)^2}$$

input	meaning	condition
`A`	mass of the nucleus in neutron masses (mass number; 1 for hydrogen)	
`E`	neutron energy before the collision	
`theta`	scattering angle in the centre-of-mass frame (deg or rad)	

output	meaning
`E_out`	neutron energy after the collision
`ratio`	E'/E
`psi`	scattering angle in the laboratory frame
`du`	lethargy gain ln(E/E')

- **Method:** E'/E = (A^2 + 2A cos theta + 1)/(A + 1)^2; tan $\psi = A$ sin theta/(1 + A cos theta)
- **Assumes:** elastic scattering from a nucleus at rest; $A \geq 1$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_elastic_scattering', A=12.0, E=3.2043532679999997e-13,
theta=1.5707963267948966)
# -> E_out = 2.7493e-13 J, ratio = 0.857988, psi = 1.48766 rad, du = 0.153165   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_elastic_scattering', A=1.0, E=1.6021766339999998e-13,
theta=1.0471975511965976)
# -> E_out = 1.20163e-13 J, ratio = 0.75, psi = 0.523599 rad, du = 0.287682   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elastic scattering: A = 12.0, E = 2 MeV, theta = 90 deg	SOLVED	E_out = 2.7493e-13 J, ratio = 0.857988, psi = 1.48766 rad, du = 0.153165
Elastic scattering: A = 1.0, E = 1 MeV, theta = 60 deg	SOLVED	E_out = 1.20163e-13 J, ratio = 0.75, psi = 0.523599 rad, du = 0.287682
Elastic scattering: E = 2 MeV, theta = 90 deg	NEEDS_INPUT	nuc_elastic_scattering needs A (mass of the nucleus in neutron masses (mass number; 1 for hydrogen)). Example: solve('nu
Elastic scattering: A = 12.0, E = 2 MeV, theta = 90 deg, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Average logarithmic energy decrement and mean scattering cosine — `nuc_logarithmic_energy_decrement`

For elastic scattering from a nucleus of mass ratio A (isotropic in the centre of mass): the minimum energy ratio alpha, the average logarithmic energy loss per collision xi and the average cosine of the laboratory scattering angle.

$$\xi = 1 + \frac{\alpha \ln \alpha}{1-\alpha}, \quad \alpha = \left(\frac{A-1}{A+1}\right)^2$$

input	meaning	condition
`A`	mass of the nucleus in neutron masses (mass number)	

output	meaning
`alpha`	minimum E'/E, (($A-1$)/($A+1$))^2
`xi`	average logarithmic energy decrement
`mu_bar`	average cosine of the lab scattering angle, $2/(3A)$

- **Method:** $\xi = 1 + \alpha$ ln(alpha)/(1 - alpha) (xi = 1 for A = 1); mu_bar = 2/(3A)
- **Assumes:** isotropic scattering in the centre-of-mass frame; $A \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_logarithmic_energy_decrement', A=12.0)
# -> alpha = 0.715976, xi = 0.157769, mu_bar = 0.0555556   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_logarithmic_energy_decrement', A=1.0)
# -> alpha = 0, xi = 1, mu_bar = 0.666667   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_logarithmic_energy_decrement', A=2.0)
# -> alpha = 0.111111, xi = 0.725347, mu_bar = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logarithmic energy decrement: A = 12.0	SOLVED	alpha = 0.715976, xi = 0.157769, mu_bar = 0.0555556
Logarithmic energy decrement: A = 1.0	SOLVED	alpha = 0, xi = 1, mu_bar = 0.666667
Logarithmic energy decrement: A = 2.0	SOLVED	alpha = 0.111111, xi = 0.725347, mu_bar = 0.333333
Logarithmic energy decrement:	NEEDS_INPUT	nuc_logarithmic_energy_decrement needs A (mass of the nucleus in neutron masses (mass number)). Example: solve('nuc_loga
Logarithmic energy decrement: A = 12.0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Logarithmic energy decrement: A = 0.5	INVALID_INPUT	Average logarithmic energy decrement and mean scattering cosine: the mass ratio A must be at least 1.

Average number of collisions to slow a neutron down — `nuc_collisions_to_thermalize`

The average number of elastic collisions with nuclei of mass ratio A needed to slow a neutron from E0 to Ef (for example from 2 MeV to 0.025 eV), from the lethargy change and xi.

$$\bar{n} = \frac{\ln(E_0/E_f)}{\xi}$$

input	meaning	condition
`A`	mass of the moderator nucleus in neutron masses	
`E0`	initial neutron energy	
`Ef`	final neutron energy	

output	meaning
`n`	average number of collisions
`xi`	average logarithmic energy decrement

- **Method:** $n = ln$(E0/Ef)/xi
- **Assumes:** isotropic CM scattering; $E_0 > Ef$; a single nuclide moderator
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_collisions_to_thermalize', A=12.0, E0=3.2043532679999997e-13,
Ef=4.005441585e-21)
# -> n = 115.343, xi = 0.157769   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_collisions_to_thermalize', A=1.0, E0=3.2043532679999997e-13,
Ef=4.005441585e-21)
# -> n = 18.1975, xi = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Collisions to thermalize: A = 12.0, E0 = 2 MeV, Ef = 0.025 eV	SOLVED	n = 115.343, xi = 0.157769
Collisions to thermalize: A = 1.0, E0 = 2 MeV, Ef = 0.025 eV	SOLVED	n = 18.1975, xi = 1
Collisions to thermalize: E0 = 2 MeV, Ef = 0.025 eV	NEEDS_INPUT	nuc_collisions_to_thermalize needs A (mass of the moderator nucleus in neutron masses). Example: solve('nuc_collisions_t
Collisions to thermalize: A = 12.0, E0 = 2 MeV, Ef = 0.025 eV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Moderating power and ratio of a compound moderator — `nuc_moderator_mixture`

The macroscopic scattering and absorption cross sections of a compound moderator (water, heavy water, graphite, beryllium oxide, ...), its average logarithmic energy decrement (weighted by scattering), moderating power and moderating ratio.

$$\bar{\xi} = \frac{\sum_i \xi_i \Sigma_{s,i}}{\sum_i \Sigma_{s,i}}, \quad MR = \frac{\bar{\xi}\Sigma_s}{\Sigma_a}$$

input	meaning	condition
`n`	atoms of each element per molecule	
`A`	mass ratio (mass number) of each element	
`sigma_s`	scattering cross section of each element, in barns	
`sigma_a`	absorption cross section of each element, in barns	
`rho`	density of the moderator	
`M`	molar mass of the molecule	

output	meaning
`xi`	average logarithmic energy decrement of the mixture
`MP`	moderating power xi Sigma_s
`MR`	moderating ratio xi Sigma_s/Sigma_a (when Sigma_a > 0)

- **Method:** $N = \rho$ N_A/M; $Sigma_x = N$ sum n_i sigma_x,i; xi = sum xi_i Sigma_s,i / Sigma_s
- **Assumes:** isotropic CM scattering for every element; constant cross sections

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_moderator_mixture', n=[2.0, 1.0], A=[1.0, 16.0], sigma_s=
[20.0, 3.8], sigma_a=[0.332, 0.00019], rho=1000.0, M=0.018015)
# -> xi = 0.923648, MP = 135.238 1/m, MR = 60.91   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_moderator_mixture', n=[1.0], A=[12.0], sigma_s=[4.8],
sigma_a=[0.0034], rho=1600.0, M=0.012011)
# -> xi = 0.157769, MP = 6.07512 1/m, MR = 222.733   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moderator mixture: n = [2.0, 1.0], A = [1.0, 16.0], sigma_s = [20.0, 3.8], sigma_a = [0.332, 0.00019], rho = 1 g/cm^3, M = 18.015 g/mol	SOLVED	xi = 0.923648, MP = 135.238 1/m, MR = 60.91
Moderator mixture: n = [1.0], A = [12.0], sigma_s = [4.8], sigma_a = [0.0034], rho = 1.6 g/cm^3, M = 12.011 g/mol	SOLVED	xi = 0.157769, MP = 6.07512 1/m, MR = 222.733
Moderator mixture: A = [1.0, 16.0], sigma_s = [20.0, 3.8], sigma_a = [0.332, 0.00019], rho = 1 g/cm^3, M = 18.015 g/mol	NEEDS_INPUT	nuc_moderator_mixture needs n (atoms of each element per molecule). Example: solve('nuc_moderator_mixture', n=[2.0, 1.0]
Moderator mixture: n = [2.0, 1.0], A = [1.0, 16.0], sigma_s = [20.0, 3.8], sigma_a = [0.332, 0.00019], rho = 1 g/cm^3, M = 18.015 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Maxwellian-averaged 1/v cross section — `nuc_maxwellian_average_cross_section`

The effective thermal cross section of a $1/v$ absorber averaged over a Maxwellian neutron flux at a stated neutron temperature, from its tabulated 2200 m/s value.

$$\bar{\sigma} = \frac{\sqrt{\pi}}{2}\,\sigma_0\sqrt{\frac{E_0}{kT}}$$

input	meaning	condition
`sigma0`	cross section at 2200 m/s (barns)	
`T`	neutron temperature	

output	meaning
`sigma_avg`	flux-averaged cross section
`sigma_avg_b`	the same in barns
`g`	ratio sigma_avg/sigma0 = (sqrt(pi)/2) sqrt(E0/kT)

- **Method:** sigma_avg = sigma0 (sqrt(pi)/2) sqrt(E0/(k T))

- **Assumes:** $1/v$ cross section; Maxwellian flux spectrum phi(E) ~ E $e^{-E/kT}$; $E_0 = m_n$ (2200 m/s)^2/2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_maxwellian_average_cross_section', sigma0=6.809e-26, T=293.6)
# -> sigma_avg = 6.03412e-26 m^2, sigma_avg_b = 603.412, g = 0.886198   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_maxwellian_average_cross_section', sigma0=2.68e-28, T=600.0)
# -> sigma_avg = 1.66138e-28 m^2, sigma_avg_b = 1.66138, g = 0.619917   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maxwellian average cross section: sigma0 = 680.9 b, T = 293.6 K	SOLVED	sigma_avg = 6.03412e-26 m^2, sigma_avg_b = 603.412, g = 0.886198
Maxwellian average cross section: sigma0 = 2.68 b, T = 600 K	SOLVED	sigma_avg = 1.66138e-28 m^2, sigma_avg_b = 1.66138, g = 0.619917
Maxwellian average cross section: T = 293.6 K	NEEDS_INPUT	nuc_maxwellian_average_cross_section needs sigma0 (cross section at 2200 m/s (barns)). Example: solve('nuc_maxwellian_av
Maxwellian average cross section: sigma0 = 680.9 b, T = 293.6 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Number densities and macroscopic cross sections of UO2 fuel — `nuc_fuel_cross_sections`

Atom densities of $U-235$, $U-238$ and oxygen in uranium dioxide of stated density and enrichment, its macroscopic absorption and fission cross sections, eta (neutrons per absorption in the fuel) and the absorption mean free path.

$$N_{25} = \frac{\rho N_A}{M_U + 2M_O}\, a_5, \quad \Sigma_a = \sum_i N_i \sigma_{a,i}$$

input	meaning	condition
`rho`	density of the UO2 (e.g. 10.4 g/cm^3)	
`w`	enrichment: mass fraction of $U-235$ in the uranium	
`sigma_a5`	$U-235$ absorption cross section (barns)	
`sigma_f5`	$U-235$ fission cross section (barns)	
`sigma_a8`	$U-238$ absorption cross section (barns)	
`sigma_a0`	oxygen absorption cross section (barns)	
`nu`	neutrons per fission of $U-235$	

output	meaning
`N5`	$U-235$ atoms per volume
`N8`	$U-238$ atoms per volume

output	meaning
`NO`	oxygen atoms per volume
`Sigma_a`	macroscopic absorption cross section
`Sigma_f`	macroscopic fission cross section
`eta`	$\eta = \nu$ Sigma_f/Sigma_a of the fuel
`mfp_a`	absorption mean free path $1/Sigma_a$

- **Method:** atom fraction $a_5 = (w/M_5)/(w/M_5 + (1-w)/M_8)$; N(UO2) = rho N_A/(M_U + 2 M_O); Sigma = sum N_i sigma_i
- **Assumes:** stoichiometric UO2 with $U-235$ and $U-238$ only; atomic masses $U-235$ 235.043928, $U-238$ 238.050787, O 15.999 g/mol
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_fuel_cross_sections', rho=10400.0, w=0.03, sigma_a5=6.809e-
26, sigma_f5=5.826e-26, sigma_a8=2.68e-28, sigma_aO=1.9e-32, nu=2.43)
# -> N5 = 7.04635e+26 1/m^3, N8 = 2.24954e+28 1/m^3, NO = 4.64001e+28 1/m^3, Sigma_a = 54.0082 1/m,
Sigma_f = 41.052 1/m, eta = 1.84706, mfp_a = 0.0185157 m   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_fuel_cross_sections', rho=10970.0, w=0.00711,
sigma_a5=6.809e-26, sigma_f5=5.826e-26, sigma_a8=2.68e-28, sigma_aO=1.9e-32, nu=2.43)
# -> N5 = 1.76157e+26 1/m^3, N8 = 2.42891e+28 1/m^3, NO = 4.89306e+28 1/m^3, Sigma_a = 18.505 1/m,
Sigma_f = 10.2629 1/m, eta = 1.34769, mfp_a = 0.0540395 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fuel cross sections: rho = 10.4 g/cm^3, w = 3%, sigma_a5 = 680.9 b, sigma_f5 = 582.6 b, sigma_a8 = 2.68 b, sigma_aO = 0.00019 b, nu = 2.43	SOLVED	N5 = 7.04635e+26 1/m^3, N8 = 2.24954e+28 1/m^3, NO = 4.64001e+28 1/m^3, Sigma_a = 54.0082 1/m, Sigma_f = 41.052 1/m, eta = 1.84706, mfp_a = 0.0185157 m
Fuel cross sections: rho = 10.97 g/cm^3, w = 0.711%, sigma_a5 = 680.9 b, sigma_f5 = 582.6 b, sigma_a8 = 2.68 b, sigma_aO = 0.00019 b, nu = 2.43	SOLVED	N5 = 1.76157e+26 1/m^3, N8 = 2.42891e+28 1/m^3, NO = 4.89306e+28 1/m^3, Sigma_a = 18.505 1/m, Sigma_f = 10.2629 1/m, eta = 1.34769, mfp_a = 0.0540395 m
Fuel cross sections: w = 3%, sigma_a5 = 680.9 b, sigma_f5 = 582.6 b, sigma_a8 = 2.68 b, sigma_aO = 0.00019 b, nu = 2.43	NEEDS_INPUT	nuc_fuel_cross_sections needs rho (density of the UO2 (e.g. 10.4 g/cm^3)). Example: solve('nuc_fuel_cross_sections', rho
Fuel cross sections: rho = 10.4 g/cm^3, w = 3%, sigma_a5 = 680.9 b, sigma_f5 = 582.6 b, sigma_a8 = 2.68 b, sigma_aO = 0.00019 b, nu = 2.43, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Critical thickness of a bare slab reactor (one group) — `nuc_critical_slab`

The critical thickness of an infinite bare slab of multiplying material in one-group diffusion theory, the extrapolated thickness and the material buckling.

$$\left(\frac{\pi}{a_{ext}}\right)^2 = \frac{\nu\Sigma_f - \Sigma_a}{D}$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	

output	meaning
`a`	critical physical thickness
`a_ext`	critical extrapolated thickness

- **Method:** B_m^2 = (nu Sigma_f - Sigma_a)/D; critical when (pi/a_ext)^2 = B_m^2; a = a_ext - 2d
- **Assumes:** one-group diffusion; k_inf > 1; bare slab, flux zero at the extrapolated faces
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_critical_slab', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='2.13D')
# -> a = 0.133711 m, a_ext = 0.172072 m   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_critical_slab', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='none')
# -> a = 0.172072 m, a_ext = 0.172072 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical slab: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D	SOLVED	a = 0.133711 m, a_ext = 0.172072 m
Critical slab: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none	SOLVED	a = 0.172072 m, a_ext = 0.172072 m
Critical slab: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D	NEEDS_INPUT	nuc_critical_slab needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_critical_slab', D='0.009 m',
Critical slab: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

question	outcome	result
Critical slab: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.12 1/cm, extrapolation = none	INVALID_INPUT	Critical thickness of a bare slab reactor (one group): k-infinity <= 1: the medium cannot be critical at any size.
Critical slab: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm	NEEDS_INPUT	nuc_critical_slab needs extrapolation (extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 tran

Critical radius of a bare spherical reactor (one group) — `nuc_critical_sphere`

The critical radius (physical and extrapolated) and volume of a bare homogeneous spherical reactor in one-group diffusion theory.

$$\left(\frac{\pi}{R_{ext}}\right)^2 = \frac{\nu\Sigma_f - \Sigma_a}{D}$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	

output	meaning
`R`	critical physical radius
`R_ext`	critical extrapolated radius
`V`	critical physical volume

- **Method:** (pi/R_ext)^2 = B_m^2; R = R_ext - d
- **Assumes:** one-group diffusion; k_inf > 1; bare homogeneous sphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_critical_sphere', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='2.13D')
# -> R = 0.152891 m, R_ext = 0.172072 m, V = 0.0149705 m^3   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_critical_sphere', D=0.01, Sigma_a=10.0, nuSigma_f=12.5,
extrapolation='2D')
# -> R = 0.178692 m, R_ext = 0.198692 m, V = 0.0239002 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical sphere: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D	SOLVED	R = 0.152891 m, R_ext = 0.172072 m, V = 0.0149705 m^3

question	outcome	result
Critical sphere: D = 1 cm, Sigma_a = 0.1 1/cm, nuSigma_f = 0.125 1/cm, extrapolation = 2D	SOLVED	R = 0.178692 m, R_ext = 0.198692 m, V = 0.0239002 m^3
Critical sphere: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D	NEEDS_INPUT	nuc_critical_sphere needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_critical_sphere', D='0.009
Critical sphere: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Critical radius of a bare cylindrical reactor of given height (one group) — `nuc_critical_cylinder`

The critical radius of a bare homogeneous cylinder of stated height in one-group diffusion theory, with its extrapolated radius and the physical volume.

$$\left(\frac{2.405}{R_{ext}}\right)^2 + \left(\frac{\pi}{H_{ext}}\right)^2 = B_m^2$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	
`H`	physical height of the cylinder	

output	meaning
`R`	critical physical radius
`R_ext`	critical extrapolated radius
`V`	critical physical volume

- **Method:** (j01/R_ext)^2 + (pi/H_ext)^2 = B_m^2, $j01 = 2.405$ (first zero of J0)
- **Assumes:** one-group diffusion; the height exceeds the critical slab thickness
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_critical_cylinder', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='2.13D', H=0.5)
# -> R = 0.119829 m, R_ext = 0.139009 m, V = 0.0225549 m^3   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_critical_cylinder', D=0.01, Sigma_a=10.0, nuSigma_f=12.5,
extrapolation='none', H=1.0)
# -> R = 0.155189 m, R_ext = 0.155189 m, V = 0.0756606 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical cylinder: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, H = 50 cm	SOLVED	R = 0.119829 m, R_ext = 0.139009 m, V = 0.0225549 m^3
Critical cylinder: D = 1 cm, Sigma_a = 0.1 1/cm, nuSigma_f = 0.125 1/cm, extrapolation = none, H = 1 m	SOLVED	R = 0.155189 m, R_ext = 0.155189 m, V = 0.0756606 m^3
Critical cylinder: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, H = 50 cm	NEEDS_INPUT	nuc_critical_cylinder needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_critical_cylinder', D='0.
Critical cylinder: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, H = 50 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Critical cylinder: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, H = 5 cm	INVALID_INPUT	Critical radius of a bare cylindrical reactor of given height (one group): this height is too small: no radius makes the

Minimum-volume critical bare cylinder (one group) — `nuc_critical_cylinder_minimum_volume`

The radius and height of the bare cylindrical reactor that is critical with the smallest volume (extrapolated height-to-diameter ratio 0.924), and that volume.

$$H_{ext} = \frac{\sqrt{3}\,\pi}{B_m}, \quad R_{ext} = \sqrt{\tfrac{3}{2}}\,\frac{2.405}{B_m}$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	

output	meaning
`R`	physical radius
`H`	physical height
`V_ext`	minimum critical volume (extrapolated dimensions)
`HD`	extrapolated height/diameter

- **Method:** minimise pi R^2 H subject to $(j01/R)^2 + (pi/H)^2 = B_m^2$: H_ext = sqrt(3) pi/B_m, R_ext = sqrt(3/2) $j01/B_m$
- **Assumes:** one-group diffusion; k_inf > 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_critical_cylinder_minimum_volume', D=0.009, Sigma_a=15.0,
nuSigma_f=18.0, extrapolation='none')
# -> R = 0.161321 m, H = 0.298038 m, V_ext = 0.0243669 m^3, HD = 0.923743   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_critical_cylinder_minimum_volume', D=0.01, Sigma_a=10.0,
nuSigma_f=12.5, extrapolation='2.13D')
# -> R = 0.164965 m, H = 0.30152 m, V_ext = 0.0375153 m^3, HD = 0.923743   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Minimum volume critical cylinder: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none	SOLVED	R = 0.161321 m, H = 0.298038 m, V_ext = 0.0243669 m^3, HD = 0.923743
Minimum volume critical cylinder: D = 1 cm, Sigma_a = 0.1 1/cm, nuSigma_f = 0.125 1/cm, extrapolation = 2.13D	SOLVED	R = 0.164965 m, H = 0.30152 m, V_ext = 0.0375153 m^3, HD = 0.923743
Minimum volume critical cylinder: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none	NEEDS_INPUT	nuc_critical_cylinder_minimum_volume needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_critical_c
Minimum volume critical cylinder: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Critical third side of a bare rectangular reactor (one group) — `nuc_critical_cuboid`

The third side that makes a bare rectangular parallelepiped of two stated sides critical in one-group diffusion theory, its extrapolated value and the physical volume.

$$\left(\frac{\pi}{a_{ext}}\right)^2+\left(\frac{\pi}{b_{ext}}\right)^2+\left(\frac{\pi}{c_{ext}}\right)^2=B_m^2$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	
`a`	first physical side	
`b`	second physical side	

output	meaning
`c`	critical physical third side
`c_ext`	critical extrapolated third side
`V`	critical physical volume

- **Method:** (pi/a_ext)^2 + (pi/b_ext)^2 + (pi/c_ext)^2 = B_m^2
- **Assumes:** one-group diffusion; the two given sides are large enough

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_critical_cuboid', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='2.13D', a=0.6, b=0.6)
# -> c = 0.147765 m, c_ext = 0.186126 m, V = 0.0531953 m^3   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_critical_cuboid', D=0.01, Sigma_a=10.0, nuSigma_f=12.5,
extrapolation='none', a=1.0, b=2.0)
# -> c = 0.203784 m, c_ext = 0.203784 m, V = 0.407567 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical cuboid: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, a = 60 cm, b = 60 cm	SOLVED	c = 0.147765 m, c_ext = 0.186126 m, V = 0.0531953 m^3
Critical cuboid: D = 1 cm, Sigma_a = 0.1 1/cm, nuSigma_f = 0.125 1/cm, extrapolation = none, a = 1 m, b = 2 m	SOLVED	c = 0.203784 m, c_ext = 0.203784 m, V = 0.407567 m^3
Critical cuboid: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, a = 60 cm, b = 60 cm	NEEDS_INPUT	nuc_critical_cuboid needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_critical_cuboid', D='0.009
Critical cuboid: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, a = 60 cm, b = 60 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

k-effective and geometric buckling of a bare reactor (one group) — `nuc_bare_reactor_keff`

The geometric buckling of a bare homogeneous slab, sphere, cylinder or rectangular reactor of stated size and its effective multiplication factor in one-group diffusion theory, with the non-leakage probability and the reactivity.

$$k_{eff} = \frac{\nu\Sigma_f}{\Sigma_a + DB_g^2}$$

input	meaning	condition
`D`	diffusion coefficient (a length, e.g. cm)	
`Sigma_a`	macroscopic absorption cross section	
`nuSigma_f`	nu times the macroscopic fission cross section	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	
`geometry`	bare reactor shape: slab (a), sphere (R), cylinder (R, H) or cuboid (a, b, c)	
`a`	slab thickness or first side of a cuboid	
`b`	second side of a cuboid	

input	meaning	condition
`c`	third side of a cuboid	
`R`	radius of a sphere or cylinder	
`H`	height of a cylinder	

output	meaning
`B2`	geometric buckling (extrapolated dimensions)
`k_eff`	effective multiplication factor
`P_NL`	non-leakage probability 1/(1 + L^2 B^2)
`rho`	reactivity $(k-1)/k$

- **Method:** B_g^2 from the shape $(pi/a)^2$, $(pi/R)^2$, $(2.405/R)^2 + (pi/H)^2$, sum $(pi/x_i)^2$; $k = \nu$ Sigma_f/(Sigma_a + D B_g^2)
- **Assumes:** one-group diffusion; bare homogeneous reactor; the dimensions of the stated geometry only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_bare_reactor_keff', D=0.009, Sigma_a=15.0, nuSigma_f=18.0,
extrapolation='2.13D', geometry='sphere', R=0.12)
# -> B2 = 509.496 1/m^2, k_eff = 0.919049, P_NL = 0.765874, rho = -0.0880815   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_bare_reactor_keff', D=0.01, Sigma_a=10.0, nuSigma_f=12.5,
extrapolation='none', geometry='cylinder', R=0.3, H=0.6)
# -> B2 = 91.6732 1/m^2, k_eff = 1.14503, P_NL = 0.916025, rho = 0.126661   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bare reactor k-effective: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, geometry = sphere, R = 12 cm	SOLVED	B2 = 509.496 1/m^2, k_eff = 0.919049, P_NL = 0.765874, rho = -0.0880815
Bare reactor k-effective: D = 1 cm, Sigma_a = 0.1 1/cm, nuSigma_f = 0.125 1/cm, extrapolation = none, geometry = cylinder, R = 30 cm, H = 60 cm	SOLVED	B2 = 91.6732 1/m^2, k_eff = 1.14503, P_NL = 0.916025, rho = 0.126661
Bare reactor k-effective: Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, geometry = sphere, R = 12 cm	NEEDS_INPUT	nuc_bare_reactor_keff needs D (diffusion coefficient (a length, e.g. cm)). Example: solve('nuc_bare_reactor_keff', D='0.
Bare reactor k-effective: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = 2.13D, geometry = sphere, R = 12 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Bare reactor k-effective: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none, geometry = sphere, R = 12 cm, H = 10 cm	INVALID_INPUT	k-effective and geometric buckling of a bare reactor (one group): H is not a dimension of a sphere (it needs R).

question	outcome	result
Bare reactor k-effective: D = 0.9 cm, Sigma_a = 0.15 1/cm, nuSigma_f = 0.18 1/cm, extrapolation = none, geometry = cylinder, R = 12 cm	INVALID_INPUT	k-effective and geometric buckling of a bare reactor (one group): the cylinder needs R, H.

k-effective of a bare reactor in two-group diffusion theory — `nuc_two_group_bare_keff`

The effective and infinite multiplication factors of a bare homogeneous reactor in two-group (fast and thermal) diffusion theory, with fission in both groups and downscatter from fast to thermal, and the thermal-to-fast flux ratio of the fundamental mode.

$$k = \frac{\nu\Sigma_{f1} + \nu\Sigma_{f2}\frac{\Sigma_{12}}{D_2B^2+\Sigma_{a2}}}{D_1B^2 + \Sigma_{a1} + \Sigma_{12}}$$

input	meaning	condition
`geometry`	bare reactor shape: slab (a), sphere (R), cylinder (R, H) or cuboid (a, b, c)	
`D1`	fast-group diffusion coefficient	
`D2`	thermal-group diffusion coefficient	
`Sigma_a1`	fast-group absorption cross section	
`Sigma_12`	downscatter (removal) cross section from fast to thermal	
`Sigma_a2`	thermal-group absorption cross section	
`nuSigma_f1`	fast-group nu Sigma_f	
`nuSigma_f2`	thermal-group nu Sigma_f	
`a`	slab thickness or first side of a cuboid (extrapolated)	
`b`	second side of a cuboid (extrapolated)	
`c`	third side of a cuboid (extrapolated)	
`R`	radius of a sphere or cylinder (extrapolated)	
`H`	height of a cylinder (extrapolated)	

output	meaning
`B2`	geometric buckling
`k_eff`	effective multiplication factor
`k_inf`	infinite multiplication factor
`flux_ratio`	thermal/fast flux ratio of the fundamental mode

- **Method:** k = [nu $Sigma_f1 + \nu$ Sigma_f2 Sigma_12/(D2 B^2 + Sigma_a2)] / (D1 B^2 + Sigma_a1 + Sigma_12)
- **Assumes:** two-group diffusion, no upscatter; all fission neutrons born fast; extrapolated dimensions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_two_group_bare_keff', geometry='cylinder', D1=0.0143, D2=0.0037, Sigma_a1=1.0, Sigma_12=1.8, Sigma_a2=8.0, nuSigma_f1=0.5, nuSigma_f2=13.5, R=1.7, H=3.7)
# -> B2 = 2.72204 1/m^2, k_eff = 1.24472, k_inf = 1.26339, flux_ratio = 0.224717   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_two_group_bare_keff', geometry='sphere', D1=0.012, D2=0.004,
Sigma_a1=0.5, Sigma_12=2.0, Sigma_a2=10.0, nuSigma_f1=0.0, nuSigma_f2=14.0, R=1.0)
# -> B2 = 9.8696 1/m^2, k_eff = 1.06514, k_inf = 1.12, flux_ratio = 0.199214   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-group k-effective: geometry = cylinder, D1 = 1.43 cm, D2 = 0.37 cm, Sigma_a1 = 0.01 1/cm, Sigma_12 = 0.018 1/cm, Sigma_a2 = 0.08 1/cm, nuSigma_f1 = 0.005 1/cm, nuSigma_f2 = 0.135 1/cm, R = 170 cm, H = 370 cm	SOLVED	B2 = 2.72204 1/m^2, k_eff = 1.24472, k_inf = 1.26339, flux_ratio = 0.224717
Two-group k-effective: geometry = sphere, D1 = 1.2 cm, D2 = 0.4 cm, Sigma_a1 = 0.005 1/cm, Sigma_12 = 0.02 1/cm, Sigma_a2 = 0.1 1/cm, nuSigma_f1 = 0 1/cm, nuSigma_f2 = 0.14 1/cm, R = 1 m	SOLVED	B2 = 9.8696 1/m^2, k_eff = 1.06514, k_inf = 1.12, flux_ratio = 0.199214
Two-group k-effective: D1 = 1.43 cm, D2 = 0.37 cm, Sigma_a1 = 0.01 1/cm, Sigma_12 = 0.018 1/cm, Sigma_a2 = 0.08 1/cm, nuSigma_f1 = 0.005 1/cm, nuSigma_f2 = 0.135 1/cm, R = 170 cm, H = 370 cm	NEEDS_INPUT	nuc_two_group_bare_keff needs geometry (bare reactor shape: slab (a), sphere (R), cylinder (R, H) or cuboid (a, b, c)).
Two-group k-effective: geometry = cylinder, D1 = 1.43 cm, D2 = 0.37 cm, Sigma_a1 = 0.01 1/cm, Sigma_12 = 0.018 1/cm, Sigma_a2 = 0.08 1/cm, nuSigma_f1 = 0.005 1/cm, nuSigma_f2 = 0.135 1/cm, R = 170 cm, H = 370 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Critical thickness of a reflected slab reactor and reflector savings (one group) — `nuc_reflected_slab_critical`

The critical core thickness of a slab reactor with a reflector of stated thickness on both faces (one-group diffusion in core and reflector) and the reflector savings compared with the bare core.

$$D_c B \tan\frac{Ba}{2} = \frac{D_r}{L_r}\coth\frac{T_r + d}{L_r}$$

input	meaning	condition
`D_c`	core diffusion coefficient	
`Sigma_ac`	core absorption cross section	
`nuSigma_fc`	core nu Sigma_f	
`D_r`	reflector diffusion coefficient	
`Sigma_ar`	reflector absorption cross section	
`T_r`	reflector thickness on each face	
`extrapolation`	extrapolation distance added at each free surface: none, 2D or 2.13D (= 0.71 transport mean free paths)	

output	meaning
`a`	critical core thickness
`delta`	reflector savings per face (bare extrapolated half-thickness minus reflected half-thickness)

- **Method:** D_c B tan(B a/2) = (D_r/L_r) coth((T_r + d)/L_r), $L_r^2 = D_r / Sigma_a r$
- **Assumes:** one-group diffusion; non-multiplying reflector; extrapolation distance of the reflector at its outer face
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_reflected_slab_critical', D_c=0.009, Sigma_ac=15.0,
nuSigma_fc=18.0, D_r=0.0016, Sigma_ar=1.97, T_r=0.2, extrapolation='2.13D')
# -> a = 0.0360662 m, delta = 0.0680029 m   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_reflected_slab_critical', D_c=0.01, Sigma_ac=10.0,
nuSigma_fc=12.5, D_r=0.009, Sigma_ar=0.03, T_r=0.5, extrapolation='none')
# -> a = 0.0180701 m, delta = 0.0903108 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reflected slab critical: D_c = 0.9 cm, Sigma_ac = 0.15 1/cm, nuSigma_fc = 0.18 1/cm, D_r = 0.16 cm, Sigma_ar = 0.0197 1/cm, T_r = 20 cm, extrapolation = 2.13D	SOLVED	a = 0.0360662 m, delta = 0.0680029 m
Reflected slab critical: D_c = 1 cm, Sigma_ac = 0.1 1/cm, nuSigma_fc = 0.125 1/cm, D_r = 0.9 cm, Sigma_ar = 0.0003 1/cm, T_r = 50 cm, extrapolation = none	SOLVED	a = 0.0180701 m, delta = 0.0903108 m
Reflected slab critical: Sigma_ac = 0.15 1/cm, nuSigma_fc = 0.18 1/cm, D_r = 0.16 cm, Sigma_ar = 0.0197 1/cm, T_r = 20 cm, extrapolation = 2.13D	NEEDS_INPUT	nuc_reflected_slab_critical needs D_c (core diffusion coefficient). Example: solve('nuc_reflected_slab_critical', D_c='0
Reflected slab critical: D_c = 0.9 cm, Sigma_ac = 0.15 1/cm, nuSigma_fc = 0.18 1/cm, D_r = 0.16 cm, Sigma_ar = 0.0197 1/cm, T_r = 20 cm, extrapolation = 2.13D, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Diffusion length from an exponential pile measurement — `nuc_exponential_pile`

The thermal diffusion length of a moderator from fluxes measured along the axis of an exponential pile (a source at the bottom of a prism of extrapolated sides a and b): least-squares relaxation constant gamma, then L from $\gamma^2 = 1/L^2 + (pi/a)^2 + (pi/b)^2$.

$$\gamma^2 = \frac{1}{L^2} + \left(\frac{\pi}{a}\right)^2 + \left(\frac{\pi}{b}\right)^2$$

input	meaning	condition
`z`	heights of the measurements along the axis, in cm (far from the source and the top)	
`phi`	measured fluxes or detector readings at those heights (any consistent unit)	
`a`	first extrapolated transverse side	
`b`	second extrapolated transverse side	

output	meaning
`gamma`	fitted relaxation constant
`L`	diffusion length
`L2`	diffusion area L^2

- **Method:** least squares of ln $\phi = c - \gamma$ z; $1/L^2 = \gamma^2 - (pi/a)^2 - (pi/b)^2$
- **Assumes:** fundamental transverse mode only (points far from the source); an effectively infinite pile (points far from the top)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_exponential_pile', z=[20.0, 40.0, 60.0, 80.0, 100.0], phi=
[10000.0, 6200.0, 3850.0, 2390.0, 1480.0], a=2.0, b=2.0)
# -> gamma = 2.38717 1/m, L = 1.14423 m, L2 = 1.30927 m^2   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_exponential_pile', z=[30.0, 50.0, 70.0], phi=[500.0, 254.0,
129.0], a=1.5, b=1.8)
# -> gamma = 3.38699 1/m, L = 0.497578 m, L2 = 0.247584 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential pile: z = [20.0, 40.0, 60.0, 80.0, 100.0], phi = [10000.0, 6200.0, 3850.0, 2390.0, 1480.0], a = 2 m, b = 2 m	SOLVED	gamma = 2.38717 1/m, L = 1.14423 m, L2 = 1.30927 m^2
Exponential pile: z = [30.0, 50.0, 70.0], phi = [500.0, 254.0, 129.0], a = 150 cm, b = 180 cm	SOLVED	gamma = 3.38699 1/m, L = 0.497578 m, L2 = 0.247584 m^2
Exponential pile: phi = [10000.0, 6200.0, 3850.0, 2390.0, 1480.0], a = 2 m, b = 2 m	NEEDS_INPUT	nuc_exponential_pile needs z (heights of the measurements along the axis, in cm (far from the source and the top)). Exam
Exponential pile: z = [20.0, 40.0, 60.0, 80.0, 100.0], phi = [10000.0, 6200.0, 3850.0, 2390.0, 1480.0], a = 2 m, b = 2 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Flux from a plane neutron source at the centre of a slab — `nuc_plane_source_slab_flux`

The one-group thermal flux at a position in a non-multiplying slab of stated (extrapolated) thickness with a plane source of neutrons at its mid-plane, the flux at the source plane and the fraction of neutrons that leak from the two faces.

$$\phi(x) = \frac{SL}{2D}\,\frac{\sinh\left[(a/2 - |x|)/L\right]}{\cosh\left[a/(2L)\right]}$$

input	meaning	condition
`S`	source strength per unit area of the plane (neutrons/(m^2 s))	
`D`	diffusion coefficient	
`Sigma_a`	absorption cross section	
`a`	extrapolated slab thickness (source at the centre)	
`x`	distance from the source plane	

output	meaning
`phi`	flux at x
`phi0`	flux at the source plane
`leak`	fraction of source neutrons leaking from the two faces

- **Method:** phi(x) = ($S\ L/(2D)$) sinh((a/2 - |x|)/L)/cosh($a/(2L)$); leakage 1/cosh($a/(2L)$)
- **Assumes:** one-group diffusion in a non-multiplying medium; flux zero at the extrapolated faces
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_plane_source_slab_flux', S=1000000000000.0, D=0.0016,
Sigma_a=1.97, a=1.0, x=0.1)
# -> phi = 2.66547e+11 1/(m^2*s), phi0 = 8.90588e+12 1/(m^2*s), leak = 4.80301e-08   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_plane_source_slab_flux', S=10000000000.0, D=0.009,
Sigma_a=0.03, a=2.0, x=0.0)
# -> phi = 2.88896e+11 1/(m^2*s), phi0 = 2.88896e+11 1/(m^2*s), leak = 0.314046   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane source in a slab: S = 1e8 1/(cm^2*s), D = 0.16 cm, Sigma_a = 0.0197 1/cm, a = 1 m, x = 10 cm	SOLVED	phi = 2.66547e+11 1/(m^2*s), phi0 = 8.90588e+12 1/(m^2*s), leak = 4.80301e-08
Plane source in a slab: S = 1e10 1/(m^2*s), D = 0.9 cm, Sigma_a = 0.03 1/m, a = 2 m, x = 0 m	SOLVED	phi = 2.88896e+11 1/(m^2*s), phi0 = 2.88896e+11 1/(m^2*s), leak = 0.314046
Plane source in a slab: D = 0.16 cm, Sigma_a = 0.0197 1/cm, a = 1 m, x = 10 cm	NEEDS_INPUT	nuc_plane_source_slab_flux needs S (source strength per unit area of the plane (neutrons/(m^2 s))). Example: solve('nuc_
Plane source in a slab: S = 1e8 1/(cm^2*s), D = 0.16 cm, Sigma_a = 0.0197 1/cm, a = 1 m, x = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Flux from a point neutron source at the centre of a sphere — `nuc_point_source_sphere_flux`

The one-group thermal flux at a radius in a non-multiplying sphere (extrapolated radius R) with an isotropic point source at its centre, the fraction of neutrons that leak out, and the flux the same source would give in an infinite medium.

$$\phi(r) = \frac{S}{4\pi D r}\,\frac{\sinh[(R-r)/L]}{\sinh(R/L)}$$

input	meaning	condition
`S`	source strength (neutrons per second)	
`D`	diffusion coefficient	
`Sigma_a`	absorption cross section	
`R`	extrapolated radius of the sphere	
`r`	distance from the source	

output	meaning
`phi`	flux at r
`leak`	fraction of source neutrons leaking out
`phi_inf`	flux at r in an infinite medium

- **Method:** phi(r) = S sinh($(R-r)/L$)/(4 pi D r sinh(R/L)); leakage (R/L)/sinh(R/L); infinite medium S e^(-r/L)/(4 pi D r)
- **Assumes:** one-group diffusion in a non-multiplying medium; $0 < r < R$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_point_source_sphere_flux', S=1000000000.0, D=0.0016,
Sigma_a=1.97, R=0.5, r=0.1)
# -> phi = 1.48856e+10 1/(m^2*s), leak = 8.42667e-07, phi_inf = 1.48856e+10 1/(m^2*s)   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_point_source_sphere_flux', S=10000000.0, D=0.009,
Sigma_a=0.03, R=1.0, r=0.5)
# -> phi = 6.11299e+07 1/(m^2*s), leak = 0.60392, phi_inf = 7.09778e+07 1/(m^2*s)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Point source in a sphere: S = 1e9 1/s, D = 0.16 cm, Sigma_a = 0.0197 1/cm, R = 50 cm, r = 10 cm	SOLVED	phi = 1.48856e+10 1/(m^2*s), leak = 8.42667e-07, phi_inf = 1.48856e+10 1/(m^2*s)
Point source in a sphere: S = 1e7 1/s, D = 0.9 cm, Sigma_a = 0.03 1/m, R = 1 m, r = 50 cm	SOLVED	phi = 6.11299e+07 1/(m^2*s), leak = 0.60392, phi_inf = 7.09778e+07 1/(m^2*s)
Point source in a sphere: D = 0.16 cm, Sigma_a = 0.0197 1/cm, R = 50 cm, r = 10 cm	NEEDS_INPUT	nuc_point_source_sphere_flux needs S (source strength (neutrons per second)). Example: solve('nuc_point_source_sphere_fl

question	outcome	result
Point source in a sphere: S = 1e9 1/s, D = 0.16 cm, Sigma_a = 0.0197 1/cm, R = 50 cm, r = 10 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Fundamental-mode flux shape of a bare reactor at a point — `nuc_flux_shape`

The neutron flux at a point of a bare slab, sphere or finite cylinder relative to its maximum at the centre (cosine, sin(x)/x, J0 times cosine) and the geometric buckling, for extrapolated dimensions.

$$\frac{\phi}{\phi_{max}} = J_0\left(\frac{2.405\,r}{R}\right)\cos\frac{\pi z}{H}$$

input	meaning	condition
`geometry`	shape: slab (a, x), sphere (R, r) or cylinder (R, H, r, z)	
`a`	extrapolated slab thickness	
`R`	extrapolated radius	
`H`	extrapolated cylinder height	
`x`	distance from the slab mid-plane	
`r`	distance from the centre or axis	
`z`	height above the cylinder mid-plane	

output	meaning
`ratio`	phi/phi_max at the point
`B2`	geometric buckling

- **Method:** slab cos(pi x/a); sphere sin(pi r/R)/(pi r/R); cylinder J0(2.405 r/R) cos(pi z/H)
- **Assumes:** fundamental mode of a bare homogeneous reactor; extrapolated dimensions; the point inside the reactor
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_flux_shape', geometry='cylinder', R=1.5, H=3.0, r=0.75,
z=0.5)
# -> ratio = 0.580176, B2 = 3.66693 1/m^2   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_flux_shape', geometry='slab', a=1.0, x=0.25)
# -> ratio = 0.707107, B2 = 9.8696 1/m^2   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_flux_shape', geometry='sphere', R=0.5, r=0.25)
# -> ratio = 0.63662, B2 = 39.4784 1/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flux shape: geometry = cylinder, R = 1.5 m, H = 3 m, r =	SOLVED	ratio = 0.580176, B2 = 3.66693 1/m^2

question	outcome	result
75 cm, z = 50 cm		
Flux shape: geometry = slab, a = 1 m, x = 25 cm	SOLVED	ratio = 0.707107, B2 = 9.8696 1/m^2
Flux shape: geometry = sphere, R = 50 cm, r = 25 cm	SOLVED	ratio = 0.63662, B2 = 39.4784 1/m^2
Flux shape: R = 1.5 m, H = 3 m, r = 75 cm, z = 50 cm	NEEDS_INPUT	nuc_flux_shape needs geometry (shape: slab (a, x), sphere (R, r) or cylinder (R, H, r, z)). Example: solve('nuc_flux_sha
Flux shape: geometry = cylinder, R = 1.5 m, H = 3 m, r = 75 cm, z = 50 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Reactor power from the peak flux (or peak flux from power) of a bare core — `nuc_core_power_from_flux`

The thermal power of a bare homogeneous sphere, cylinder or rectangular core from its peak thermal flux (or the peak flux from the power), with the average flux, the peak-to-average ratio and the volume, for the fundamental-mode flux shape.

$$P = E_f \Sigma_f \int_V \phi \, dV = E_f \Sigma_f \, \phi_{max} \, \frac{\bar{\phi}}{\phi_{max}} \, V$$

input	meaning	condition
`geometry`	core shape: sphere (R), cylinder (R, H) or cuboid (a, b, c)	
`Sigma_f`	macroscopic fission cross section	
`E_f`	recoverable energy per fission (e.g. 200 MeV)	
`phi_max`	peak (centre) thermal flux (give this or P)	
`P`	thermal power (give this or phi_max)	
`R`	extrapolated radius	
`H`	extrapolated cylinder height	
`a`	first extrapolated side of a cuboid	
`b`	second side	
`c`	third side	

output	meaning
`P`	thermal power
`phi_max`	peak flux
`phi_avg`	volume-average flux
`peaking`	peak-to-average flux ratio

- **Method:** $P = E_f$ Sigma_f phi_max <shape> V; <shape> = $3/pi^2$ (sphere), (2 J1(2.405)/2.405)$(2/pi)$ (cylinder), $(2/pi)^3$ (cuboid)
- **Assumes:** fundamental-mode flux of a bare homogeneous core; extrapolated dimensions taken as the core size; uniform Sigma_f

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_core_power_from_flux', geometry='cylinder', Sigma_f=0.2,
E_f=3.204353268e-11, phi_max=1e+17, R=1.7, H=3.7)
# -> P = 5.91748e+06 W, phi_max = 1e+17 1/(m^2*s), phi_avg = 2.74864e+16 1/(m^2*s), peaking =
3.63817   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_core_power_from_flux', geometry='sphere', Sigma_f=0.5,
E_f=3.204353268e-11, P=1000000.0, R=0.5)
# -> P = 1e+06 W, phi_max = 3.92166e+17 1/(m^2*s), phi_avg = 1.19204e+17 1/(m^2*s), peaking =
3.28987   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Core power from flux: geometry = cylinder, Sigma_f = 0.002 1/cm, E_f = 200 MeV, phi_max = 1e13 1/(cm^2*s), R = 170 cm, H = 370 cm	SOLVED	P = 5.91748e+06 W, phi_max = 1e+17 1/(m^2*s), phi_avg = 2.74864e+16 1/(m^2*s), peaking = 3.63817
Core power from flux: geometry = sphere, Sigma_f = 0.005 1/cm, E_f = 200 MeV, P = 1 MW, R = 50 cm	SOLVED	P = 1e+06 W, phi_max = 3.92166e+17 1/(m^2*s), phi_avg = 1.19204e+17 1/(m^2*s), peaking = 3.28987
Core power from flux: Sigma_f = 0.002 1/cm, E_f = 200 MeV, phi_max = 1e13 1/(cm^2*s), R = 170 cm, H = 370 cm	NEEDS_INPUT	nuc_core_power_from_flux needs geometry (core shape: sphere (R), cylinder (R, H) or cuboid (a, b, c)). Example: solve('n
Core power from flux: geometry = cylinder, Sigma_f = 0.002 1/cm, E_f = 200 MeV, phi_max = 1e13 1/(cm^2*s), R = 170 cm, H = 370 cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Core power from flux: geometry = sphere, Sigma_f = 0.005 1/cm, E_f = 200 MeV, P = 1 MW, phi_max = 1e13 1/(cm^2*s), R = 50 cm	INVALID_INPUT	Reactor power from the peak flux (or peak flux from power) of a bare core: give either the peak flux phi_max or the powe

Stable reactor period after a reactivity step (one delayed group) — `nuc_reactor_period_one_group`

The stable (asymptotic) period, inverse period and doubling time of a reactor after a step reactivity change, with one effective delayed-neutron group (the stable root of the inhour equation).

$$\rho = \omega\Lambda + \frac{\beta\,\omega}{\omega + \lambda}$$

input	meaning	condition
`rho`	step reactivity (absolute, or in pcm, %, dollars or cents)	
`beta`	delayed-neutron fraction beta	
`lam`	precursor decay constant of the single delayed group (e.g. 0.08 1/s)	
`Lambda`	prompt neutron generation time	

output	meaning
`omega`	inverse stable period (largest inhour root)
`T`	stable period (negative: falling power)
`T_d`	doubling time T ln 2 (negative: halving time)

- **Method:** largest root of $\rho = \omega$ Lambda + beta omega/(omega + lambda)
- **Assumes:** point kinetics with one delayed group; rho below prompt critical $(\rho < \beta)$; $\rho \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_reactor_period_one_group', rho=0.001, beta=0.0065, lam=0.08,
Lambda=0.0001)
# -> omega = 0.0145205 1/s, T = 68.8682 s, T_d = 47.7358 s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_reactor_period_one_group', rho=-0.0065, beta=0.0065,
lam=0.0767, Lambda=5e-05)
# -> omega = -0.0383443 1/s, T = -26.0795 s, T_d = -18.0769 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reactor period one group: rho = 100 pcm, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s	SOLVED	omega = 0.0145205 1/s, T = 68.8682 s, T_d = 47.7358 s
Reactor period one group: rho = -1 $, beta = 0.0065, lam = 0.0767 1/s, Lambda = 50 us	SOLVED	omega = -0.0383443 1/s, T = -26.0795 s, T_d = -18.0769 s
Reactor period one group: beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s	NEEDS_INPUT	nuc_reactor_period_one_group needs rho (step reactivity (absolute, or in pcm, %, dollars or cents)). Example: solve('nuc
Reactor period one group: rho = 100 pcm, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Reactor period one group: rho = 1.2 $, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s	INVALID_INPUT	Stable reactor period after a reactivity step (one delayed group): prompt critical or above (rho >= beta) is not support
Reactor period one group: rho = 0.1 $, lam = 0.08 1/s, Lambda = 1e-4 s	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Power after a reactivity step (one delayed group, exact) — `nuc_power_after_step_one_group`

The relative power P(t)/P0 a time t after a step reactivity change from steady state, from the exact two-mode solution with one delayed group, with the amplitude of the stable period (the exact prompt jump or drop) and the inverse period.

$$\frac{P(t)}{P_0} = \sum_j A_j e^{\omega_j t}$$

input	meaning	condition
`rho`	step reactivity (absolute, or in pcm, %, dollars or cents)	
`beta`	delayed-neutron fraction beta	
`lam`	precursor decay constant of the single delayed group (e.g. 0.08 1/s)	
`Lambda`	prompt neutron generation time	
`t`	time after the step	

output	meaning
`P`	relative power P(t)/P0
`A1`	amplitude of the stable mode (exact prompt jump or drop)
`omega`	inverse stable period

- **Method:** $P/P_0 = A_1$ e^(w1 t) + A2 e^(w2 t), w_j the inhour roots, A_j = 1/(1 - w_j S2(w_j)/N(w_j)) (residues)
- **Assumes:** point kinetics, one delayed group, steady state before the step; $\rho < \beta$, $\rho \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_power_after_step_one_group', rho=0.001, beta=0.0065,
lam=0.08, Lambda=0.0001, t=10.0)
# -> P = 1.36579, A1 = 1.18119, omega = 0.0145205 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_power_after_step_one_group', rho=-0.0065, beta=0.0065,
lam=0.08, Lambda=0.0001, t=1.0)
# -> P = 0.480696, A1 = 0.500308, omega = -0.0399877 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power after a reactivity step one group: rho = 100 pcm, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s, t = 10 s	SOLVED	P = 1.36579, A1 = 1.18119, omega = 0.0145205 1/s
Power after a reactivity step one group: rho = -1 $, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s, t = 1 s	SOLVED	P = 0.480696, A1 = 0.500308, omega = -0.0399877 1/s
Power after a reactivity step one group: beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s, t = 10 s	NEEDS_INPUT	nuc_power_after_step_one_group needs rho (step reactivity (absolute, or in pcm, %, dollars or cents)). Example: solve('n
Power after a reactivity step one group: rho = 100 pcm, beta = 0.0065, lam = 0.08 1/s, Lambda = 1e-4 s, t = 10 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Power after a reactivity step (six delayed groups, exact) — `nuc_power_after_step_six_group`

The relative power P(t)/P0 a time t after a step reactivity change from steady state with six (or any number up to eight) delayed-neutron groups, from all the inhour roots and their residues, and the stable period.

$$\rho = \omega\Lambda + \sum_i \frac{\beta_i\,\omega}{\omega+\lambda_i}, \qquad \frac{P}{P_0} = \sum_j A_j e^{\omega_j t}$$

input	meaning	condition
`rho`	step reactivity (absolute, or in pcm, %, dollars or cents)	
`beta`	delayed-neutron fraction of each group	
`lam`	decay constant of each group, in $1/s$	
`Lambda`	prompt neutron generation time	
`t`	time after the step	

output	meaning
`P`	relative power P(t)/P0
`omega`	inverse stable period
`T`	stable period

- **Method:** P/P0 = sum_j A_j e^(w_j t) over the $n+1$ inhour roots (bisection between the poles, 40 digits), A_j = 1/(1 - w_j sum $beta_i/(w_j+lambda_i)^2/N$(w_j))
- **Assumes:** point kinetics, steady state before the step; $\rho < \beta$ (sum of the groups), $\rho \neq 0$; distinct decay constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_power_after_step_six_group', rho=0.001, beta=[0.000215,
0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam=[0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01],
Lambda=5e-05, t=10.0)
# -> P = 1.61699, omega = 0.0181937 1/s, T = 54.964 s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_power_after_step_six_group', rho=-0.006502, beta=[0.000215,
0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam=[0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01],
Lambda=5e-05, t=30.0)
# -> P = 0.126561, omega = -0.0119183 1/s, T = -83.9044 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power after a reactivity step six groups: rho = 100 pcm, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, t = 10 s	SOLVED	P = 1.61699, omega = 0.0181937 1/s, T = 54.964 s
Power after a reactivity step six groups: rho = -1 $, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, t = 30 s	SOLVED	P = 0.126561, omega = -0.0119183 1/s, T = -83.9044 s
Power after a reactivity step six groups: beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305,	NEEDS_INPUT	nuc_power_after_step_six_group needs rho (step reactivity (absolute, or in pcm, %, dollars or cents)). Example: solve('n

question	outcome	result
0.111, 0.301, 1.14, 3.01], Lambda = 50 us, t = 10 s		
Power after a reactivity step six groups: rho = 100 pcm, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, t = 10 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Power after a reactivity step six groups: rho = 100 pcm, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01] 1/s, Lambda = 50 us, t = 10 s	SOLVED	P = 1.61699, omega = 0.0181937 1/s, T = 54.964 s

Stable period and prompt root of the inhour equation (six delayed groups) — `nuc_stable_period_six_group`

The stable reactor period, inverse period and doubling time after a step reactivity change with six (or any number up to eight) delayed groups, and the prompt (most negative) root that sets the prompt jump's time constant.

$$\rho = \omega\Lambda + \sum_i \frac{\beta_i\,\omega}{\omega + \lambda_i}$$

input	meaning	condition
`rho`	step reactivity (absolute, or in pcm, %, dollars or cents)	
`beta`	delayed-neutron fraction of each group	
`lam`	decay constant of each group, in $1/s$	
`Lambda`	prompt neutron generation time	

output	meaning
`omega`	inverse stable period (largest root)
`T`	stable period
`T_d`	doubling time (negative: halving time)
`omega_p`	prompt root (most negative)

- **Method:** largest and smallest roots of the inhour equation (bisection between the poles, 40 digits)
- **Assumes:** point kinetics; $\rho < \beta$, $\rho \neq 0$; distinct decay constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_stable_period_six_group', rho=0.001, beta=[0.000215,
0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam=[0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01],
Lambda=5e-05)
# -> omega = 0.0181937 1/s, T = 54.964 s, T_d = 38.0982 s, omega_p = -110.523 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_stable_period_six_group', rho=0.003251, beta=[0.000215,
0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam=[0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01],
Lambda=0.0001)
# -> omega = 0.172398 1/s, T = 5.80053 s, T_d = 4.02062 s, omega_p = -33.3362 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stable period six groups: rho = 100 pcm, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us	SOLVED	omega = 0.0181937 1/s, T = 54.964 s, T_d = 38.0982 s, omega_p = -110.523 1/s
Stable period six groups: rho = 50 cents, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 100 us	SOLVED	omega = 0.172398 1/s, T = 5.80053 s, T_d = 4.02062 s, omega_p = -33.3362 1/s
Stable period six groups: beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us	NEEDS_INPUT	nuc_stable_period_six_group needs rho (step reactivity (absolute, or in pcm, %, dollars or cents)). Example: solve('nuc_
Stable period six groups: rho = 100 pcm, beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Reactivity from a measured stable period or doubling time (inhour) —

`nuc_reactivity_from_period`

The reactivity, in absolute units, dollars and pcm, that produces a measured stable period (or doubling time) with stated delayed-neutron data, from the inhour equation.

$$\rho = \frac{\Lambda}{T} + \sum_i \frac{\beta_i}{1 + \lambda_i T}$$

input	meaning	condition
`beta`	delayed-neutron fraction of each group	
`lam`	decay constant of each group, in $1/s$	
`Lambda`	prompt neutron generation time	
`T`	stable period (give this or T_d; negative for falling power)	
`T_d`	doubling time (give this or T)	

output	meaning
`rho`	reactivity
`rho_dollars`	reactivity in dollars
`rho_pcm`	reactivity in pcm

- **Method:** $\rho = \omega$ Lambda + sum beta_i omega/(omega + lambda_i), $\omega = 1/T = ln\ 2/T_d$
- **Assumes:** point kinetics, asymptotic period; for a negative period: 1/T > -lambda_min
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_reactivity_from_period', beta=[0.000215, 0.001424, 0.001274,
0.002568, 0.000748, 0.000273], lam=[0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda=5e-05,
T=60.0)
# -> rho = 0.000940626, rho_dollars = 0.144667, rho_pcm = 94.0626   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_reactivity_from_period', beta=[0.0065], lam=[0.08],
Lambda=0.0001, T_d=30.0)
# -> rho = 0.0014589, rho_dollars = 0.224447, rho_pcm = 145.89   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reactivity from period: beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, T = 60 s	SOLVED	rho = 0.000940626, rho_dollars = 0.144667, rho_pcm = 94.0626
Reactivity from period: beta = [0.0065], lam = [0.08], Lambda = 1e-4 s, T_d = 30 s	SOLVED	rho = 0.0014589, rho_dollars = 0.224447, rho_pcm = 145.89
Reactivity from period: lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, T = 60 s	NEEDS_INPUT	nuc_reactivity_from_period needs beta (delayed-neutron fraction of each group). Example: solve('nuc_reactivity_from_peri
Reactivity from period: beta = [0.000215, 0.001424, 0.001274, 0.002568, 0.000748, 0.000273], lam = [0.0124, 0.0305, 0.111, 0.301, 1.14, 3.01], Lambda = 50 us, T = 60 s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Reactivity from period: beta = [0.0065], lam = [0.08] 1/s, Lambda = 1e-4 s, T = 60 s, T_d = 30 s	INVALID_INPUT	Reactivity from a measured stable period or doubling time (inhour): give either the stable period T or the doubling time

Integral and differential control-rod worth (sine-squared weighting) — `nuc_control_rod_worth`

The reactivity worth of a control rod inserted to a depth x in a core of height H with a cosine axial flux (first-order perturbation: worth density ~ sin^2), as a fraction of its total worth, and the differential worth at that depth.

$$\frac{\rho(x)}{\rho_{tot}} = \frac{x}{H} - \frac{1}{2\pi}\sin\frac{2\pi x}{H}$$

input	meaning	condition
`rho_tot`	total worth of the fully inserted rod (absolute, or in pcm, %, dollars or cents)	
`H`	core height	
`x`	insertion depth from the top of the core	

input	meaning	condition
`beta`	delayed-neutron fraction (optional: converts a worth in dollars and reports dollars)	

output	meaning
`rho`	worth inserted (absolute)
`frac`	fraction of the total worth
`drho_dx`	differential worth at x (reactivity per metre)
`rho_dollars`	worth in dollars (when beta is given)

- **Method:** rho(x)/rho_tot = x/H - sin(2 pi x/H)/(2 pi); d rho/dx = (rho_tot/H)(1 - cos(2 pi x/H))
- **Assumes:** cosine axial flux, flux zero at the core ends; first-order perturbation (small rod)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_control_rod_worth', rho_tot=0.01, H=3.66, x=1.83)
# -> rho = 0.005, frac = 0.5, drho_dx = 0.00546448 1/m   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_control_rod_worth', rho_tot=0.013, H=1.0, x=0.25,
beta=0.0065)
# -> rho = 0.00118099, frac = 0.0908451, drho_dx = 0.013 1/m, rho_dollars = 0.18169   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Control rod worth: rho_tot = 1000 pcm, H = 3.66 m, x = 1.83 m	SOLVED	rho = 0.005, frac = 0.5, drho_dx = 0.00546448 1/m
Control rod worth: rho_tot = 2 $, H = 1 m, x = 25 cm, beta = 0.0065	SOLVED	rho = 0.00118099, frac = 0.0908451, drho_dx = 0.013 1/m, rho_dollars = 0.18169
Control rod worth: H = 3.66 m, x = 1.83 m	NEEDS_INPUT	nuc_control_rod_worth needs rho_tot (total worth of the fully inserted rod (absolute, or in pcm, %, dollars or cents)).
Control rod worth: rho_tot = 1000 pcm, H = 3.66 m, x = 1.83 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Equilibrium iodine-135 and xenon-135 concentrations — `nuc_xenon_equilibrium`

The equilibrium $I-135$ and $Xe-135$ atom densities in fuel operating at a steady flux, the xenon absorptions per fission (its poisoning strength) and the flux at which xenon burn-out equals its decay.

$$X_\infty = \frac{(\gamma_I + \gamma_X)\Sigma_f \phi}{\lambda_X + \sigma_X \phi}$$

input	meaning	condition
`Sigma_f`	macroscopic fission cross section of the fuel	
`phi`	thermal flux during operation	

input	meaning	condition
`gamma_I`	cumulative fission yield of $I-135$ (e.g. 0.0639)	
`gamma_X`	direct fission yield of $Xe-135$ (e.g. 0.00237)	
`sigma_X`	$Xe-135$ absorption cross section (e.g. 2.65e6 b)	

output	meaning
`I_eq`	equilibrium $I-135$ atoms per volume
`X_eq`	equilibrium $Xe-135$ atoms per volume
`abs_per_fission`	$Xe-135$ absorptions per fission at equilibrium

- **Method:** $I = gamma_I$ Sigma_f phi/lambda_I; $X = (gamma_I + gamma_X)$ Sigma_f phi/(lambda_X + sigma_X phi)
- **Assumes:** constant flux; half-lives of $I-135$ (6.57 h) and $Xe-135$ (9.14 h) of radioactivedecay 0.6.1; $Xe-135m$ neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_xenon_equilibrium', Sigma_f=10.0, phi=3e+17, gamma_I=0.0639,
gamma_X=0.00237, sigma_X=2.65e-22)
# -> I_eq = 6.54131e+21 1/m^3, X_eq = 1.97692e+21 1/m^3, abs_per_fission = 0.0523883   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_xenon_equilibrium', Sigma_f=10.0, phi=1e+16, gamma_I=0.0639,
gamma_X=0.00237, sigma_X=2.65e-22)
# -> I_eq = 2.18044e+20 1/m^3, X_eq = 2.79435e+20 1/m^3, abs_per_fission = 0.00740502   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Xenon equilibrium: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b	SOLVED	I_eq = 6.54131e+21 1/m^3, X_eq = 1.97692e+21 1/m^3, abs_per_fission = 0.0523883
Xenon equilibrium: Sigma_f = 0.1 1/cm, phi = 1e12 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65 Mb	SOLVED	I_eq = 2.18044e+20 1/m^3, X_eq = 2.79435e+20 1/m^3, abs_per_fission = 0.00740502
Xenon equilibrium: phi = 3e13 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b	NEEDS_INPUT	nuc_xenon_equilibrium needs Sigma_f (macroscopic fission cross section of the fuel). Example: solve('nuc_xenon_equilibri
Xenon equilibrium: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Xenon-135 after shutdown: concentration and peak — `nuc_xenon_after_shutdown`

The $Xe-135$ and $I-135$ atom densities a time t after shutdown from equilibrium operation, the xenon relative to its equilibrium value, and the time and height of the post-shutdown xenon peak (none at low flux).

$$X(t) = X_0 e^{-\lambda_X t} + \frac{\lambda_I I_0}{\lambda_X - \lambda_I}\left(e^{-\lambda_I t} - e^{-\lambda_X t}\right)$$

input	meaning	condition
`Sigma_f`	macroscopic fission cross section of the fuel	
`phi`	thermal flux during operation	
`gamma_I`	cumulative fission yield of $I-135$ (e.g. 0.0639)	
`gamma_X`	direct fission yield of $Xe-135$ (e.g. 0.00237)	
`sigma_X`	$Xe-135$ absorption cross section (e.g. 2.65e6 b)	
`t`	time after shutdown	

output	meaning
`X`	$Xe-135$ atoms per volume at t
`ratio`	X(t)/X_eq
`t_peak`	time of the xenon peak after shutdown (0 when xenon only falls)
`peak_ratio`	peak xenon / equilibrium xenon
`I`	$I-135$ atoms per volume at t

- **Method:** X(t) = X0 e^(-lambda_X t) + I0 lambda_I/(lambda_X - lambda_I)(e^(-lambda_I $t) - e^($ $-lambda_X$ t)); $dX/dt = 0$ for the peak
- **Assumes:** equilibrium before shutdown; no flux after shutdown
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_xenon_after_shutdown', Sigma_f=10.0, phi=3e+17,
gamma_I=0.0639, gamma_X=0.00237, sigma_X=2.65e-22, t=36000.0)
# -> X = 3.72338e+21 1/m^3, ratio = 1.88343, t_peak = 30167 s, peak_ratio = 1.90155, I =
2.27759e+21 1/m^3   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_xenon_after_shutdown', Sigma_f=10.0, phi=1000000000000000.0,
gamma_I=0.0639, gamma_X=0.00237, sigma_X=2.65e-22, t=3600.0)
# -> X = 3.09e+19 1/m^3, ratio = 0.994596, t_peak = 0 s, peak_ratio = 1, I = 1.96211e+19 1/m^3
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Xenon after shutdown: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b, t = 10 h	SOLVED	X = 3.72338e+21 1/m^3, ratio = 1.88343, t_peak = 30167 s, peak_ratio = 1.90155, I = 2.27759e+21 1/m^3
Xenon after shutdown: Sigma_f = 0.1 1/cm, phi = 1e11 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b, t = 1 h	SOLVED	X = 3.09e+19 1/m^3, ratio = 0.994596, t_peak = 0 s, peak_ratio = 1, I = 1.96211e+19 1/m^3
Xenon after shutdown: phi = 3e13 1/(cm^2*s), gamma_I = 0.0639,	NEEDS_INPUT	nuc_xenon_after_shutdown needs Sigma_f (macroscopic fission cross section of the fuel).

question	outcome	result
gamma_X = 0.00237, sigma_X = 2.65e6 b, t = 10 h		Example: solve('nuc_xenon_after_
Xenon after shutdown: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_I = 0.0639, gamma_X = 0.00237, sigma_X = 2.65e6 b, t = 10 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Samarium-149 poisoning: equilibrium and after shutdown — `nuc_samarium`

The equilibrium $Pm-149$ and $Sm-149$ atom densities during operation (Sm-149 is independent of the flux), the $Sm-149$ level reached after shutdown when all the promethium has decayed, and (optionally) the level a time t after shutdown.

$$S_\infty = \frac{\gamma_{Pm}\Sigma_f}{\sigma_{Sm}}, \quad P_\infty = \frac{\gamma_{Pm}\Sigma_f\phi}{\lambda_{Pm}}$$

input	meaning	condition
`Sigma_f`	macroscopic fission cross section	
`phi`	thermal flux during operation	
`gamma_Pm`	fission yield of $Pm-149$ (e.g. 0.0113)	
`sigma_Sm`	$Sm-149$ absorption cross section (e.g. 40800 b)	
`t`	time after shutdown (optional)	

output	meaning
`Pm_eq`	equilibrium $Pm-149$
`Sm_eq`	equilibrium $Sm-149$
`Sm_max`	$Sm-149$ after all $Pm-149$ has decayed
`max_ratio`	Sm_max/Sm_eq
`Sm_t`	$Sm-149$ at t after shutdown (when t is given)

- **Method:** $Pm = \gamma$ Sigma_f phi/lambda_Pm; $Sm = \gamma$ Sigma_f/sigma_Sm; after shutdown Sm + Pm(1 - e^(-lambda_Pm t))
- **Assumes:** constant flux before shutdown; $Pm-149$ half-life of radioactivedecay 0.6.1 (53.08 h); $Pm-149$ absorption neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_samarium', Sigma_f=10.0, phi=3e+17, gamma_Pm=0.0113,
sigma_Sm=4.08e-24)
# -> Pm_eq = 9.34561e+21 1/m^3, Sm_eq = 2.76961e+22 1/m^3, Sm_max = 3.70417e+22 1/m^3, max_ratio =
1.33743   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_samarium', Sigma_f=10.0, phi=3e+17, gamma_Pm=0.0113,
sigma_Sm=4.08e-24, t=172800.0)
# -> Pm_eq = 9.34561e+21 1/m^3, Sm_eq = 2.76961e+22 1/m^3, Sm_max = 3.70417e+22 1/m^3, max_ratio =
1.33743, Sm_t = 3.20484e+22 1/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Samarium poisoning: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_Pm = 0.0113, sigma_Sm = 40800 b	SOLVED	Pm_eq = 9.34561e+21 1/m^3, Sm_eq = 2.76961e+22 1/m^3, Sm_max = 3.70417e+22 1/m^3, max_ratio = 1.33743
Samarium poisoning: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_Pm = 0.0113, sigma_Sm = 40800 b, t = 48 h	SOLVED	Pm_eq = 9.34561e+21 1/m^3, Sm_eq = 2.76961e+22 1/m^3, Sm_max = 3.70417e+22 1/m^3, max_ratio = 1.33743, Sm_t = 3.20484e+22 1/m^3
Samarium poisoning: phi = 3e13 1/(cm^2*s), gamma_Pm = 0.0113, sigma_Sm = 40800 b	NEEDS_INPUT	nuc_samarium needs Sigma_f (macroscopic fission cross section). Example: solve('nuc_samarium', Sigma_f='10 1/m', phi='3e
Samarium poisoning: Sigma_f = 0.1 1/cm, phi = 3e13 1/(cm^2*s), gamma_Pm = 0.0113, sigma_Sm = 40800 b, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

U-235 depletion and energy released at constant flux — `nuc_depletion_constant_flux`

How much of an initial mass of $U-235$ remains after irradiation at a constant flux, the fraction burnt, the fission energy released (J and MWd) and the power at the start and at the end.

$$N(t) = N_0 e^{-\sigma_a \phi t}, \quad E = E_f \frac{\sigma_f}{\sigma_a}(N_0 - N)$$

input	meaning	condition
`m0`	initial mass of $U-235$	
`sigma_a`	$U-235$ absorption cross section (barns)	
`sigma_f`	$U-235$ fission cross section (barns)	
`phi`	neutron flux	
`t`	irradiation time	
`E_f`	recoverable energy per fission (e.g. 200 MeV)	

output	meaning
`m`	$U-235$ mass remaining
`burnt`	fraction of $U-235$ consumed
`E`	fission energy released
`E_MWd`	energy released in MWd
`P_t`	fission power at t

- **Method:** $N = N_0$ e^(-sigma_a phi t); $E = E_f$ (sigma_f/sigma_a) $(N_0 - N)$; $P = E_f$ sigma_f phi N
- **Assumes:** constant flux, one group; $U-235$ only (no breeding); M(U-235) = 235.043928 g/mol
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_depletion_constant_flux', m0=1.0, sigma_a=6.809e-26,
sigma_f=5.826e-26, phi=3e+17, t=31557600.0, E_f=3.204353268e-11)
# -> m = 0.524858 kg, burnt = 0.475142, E = 3.33774e+13 J, E_MWd = 386.313, P_t = 753140 W
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_depletion_constant_flux', m0=0.001, sigma_a=6.809e-26,
sigma_f=5.826e-26, phi=1e+18, t=86400.0, E_f=3.204353268e-11)
# -> m = 0.000994134 kg, burnt = 0.00586571, E = 4.1205e+08 J, E_MWd = 0.00476909, P_t = 4755.08 W
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Depletion at constant flux: m0 = 1 kg, sigma_a = 680.9 b, sigma_f = 582.6 b, phi = 3e13 1/(cm^2*s), t = 1 y, E_f = 200 MeV	SOLVED	m = 0.524858 kg, burnt = 0.475142, E = 3.33774e+13 J, E_MWd = 386.313, P_t = 753140 W
Depletion at constant flux: m0 = 1 g, sigma_a = 680.9 b, sigma_f = 582.6 b, phi = 1e14 1/(cm^2*s), t = 1 d, E_f = 200 MeV	SOLVED	m = 0.000994134 kg, burnt = 0.00586571, E = 4.1205e+08 J, E_MWd = 0.00476909, P_t = 4755.08 W
Depletion at constant flux: sigma_a = 680.9 b, sigma_f = 582.6 b, phi = 3e13 1/(cm^2*s), t = 1 y, E_f = 200 MeV	NEEDS_INPUT	nuc_depletion_constant_flux needs m0 (initial mass of U-235). Example: solve('nuc_depletion_constant_flux', m0='1 kg', s
Depletion at constant flux: m0 = 1 kg, sigma_a = 680.9 b, sigma_f = 582.6 b, phi = 3e13 1/(cm^2*s), t = 1 y, E_f = 200 MeV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Uranium enrichment: feed, tails and separative work (SWU) — `nuc_enrichment_swu`

The natural-uranium feed and tails masses and the separative work (kg SWU) needed to produce a mass of enriched uranium from stated product, feed and tails assays, per kg of product too.

$$SWU = P\,V(x_p) + W\,V(x_w) - F\,V(x_f), \quad V(x) = (2x-1)\ln\frac{x}{1-x}$$

input	meaning	condition
`x_p`	$U-235$ assay of the product (weight fraction, e.g. 4.5%)	
`x_f`	$U-235$ assay of the feed (0.711% for natural uranium)	
`x_w`	$U-235$ assay of the tails (e.g. 0.25%)	
`P`	mass of product	

output	meaning
`F`	feed mass
`W`	tails mass
`SWU`	separative work (kg SWU)
`F_per_P`	feed per unit product

output	meaning
`SWU_per_P`	kg SWU per kg of product

- **Method:** $F = P$ (x_p - x_w)/(x_f - x_w), $W = F - P$; SWU = P V(x_p) + W V(x_w) - F V(x_f)
- **Assumes:** tails < feed < product assays; value function V(x) = (2x - 1) ln($x/(1-x)$)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_enrichment_swu', x_p=0.045, x_f=0.00711, x_w=0.0025, P=1.0)
# -> F = 9.21909 kg, W = 8.21909 kg, SWU = 6.87111 kg, F_per_P = 9.21909, SWU_per_P = 6.87111
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_enrichment_swu', x_p=0.2, x_f=0.00711, x_w=0.003, P=100.0)
# -> F = 4793.19 kg, W = 4693.19 kg, SWU = 3831.51 kg, F_per_P = 47.9319, SWU_per_P = 38.3151
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Enrichment SWU: x_p = 4.5%, x_f = 0.711%, x_w = 0.25%, P = 1 kg	SOLVED	F = 9.21909 kg, W = 8.21909 kg, SWU = 6.87111 kg, F_per_P = 9.21909, SWU_per_P = 6.87111
Enrichment SWU: x_p = 20%, x_f = 0.711%, x_w = 0.3%, P = 100 kg	SOLVED	F = 4793.19 kg, W = 4693.19 kg, SWU = 3831.51 kg, F_per_P = 47.9319, SWU_per_P = 38.3151
Enrichment SWU: x_f = 0.711%, x_w = 0.25%, P = 1 kg	NEEDS_INPUT	nuc_enrichment_swu needs x_p (U-235 assay of the product (weight fraction, e.g. 4.5%)). Example: solve('nuc_enrichment_s
Enrichment SWU: x_p = 4.5%, x_f = 0.711%, x_w = 0.25%, P = 1 kg, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Enrichment SWU: x_p = 4.5%, x_f = 0.711%, x_w = 1%, P = 1 kg	INVALID_INPUT	Uranium enrichment: feed, tails and separative work (SWU): the assays must satisfy tails < feed < product.

Fuel-rod radial temperatures: centreline, pellet surface and cladding — `nuc_fuel_rod_temperatures`

The temperatures at the fuel centreline, the pellet surface and the cladding inner and outer surfaces of a cylindrical fuel rod at a stated linear heat rate, from the coolant temperature through the film, cladding, gap and pellet (constant conductivities).

$$T_0 - T_b = q' \left[\frac{1}{4\pi k_f} + \frac{1}{2\pi R_f h_g} + \frac{\ln(R_o/R_f)}{2\pi k_c} + \frac{1}{2\pi R_o h} \right]$$

input	meaning	condition
`q_lin`	linear heat rate q' (W/m)	
`R_f`	pellet radius	
`k_f`	fuel thermal conductivity	

input	meaning	condition
`h_g`	gap conductance	
`t_c`	cladding thickness	
`k_c`	cladding thermal conductivity	
`h`	coolant heat transfer coefficient	
`T_cool`	bulk coolant temperature	

output	meaning
`T_0`	fuel centreline temperature
`T_fs`	pellet surface temperature
`T_ci`	cladding inner surface temperature

- **Method:** $T_co = T_b + q'$/(2 pi R_o h); $T_ci = T_co + q'$ ln$(R_o/R_f)/(2$ pi k_c); $T_fs = T_ci + q'$/(2 pi R_f h_g); $T_0 = T_fs + q'$/(4 pi k_f)
- **Assumes:** steady state, radial conduction only; uniform heat generation in the pellet; thin gap of conductance h_g at the pellet radius
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_fuel_rod_temperatures', q_lin=20000.0, R_f=0.0041, k_f=3.0,
h_g=5000.0, t_c=0.00057, k_c=15.0, h=34000.0, T_cool=580.0)
# -> T_0 = 1313.46 K, T_fs = 782.944 K, T_ci = 627.671 K   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_fuel_rod_temperatures', q_lin=40000.0, R_f=0.005, k_f=2.5,
h_g=10000.0, t_c=0.0007, k_c=17.0, h=50000.0, T_cool=600.0)
# -> T_0 = 2071.97 K, T_fs = 798.729 K, T_ci = 671.405 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fuel rod temperatures: q_lin = 200 W/cm, R_f = 4.1 mm, k_f = 3 W/(m*K), h_g = 5000 W/(m^2*K), t_c = 0.57 mm, k_c = 15 W/(m*K), h = 34000 W/(m^2*K), T_cool = 580 K	SOLVED	T_0 = 1313.46 K, T_fs = 782.944 K, T_ci = 627.671 K
Fuel rod temperatures: q_lin = 40 kW/m, R_f = 5 mm, k_f = 2.5 W/(m*K), h_g = 10000 W/(m^2*K), t_c = 0.7 mm, k_c = 17 W/(m*K), h = 50000 W/(m^2*K), T_cool = 600 K	SOLVED	T_0 = 2071.97 K, T_fs = 798.729 K, T_ci = 671.405 K
Fuel rod temperatures: R_f = 4.1 mm, k_f = 3 W/(m*K), h_g = 5000 W/(m^2*K), t_c = 0.57 mm, k_c = 15 W/(m*K), h = 34000 W/(m^2*K), T_cool = 580 K	NEEDS_INPUT	nuc_fuel_rod_temperatures needs q_lin (linear heat rate q' (W/m)). Example: solve('nuc_fuel_rod_temperatures', q_lin='20
Fuel rod temperatures: q_lin = 200 W/cm, R_f = 4.1 mm, k_f = 3 W/(m*K), h_g = 5000 W/(m^2*K), t_c = 0.57 mm, k_c = 15 W/(m*K), h = 34000 W/(m^2*K), T_cool = 580 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Fuel centreline temperature with temperature-dependent conductivity — `nuc_centreline_temperature_variable_k`

The centreline temperature of a cylindrical fuel pellet at a stated linear heat rate and surface temperature when its conductivity varies as k = 1/(A + B T) (the usual form for UO2), through the conductivity integral, with the conductivities at the surface and the centre.

$$\int_{T_s}^{T_0} k(T)\,dT = \frac{q'}{4\pi}$$

input	meaning	condition
`q_lin`	linear heat rate q'	
`T_s`	pellet surface temperature	
`A`	A in k = 1/(A + B T) (mK/W)	
`B`	B in k = 1/(A + B T) (m/W)	

output	meaning
`T_0`	centreline temperature
`dT`	centre-to-surface temperature rise
`k_s`	conductivity at the surface
`k_0`	conductivity at the centre

- **Method:** int_{Ts}^{T0} k $dT = q'$/(4 pi): T0 = [(A + B Ts) e^(B q'/(4 pi)) - A]/B
- **Assumes:** uniform heat generation; radial conduction, steady state; k(T) $= 1/(A + B\,\mathrm{T})$ with T in K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_centreline_temperature_variable_k', q_lin=30000.0, T_s=800.0, A=0.0452, B=0.000246)
# -> T_0 = 1586.1 K, dT = 786.098 K, k_s = 4.13223 W/(m*K), k_0 = 2.29684 W/(m*K)   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_centreline_temperature_variable_k', q_lin=15000.0, T_s=700.0, A=0.04, B=0.0002)
# -> T_0 = 942.675 K, dT = 242.675 K, k_s = 5.55556 W/(m*K), k_0 = 4.3757 W/(m*K)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Centreline temperature variable conductivity: q_lin = 300 W/cm, T_s = 800 K, A = 0.0452 m*K/W, B = 2.46e-4 m/W	SOLVED	T_0 = 1586.1 K, dT = 786.098 K, k_s = 4.13223 W/(m*K), k_0 = 2.29684 W/(m*K)
Centreline temperature variable conductivity: q_lin = 15 kW/m, T_s = 700 K, A = 0.04 m*K/W, B = 2e-4 m/W	SOLVED	T_0 = 942.675 K, dT = 242.675 K, k_s = 5.55556 W/(m*K), k_0 = 4.3757 W/(m*K)
Centreline temperature variable conductivity: T_s = 800 K, A = 0.0452	NEEDS_INPUT	nuc_centreline_temperature_variable_k needs q_lin (linear heat rate q'). Example:

question	outcome	result
m*K/W, B = 2.46e-4 m/W		solve('nuc_centreline_temperature_vari
Centreline temperature variable conductivity: q_lin = 300 W/cm, T_s = 800 K, A = 0.0452 m*K/W, B = 2.46e-4 m/W, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Coolant temperature along a channel with a chopped-cosine power shape — `nuc_channel_coolant_temperature`

The bulk coolant temperature at a height z (from the core mid-plane) in a heated channel whose linear heat rate follows a chopped cosine, the outlet temperature and the channel's total power.

$$T(z) = T_{in} + \frac{q_0 H_e}{\pi \dot{m} c_p}\left[\sin\frac{\pi z}{H_e} + \sin\frac{\pi H}{2H_e}\right]$$

input	meaning	condition
`q0`	peak linear heat rate (at the core mid-plane)	
`H`	heated (core) height	
`H_e`	extrapolated height of the chopped-cosine power shape $(H_e \geq H)$	
`m_dot`	coolant mass flow rate in the channel	
`c_p`	coolant specific heat	
`T_in`	coolant inlet temperature	
`z`	height above the core mid-plane (negative below)	

output	meaning
`T`	coolant temperature at z
`T_out`	coolant outlet temperature
`Q`	total heat added to the channel

- **Method:** T(z) = T_in + (q_0 H_e/(pi m c_p)) [sin(pi z/H_e) + sin(pi H/(2 H_e))]
- **Assumes:** steady state, single-phase coolant, constant c_p; q'(z) = q_0 cos(pi z/H_e)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_channel_coolant_temperature', q0=43000.0, H=3.66, H_e=4.0,
m_dot=0.3, c_p=5500.0, T_in=565.0, z=0.0)
# -> T = 597.886 K, T_out = 630.772 K, Q = 108524 W   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_channel_coolant_temperature', q0=30000.0, H=2.0, H_e=2.0,
m_dot=0.2, c_p=4200.0, T_in=520.0, z=0.5)
# -> T = 558.813 K, T_out = 565.473 K, Q = 38197.2 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Channel coolant temperature: q0 = 43 kW/m, H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, z = 0 m	SOLVED	T = 597.886 K, T_out = 630.772 K, Q = 108524 W
Channel coolant temperature: q0 = 30 kW/m, H = 2 m, H_e = 2 m, m_dot = 0.2 kg/s, c_p = 4200 J/(kg*K), T_in = 520 K, z = 50 cm	SOLVED	T = 558.813 K, T_out = 565.473 K, Q = 38197.2 W
Channel coolant temperature: H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, z = 0 m	NEEDS_INPUT	nuc_channel_coolant_temperature needs q0 (peak linear heat rate (at the core mid-plane)). Example: solve('nuc_channel_co
Channel coolant temperature: q0 = 43 kW/m, H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, z = 0 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Location and value of the maximum cladding surface temperature — `nuc_max_clad_temperature`

Where along a channel with a chopped-cosine power shape the cladding surface is hottest (above the mid-plane, because the coolant warms up), its temperature there and the coolant outlet temperature.

$$\tan\frac{\pi z_{max}}{H_e} = \frac{2R_o h H_e}{\dot{m} c_p}$$

input	meaning	condition
`q0`	peak linear heat rate (at the core mid-plane)	
`H`	heated (core) height	
`H_e`	extrapolated height of the chopped-cosine power shape $(H_e \geq H)$	
`m_dot`	coolant mass flow rate in the channel	
`c_p`	coolant specific heat	
`T_in`	coolant inlet temperature	
`R_o`	cladding outer radius	
`h`	coolant heat transfer coefficient	

output	meaning
`z_max`	height of the maximum cladding temperature above the mid-plane
`T_max`	maximum cladding surface temperature
`T_out`	coolant outlet temperature

- **Method:** $dT_c/dz = 0$: tan(pi z/H_e) = 2 R_o h H_e/(m c_p); $T_c = T$(z) + q'(z)/(2 pi R_o h)
- **Assumes:** single-phase coolant, constant h and c_p; q'$(z) = q_0$ cos(pi z/H_e); the maximum may sit at the core exit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_max_clad_temperature', q0=43000.0, H=3.66, H_e=4.0,
m_dot=0.3, c_p=5500.0, T_in=565.0, R_o=0.00475, h=34000.0)
# -> z_max = 0.845823 m, T_max = 651.707 K, T_out = 630.772 K   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_max_clad_temperature', q0=30000.0, H=2.0, H_e=2.2, m_dot=0.2,
c_p=4200.0, T_in=520.0, R_o=0.005, h=20000.0)
# -> z_max = 0.337895 m, T_max = 598.656 K, T_out = 569.511 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum cladding temperature: q0 = 43 kW/m, H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, R_o = 4.75 mm, h = 34000 W/(m^2*K)	SOLVED	z_max = 0.845823 m, T_max = 651.707 K, T_out = 630.772 K
Maximum cladding temperature: q0 = 30 kW/m, H = 2 m, H_e = 2.2 m, m_dot = 0.2 kg/s, c_p = 4200 J/(kg*K), T_in = 520 K, R_o = 5 mm, h = 20000 W/(m^2*K)	SOLVED	z_max = 0.337895 m, T_max = 598.656 K, T_out = 569.511 K
Maximum cladding temperature: H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, R_o = 4.75 mm, h = 34000 W/(m^2*K)	NEEDS_INPUT	nuc_max_clad_temperature needs q0 (peak linear heat rate (at the core mid-plane)). Example: solve('nuc_max_clad_temperat
Maximum cladding temperature: q0 = 43 kW/m, H = 3.66 m, H_e = 4 m, m_dot = 0.3 kg/s, c_p = 5500 J/(kg*K), T_in = 565 K, R_o = 4.75 mm, h = 34000 W/(m^2*K), zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Speeds and mean energy of thermal neutrons (Maxwell distribution) —

`nuc_thermal_neutron_speeds`

The most probable, mean and root-mean-square speeds and the mean kinetic energy of neutrons in thermal equilibrium (Maxwell-Boltzmann distribution) at a stated temperature.

$$v_p = \sqrt{\frac{2kT}{m_n}}, \quad \bar{v} = \frac{2}{\sqrt{\pi}} v_p$$

input	meaning	condition
`T`	neutron (moderator) temperature	

output	meaning
`v_p`	most probable speed sqrt(2kT/m)
`v_mean`	mean speed
`v_rms`	root-mean-square speed
`E_mean`	mean kinetic energy $3kT/2$
`E_mean_eV`	mean kinetic energy in eV

- **Method:** $v_p = sqrt$(2kT/m_n), v_mean = 2 v_p/sqrt(pi), v_rms = sqrt(3/2) v_p, E_mean = 3kT/2
- **Assumes:** Maxwell-Boltzmann distribution of the neutron density; neutron mass $1.67492750056e-27$ kg
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_thermal_neutron_speeds', T=293.6)
# -> v_p = 2200.07 m/s, v_mean = 2482.51 m/s, v_rms = 2694.53 m/s, E_mean = 6.08038e-21 J,
E_mean_eV = 0.0379507   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_thermal_neutron_speeds', T=600.0)
# -> v_p = 3145.1 m/s, v_mean = 3548.86 m/s, v_rms = 3851.94 m/s, E_mean = 1.24258e-20 J, E_mean_eV
= 0.077556   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal neutron speeds: T = 293.6 K	SOLVED	v_p = 2200.07 m/s, v_mean = 2482.51 m/s, v_rms = 2694.53 m/s, E_mean = 6.08038e-21 J, E_mean_eV = 0.0379507
Thermal neutron speeds: T = 600 K	SOLVED	v_p = 3145.1 m/s, v_mean = 3548.86 m/s, v_rms = 3851.94 m/s, E_mean = 1.24258e-20 J, E_mean_eV = 0.077556
Thermal neutron speeds:	NEEDS_INPUT	nuc_thermal_neutron_speeds needs T (neutron (moderator) temperature). Example: solve('nuc_thermal_neutron_speeds', T='29
Thermal neutron speeds: T = 293.6 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Linear attenuation coefficient from narrow-beam transmission measurements — `nuc_attenuation_coefficient_fit`

The linear attenuation coefficient of an absorber from intensities measured behind several thicknesses (least-squares fit of ln I against x), with the half- and tenth-value layers and the fitted unattenuated intensity.

$$\ln I = \ln I_0 - \mu x$$

input	meaning	condition
`x`	absorber thicknesses, in cm	
`I`	measured intensities or count rates (any consistent unit)	

output	meaning
`mu`	fitted linear attenuation coefficient
`HVL`	half-value layer
`TVL`	tenth-value layer
`I0`	fitted intensity with no absorber

- **Method:** least squares of ln $I = ln\ I_0 - mu$ x; HVL = ln $2/mu$, TVL = ln $10/mu$
- **Assumes:** narrow-beam geometry (no buildup); background already subtracted; unweighted least squares of ln I

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_attenuation_coefficient_fit', x=[0.0, 1.0, 2.0, 3.0, 4.0], I=
[1000.0, 520.0, 270.0, 141.0, 73.0])
# -> mu = 65.3966 1/m, HVL = 0.0105991 m, TVL = 0.0352096 m, I0 = 1000.02   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_attenuation_coefficient_fit', x=[0.0, 5.0, 10.0], I=[5000.0,
2450.0, 1210.0])
# -> mu = 14.1882 1/m, HVL = 0.0488539 m, TVL = 0.162289 m, I0 = 4993.44   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Attenuation coefficient fit: x = [0.0, 1.0, 2.0, 3.0, 4.0], I = [1000.0, 520.0, 270.0, 141.0, 73.0]	SOLVED	mu = 65.3966 1/m, HVL = 0.0105991 m, TVL = 0.0352096 m, I0 = 1000.02
Attenuation coefficient fit: x = [0.0, 5.0, 10.0], I = [5000.0, 2450.0, 1210.0]	SOLVED	mu = 14.1882 1/m, HVL = 0.0488539 m, TVL = 0.162289 m, I0 = 4993.44
Attenuation coefficient fit: I = [1000.0, 520.0, 270.0, 141.0, 73.0]	NEEDS_INPUT	nuc_attenuation_coefficient_fit needs x (absorber thicknesses, in cm). Example: solve('nuc_attenuation_coefficient_fit',
Attenuation coefficient fit: x = [0.0, 1.0, 2.0, 3.0, 4.0], I = [1000.0, 520.0, 270.0, 141.0, 73.0], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Attenuation coefficient fit: x = [0, 1, 2, 3, 4] cm, I = [1000, 520, 270, 141, 73]	SOLVED	mu = 65.3966 1/m, HVL = 0.0105991 m, TVL = 0.0352096 m, I0 = 1000.02

Expected count rate from a point source on a detector's axis — `nuc_count_rate_point_source`

The count rate a detector of stated intrinsic efficiency records from a point source on its axis, from the activity, the emission probability per decay and the geometric efficiency of the detector face.

$$R = A\,y\,\varepsilon_{int}\,\frac{1}{2}\left(1 - \frac{d}{\sqrt{d^2 + a^2}}\right)$$

input	meaning	condition
`A`	source activity	
`y`	emission probability of the counted radiation per decay	
`d`	source-to-face distance	
`a`	radius of the detector face	
`eps_int`	intrinsic efficiency (counts per radiation reaching the face)	

output	meaning
`R`	expected count rate
`g`	geometric efficiency

output	meaning
`R_cpm`	expected count rate in counts per minute

- **Method:** $R = A$ y g eps_int, $g = \frac{1-\frac{d}{\sqrt{d^2+a^2}}}{2}$
- **Assumes:** point source on the axis; no dead time, no attenuation between source and detector
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_count_rate_point_source', A=37000.0, y=0.851, d=0.1,
a=0.0254, eps_int=0.6)
# -> R = 290.719 1/s, g = 0.0153883, R_cpm = 17443.1   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_count_rate_point_source', A=100000.0, y=1.0, d=0.01, a=0.01,
eps_int=1.0)
# -> R = 14644.7 1/s, g = 0.146447, R_cpm = 878680   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Count rate from a point source: A = 1 uCi, y = 85.1%, d = 10 cm, a = 2.54 cm, eps_int = 60%	SOLVED	R = 290.719 1/s, g = 0.0153883, R_cpm = 17443.1
Count rate from a point source: A = 100 kBq, y = 1.0, d = 1 cm, a = 1 cm, eps_int = 1.0	SOLVED	R = 14644.7 1/s, g = 0.146447, R_cpm = 878680
Count rate from a point source: y = 85.1%, d = 10 cm, a = 2.54 cm, eps_int = 60%	NEEDS_INPUT	nuc_count_rate_point_source needs A (source activity). Example: solve('nuc_count_rate_point_source', A='37000 1/s', y=0.
Count rate from a point source: A = 1 uCi, y = 85.1%, d = 10 cm, a = 2.54 cm, eps_int = 60%, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Neutron flux from a measured foil activation — `nuc_flux_from_foil_activation`

The neutron flux that irradiated an activation foil, from the foil's measured activity a decay time after an irradiation of known length, its mass, molar mass, isotopic abundance and activation cross section; with the saturation activity and the reaction rate per atom.

$$\phi = \frac{A_d\, e^{\lambda t_d}}{N\sigma\,(1 - e^{-\lambda t_{irr}})}$$

input	meaning	condition
`A_d`	foil activity measured at the count	
`m`	foil mass (target element)	
`M`	molar mass of the target element	
`theta`	isotopic abundance of the target nuclide	
`sigma`	activation cross section (barns)	

input	meaning	condition
`T_half`	half-life of the product	
`t_irr`	irradiation time	
`t_d`	time from the end of irradiation to the count	

output	meaning
`phi`	neutron flux
`A_sat`	saturation activity
`R_atom`	reaction rate per target atom sigma phi $(1/s)$

- **Method:** A_sat = A_d e^(lambda t_d)/(1 - e^(-lambda t_irr)); phi = A_sat/(N sigma), $N = m$ N_A theta/M
- **Assumes:** constant flux during the irradiation; thin foil (no self-shielding), no target depletion
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_flux_from_foil_activation', A_d=20000.0, m=0.0001, M=0.19697,
theta=1.0, sigma=9.865e-27, T_half=232848.0, t_irr=3600.0, t_d=1800.0)
# -> phi = 6.2543e+11 1/(m^2*s), A_sat = 1.88637e+06 1/s, R_atom = 6.16987e-15 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_flux_from_foil_activation', A_d=500.0, m=0.0005, M=0.11482,
theta=0.957, sigma=1.57e-26, T_half=3257.4, t_irr=7200.0, t_d=600.0)
# -> phi = 1.83922e+07 1/(m^2*s), A_sat = 724.682 1/s, R_atom = 2.88757e-19 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flux from foil activation: A_d = 20 kBq, m = 0.1 g, M = 196.97 g/mol, theta = 1.0, sigma = 98.65 b, T_half = 2.695 d, t_irr = 1 h, t_d = 30 min	SOLVED	phi = 6.2543e+11 1/(m^2*s), A_sat = 1.88637e+06 1/s, R_atom = 6.16987e-15 1/s
Flux from foil activation: A_d = 500 Bq, m = 0.5 g, M = 114.82 g/mol, theta = 95.7%, sigma = 157 b, T_half = 54.29 min, t_irr = 2 h, t_d = 10 min	SOLVED	phi = 1.83922e+07 1/(m^2*s), A_sat = 724.682 1/s, R_atom = 2.88757e-19 1/s
Flux from foil activation: m = 0.1 g, M = 196.97 g/mol, theta = 1.0, sigma = 98.65 b, T_half = 2.695 d, t_irr = 1 h, t_d = 30 min	NEEDS_INPUT	nuc_flux_from_foil_activation needs A_d (foil activity measured at the count). Example: solve('nuc_flux_from_foil_activa
Flux from foil activation: A_d = 20 kBq, m = 0.1 g, M = 196.97 g/mol, theta = 1.0, sigma = 98.65 b, T_half = 2.695 d, t_irr = 1 h, t_d = 30 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

Effective dose from organ equivalent doses and tissue weighting factors — `nuc_effective_dose`

The effective dose E = sum w_T H_T from the equivalent doses of the irradiated tissues and their tissue weighting factors (stated; ICRP 103: 0.12 for red marrow, colon, lung, stomach, breast and the remainder, 0.08

gonads, 0.04 bladder, oesophagus, liver, thyroid, 0.01 bone surface, brain, salivary glands, skin).

$$E = \sum_T w_T \, H_T$$

input	meaning	condition
`H`	equivalent dose of each tissue in mSv	
`w`	tissue weighting factor of each tissue (as stated)	

output	meaning
`E_mSv`	effective dose in mSv
`E`	effective dose (Sv)
`w_sum`	sum of the weighting factors given

- **Method:** E = sum_T w_T H_T
- **Assumes:** one weighting factor per tissue, $0 < w_T \leq 1$, sum at most 1; tissues not listed received no dose
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_effective_dose', H=[10.0, 5.0, 2.0], w=[0.12, 0.04, 0.01])
# -> E_mSv = 1.42, E = 0.00142 J/kg, w_sum = 0.17   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_effective_dose', H=[3.0, 3.0], w=[0.12, 0.12])
# -> E_mSv = 0.72, E = 0.00072 J/kg, w_sum = 0.24   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective dose: H = [10.0, 5.0, 2.0], w = [0.12, 0.04, 0.01]	SOLVED	E_mSv = 1.42, E = 0.00142 J/kg, w_sum = 0.17
Effective dose: H = [3.0, 3.0], w = [0.12, 0.12]	SOLVED	E_mSv = 0.72, E = 0.00072 J/kg, w_sum = 0.24
Effective dose: w = [0.12, 0.04, 0.01]	NEEDS_INPUT	nuc_effective_dose needs H (equivalent dose of each tissue in mSv). Example: solve('nuc_effective_dose', H=[10.0, 5.0, 2
Effective dose: H = [10.0, 5.0, 2.0], w = [0.12, 0.04, 0.01], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Effective dose: H = [10, 5], w = [0.6, 0.7]	INVALID_INPUT	Effective dose from organ equivalent doses and tissue weighting factors: the tissue weighting factors must not sum to mo

Committed effective dose from a dose coefficient — `nuc_committed_dose_coefficient`

The committed effective dose E(50) = e(50) I of an intake I of a radionuclide with the dose coefficient e(50) (Sv/Bq, from ICRP tables for the route and chemical form).

$$E(50) = e(50)\, I$$

input	meaning	condition
`I`	activity taken in (Bq, kBq, MBq, Ci ...)	
`e`	dose coefficient e(50) (Sv/Bq)	

output	meaning
`E`	committed effective dose (Sv)
`E_mSv`	committed effective dose in mSv
`I_Bq`	intake in Bq

- **Method:** E(50) = e(50) I
- **Assumes:** dose coefficient for the route, particle size and chemical form of the intake; $e \leq 1e-3$ Sv/Bq
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_committed_dose_coefficient', I=1000000.0, e=2.2e-08)
# -> E = 0.022 J/kg, E_mSv = 22, I_Bq = 1e+06 1/s   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_committed_dose_coefficient', I=37000.0, e=1.3e-08)
# -> E = 0.000481 J/kg, E_mSv = 0.481, I_Bq = 37000 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Committed dose from a dose coefficient: I = 1 MBq, e = 2.2e-8 Sv/Bq	SOLVED	E = 0.022 J/kg, E_mSv = 22, I_Bq = 1e+06 1/s
Committed dose from a dose coefficient: I = 1 uCi, e = 1.3e-8 Sv/Bq	SOLVED	E = 0.000481 J/kg, E_mSv = 0.481, I_Bq = 37000 1/s
Committed dose from a dose coefficient: e = 2.2e-8 Sv/Bq	NEEDS_INPUT	nuc_committed_dose_coefficient needs I (activity taken in (Bq, kBq, MBq, Ci ...)). Example: solve('nuc_committed_dose_co
Committed dose from a dose coefficient: I = 1 MBq, e = 2.2e-8 Sv/Bq, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Committed dose from a dose coefficient: I = 1 MBq, e = -2.2e-8 Sv/Bq	INVALID_INPUT	Committed effective dose from a dose coefficient: the intake and the dose coefficient must be a positive number.

Power density and power from the fission reaction rate — `nuc_power_from_reaction_rate`

The fission reaction rate density $R = N$ sigma_f phi, the power density q''' = E_f N sigma_f phi and, with a volume, the power, from the fissile number density, the fission cross section, the flux and the energy per fission.

$$q''' = E_f N \sigma_f \phi, \quad P = q''' V$$

input	meaning	condition
`E_f`	recoverable energy per fission (e.g. 200 MeV)	

input	meaning	condition
`N`	fissile atoms per unit volume	
`sigma`	fission cross section (barns)	
`phi`	neutron flux	
`V`	volume (optional, for the power)	

output	meaning
`R`	fission rate density (1/(m^3 s))
`q`	power density
`q_MW_m3`	power density in MW/m^3
`P`	power E_f N sigma phi V
`P_MW`	power in MW

- **Method:** $R = N$ sigma phi, q''' = E_f R, $P = q$''' V
- **Assumes:** one-group cross section and flux; uniform over the volume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_power_from_reaction_rate', E_f=3.204353268e-11, N=1e+27,
sigma=5.826e-26, phi=3e+17)
# -> R = 1.7478e+19 1/(m^3*s), q = 5.60057e+08 W/m^3, q_MW_m3 = 560.057   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_power_from_reaction_rate', E_f=3.204353268e-11, N=4.8e+26,
sigma=5.826e-26, phi=1e+17, V=30.0)
# -> R = 2.79648e+18 1/(m^3*s), q = 8.96091e+07 W/m^3, q_MW_m3 = 89.6091, P = 2.68827e+09 W, P_MW =
2688.27   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power from reaction rate: E_f = 200 MeV, N = 1e21 1/cm^3, sigma = 582.6 b, phi = 3e13 1/(cm^2*s)	SOLVED	R = 1.7478e+19 1/(m^3*s), q = 5.60057e+08 W/m^3, q_MW_m3 = 560.057
Power from reaction rate: E_f = 200 MeV, N = 4.8e26 1/m^3, sigma = 582.6 b, phi = 1e13 1/(cm^2*s), V = 30 m^3	SOLVED	R = 2.79648e+18 1/(m^3*s), q = 8.96091e+07 W/m^3, q_MW_m3 = 89.6091, P = 2.68827e+09 W, P_MW = 2688.27
Power from reaction rate: N = 1e21 1/cm^3, sigma = 582.6 b, phi = 3e13 1/(cm^2*s)	NEEDS_INPUT	nuc_power_from_reaction_rate needs E_f (recoverable energy per fission (e.g. 200 MeV)). Example: solve('nuc_power_from_r
Power from reaction rate: E_f = 200 MeV, N = 1e21 1/cm^3, sigma = 582.6 b, phi = 3e13 1/(cm^2*s), zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Power from reaction rate: E_f = 200 MeV, N = 1e21 1/cm^3, sigma = -582.6 b, phi = 3e13 1/(cm^2*s)	INVALID_INPUT	Power density and power from the fission reaction rate: E_f, N, sigma and phi must be a positive number.

Multiplication factor by the four- and six-factor formulas — `nuc_six_factor_formula`

k_inf = eta f p eps (four-factor formula) and, with the fast and thermal non-leakage probabilities, k_eff = k_inf P_FNL P_TNL (six-factor formula), with the reactivity $(k-1)/k$.

$$k_{eff} = \eta f p \varepsilon P_{FNL} P_{TNL}$$

input	meaning	condition
`eta`	reproduction factor eta	
`f`	thermal utilization f	
`p`	resonance escape probability p	
`eps`	fast fission factor epsilon	
`P_FNL`	fast non-leakage probability (optional)	
`P_TNL`	thermal non-leakage probability (optional)	

output	meaning
`k_inf`	infinite multiplication factor
`k_eff`	effective multiplication factor (k_inf without leakage factors)
`P_NL`	total non-leakage probability
`rho`	reactivity (k_eff - 1)/k_eff

- **Method:** k_inf = eta f p eps; k_eff = k_inf P_FNL P_TNL
- **Assumes:** $0 < f$, p, P_FNL, P_TNL <= 1; missing non-leakage factors are taken as 1 (an infinite medium)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_six_factor_formula', eta=2.02, f=0.71, p=0.87, eps=1.03)
# -> k_inf = 1.28519, k_eff = 1.28519, P_NL = 1, rho = 0.221903   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_six_factor_formula', eta=1.65, f=0.9, p=0.75, eps=1.02,
P_FNL=0.97, P_TNL=0.99)
# -> k_inf = 1.13603, k_eff = 1.09092, P_NL = 0.9603, rho = 0.0833465   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Six-factor formula: eta = 2.02, f = 0.71, p = 0.87, eps = 1.03	SOLVED	k_inf = 1.28519, k_eff = 1.28519, P_NL = 1, rho = 0.221903
Six-factor formula: eta = 1.65, f = 0.9, p = 0.75, eps = 1.02, P_FNL = 0.97, P_TNL = 0.99	SOLVED	k_inf = 1.13603, k_eff = 1.09092, P_NL = 0.9603, rho = 0.0833465
Six-factor formula: f = 0.71, p = 0.87, eps = 1.03	NEEDS_INPUT	nuc_six_factor_formula needs eta (reproduction factor eta). Example: solve('nuc_six_factor_formula', eta=2.02, f=0.71, p
Six-factor formula: eta = 2.02, f = 0.71, p = 0.87, eps = 1.03, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa

question	outcome	result
Six-factor formula: eta = 2.02, f = 1.3, p = 0.87, eps = 1.03	INVALID_INPUT	Multiplication factor by the four- and six-factor formulas: f must lie in (0, 1].

Thermal diffusion length and migration length — `nuc_diffusion_length`

The thermal diffusion length $L = sqrt$(D/Sigma_a) of a medium and, with the Fermi age tau, the migration area $M^2 = L^2 + \tau$ and migration length.

$$L = \sqrt{\frac{D}{\Sigma_a}}, \quad M^2 = L^2 + \tau$$

input	meaning	condition
`D`	diffusion coefficient (cm)	
`Sigma_a`	macroscopic absorption cross section	
`tau`	Fermi age to thermal (cm^2, optional)	

output	meaning
`L`	diffusion length
`L2`	diffusion area L^2
`L_cm`	diffusion length in cm
`M2`	migration area $L^2 + \tau$
`M`	migration length

- **Method:** $L^2 = D/Sigma_a$; $M^2 = L^2 + \tau$
- **Assumes:** D, $Sigma_a > 0$; one-group (thermal) diffusion theory
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_diffusion_length', D=0.0016, Sigma_a=1.97)
# -> L = 0.0284988 m, L2 = 0.000812183 m^2, L_cm = 2.84988   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_diffusion_length', D=0.0084, Sigma_a=0.032, tau=0.0368)
# -> L = 0.512348 m, L2 = 0.2625 m^2, L_cm = 51.2348, M2 = 0.2993 m^2, M = 0.547083 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion length: D = 0.16 cm, Sigma_a = 0.0197 1/cm	SOLVED	L = 0.0284988 m, L2 = 0.000812183 m^2, L_cm = 2.84988
Diffusion length: D = 0.84 cm, Sigma_a = 0.00032 1/cm, tau = 368 cm^2	SOLVED	L = 0.512348 m, L2 = 0.2625 m^2, L_cm = 51.2348, M2 = 0.2993 m^2, M = 0.547083 m
Diffusion length: Sigma_a = 0.0197 1/cm	NEEDS_INPUT	nuc_diffusion_length needs D (diffusion coefficient (cm)). Example: solve('nuc_diffusion_length', D='0.0016 m', Sigma_a=

question	outcome	result
Diffusion length: D = 0.16 cm, Sigma_a = 0.0197 1/cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Diffusion length: D = 0.16 cm, Sigma_a = 0 1/cm	INVALID_INPUT	Thermal diffusion length and migration length: D and Sigma_a must be a positive number.

Currie critical level, detection limit and minimum detectable activity — `nuc_currie_detection_limits`

Currie's (1968) decision threshold $L_C = k$ sqrt(2B) and detection limit $L_D = k^2 + 2$ L_C (net counts, paired blank, $\alpha = \beta = 0.05$, k = 1.645) from the background counts B, and with the counting time, efficiency and emission yield the minimum detectable activity L_D/(eff y t).

$$L_C = k\sqrt{2B}, \quad L_D = k^2 + 2k\sqrt{2B}, \quad MDA = \frac{L_D}{\varepsilon\, y\, t}$$

input	meaning	condition
`B`	background counts in the counting time	
`t`	counting time (optional, for the MDA)	
`eff`	counting efficiency (optional, with t)	
`y`	emission yield per decay (default 1)	

output	meaning
`L_C`	critical level (net counts)
`L_D`	detection limit (net counts)
`MDA`	minimum detectable activity (Bq)

- **Method:** $L_C = k$ sqrt(2B), $L_D = k^2 + 2$ k sqrt(2B), $k = z$(0.95) = 1.645; MDA = L_D/(eff y t)
- **Assumes:** paired blank (same counting time); $\alpha = \beta = 0.05$; Gaussian approximation of the counts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_currie_detection_limits', B=100.0)
# -> L_C = 23.2617, L_D = 49.229   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_currie_detection_limits', B=400.0, t=600.0, eff=0.25,
y=0.851)
# -> L_C = 46.5235, L_D = 95.7525, MDA = 0.750118 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Currie detection limits: B = 100.0	SOLVED	L_C = 23.2617, L_D = 49.229
Currie detection limits: B = 400.0, t = 10 min, eff = 25%, y = 85.1%	SOLVED	L_C = 46.5235, L_D = 95.7525, MDA = 0.750118 1/s

question	outcome	result
Currie detection limits:	NEEDS_INPUT	nuc_currie_detection_limits needs B (background counts in the counting time). Example: solve('nuc_currie_detection_limit
Currie detection limits: B = 100.0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Currie detection limits: B = 100, t = 10 min	INVALID_INPUT	Currie critical level, detection limit and minimum detectable activity: the minimum detectable activity needs both the c

Single and double escape peaks of a gamma ray — `nuc_escape_peaks`

The energies of the single- and double-escape peaks (E - m_e c^2 and $E - 2$ m_e c^2) that pair production by a gamma ray above 1.022 MeV produces in a detector.

$$E_{SE} = E_\gamma - m_e c^2, \quad E_{DE} = E_\gamma - 2 m_e c^2$$

input	meaning	condition
`E`	gamma-ray energy (MeV, keV)	

output	meaning
`single_escape_keV`	single-escape peak in keV
`double_escape_keV`	double-escape peak in keV
`annihilation_keV`	annihilation photon energy in keV
`E_keV`	full-energy peak in keV

- **Method:** $E_S E = E - m_e\ c^2$, $E_D E = E - 2$ m_e c^2 (m_e $c^2 = 0.51099895069$ MeV)
- **Assumes:** $E > 2$ m_e $c^2 = 1.022$ MeV
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_escape_peaks', E=4.188089721276e-13)
# -> single_escape_keV = 2103, double_escape_keV = 1592, annihilation_keV = 510.999, E_keV = 2614
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_escape_peaks', E=2.1349003648050002e-13)
# -> single_escape_keV = 821.501, double_escape_keV = 310.502, annihilation_keV = 510.999, E_keV =
1332.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Escape peaks: E = 2614 keV	SOLVED	single_escape_keV = 2103, double_escape_keV = 1592, annihilation_keV = 510.999, E_keV = 2614
Escape peaks: E = 1.3325 MeV	SOLVED	single_escape_keV = 821.501, double_escape_keV = 310.502, annihilation_keV = 510.999, E_keV = 1332.5
Escape peaks:	NEEDS_INPUT	nuc_escape_peaks needs E (gamma-ray energy (MeV, keV)). Example: solve('nuc_escape_peaks', E='4.18809e-13 J').

question	outcome	result
Escape peaks: E = 2614 keV, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Escape peaks: E = 662 keV	INVALID_INPUT	Single and double escape peaks of a gamma ray: escape peaks need pair production: the energy must exceed 1.022 MeV.

Half-value and tenth-value layers — `nuc_half_value_layer`

The half-value layer ln $2/mu$, tenth-value layer ln $10/mu$ and mean free path $1/mu$ of a narrow beam in an absorber of linear attenuation coefficient mu, and with a thickness the transmission and number of half-value layers.

$$HVL = \frac{\ln 2}{\mu}, \quad TVL = \frac{\ln 10}{\mu}$$

input	meaning	condition
`mu`	linear attenuation coefficient	
`x`	absorber thickness (optional)	

output	meaning
`HVL`	half-value layer
`TVL`	tenth-value layer
`mfp`	mean free path $1/mu$
`HVL_cm`	half-value layer in cm
`TVL_cm`	tenth-value layer in cm
`transmission`	narrow-beam transmission exp(-mu x)
`n_HVL`	thickness in half-value layers

- **Method:** HVL = ln $2/mu$, TVL = ln $10/mu$, T = e^(-mu x)
- **Assumes:** narrow beam (no buildup); $mu > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_half_value_layer', mu=65.6)
# -> HVL = 0.0105663 m, TVL = 0.0351004 m, mfp = 0.0152439 m, HVL_cm = 1.05663, TVL_cm = 3.51004
(SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_half_value_layer', mu=14.9, x=0.3)
# -> HVL = 0.0465199 m, TVL = 0.154536 m, mfp = 0.0671141 m, HVL_cm = 4.65199, TVL_cm = 15.4536,
transmission = 0.0114473, n_HVL = 6.44885   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Half-value layer: mu = 0.656 1/cm	SOLVED	HVL = 0.0105663 m, TVL = 0.0351004 m, mfp = 0.0152439 m, HVL_cm = 1.05663, TVL_cm = 3.51004

question	outcome	result
Half-value layer: mu = 0.149 1/cm, x = 30 cm	SOLVED	HVL = 0.0465199 m, TVL = 0.154536 m, mfp = 0.0671141 m, HVL_cm = 4.65199, TVL_cm = 15.4536, transmission = 0.0114473, n_HVL = 6.44885
Half-value layer:	NEEDS_INPUT	nuc_half_value_layer needs mu (linear attenuation coefficient). Example: solve('nuc_half_value_layer', mu='65.6 1/m').
Half-value layer: mu = 0.656 1/cm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Half-value layer: mu = -0.656 1/cm	INVALID_INPUT	Half-value and tenth-value layers: the attenuation coefficient must be a positive number.

Radiation unit conversion (Bq and Ci, Gy and rad, Sv and rem) — `nuc_radiation_unit_conversion`

An activity, absorbed dose or equivalent dose written in any of the SI or traditional units, expressed in both systems (1 $Ci = 3.7e10$ Bq, 1 rad = 0.01 Gy, 1 rem = 0.01 Sv).

$$1\ \mathrm{Ci} = 3.7 \times 10^{10}\ \mathrm{Bq}, \quad 1\ \mathrm{rad} = 0.01\ \mathrm{Gy}, \quad 1\ \mathrm{rem} = 0.01\ \mathrm{Sv}$$

input	meaning	condition
`A`	an activity (Bq, kBq, MBq, Ci, mCi, uCi ...)	
`D`	an absorbed dose (Gy, mGy, rad, mrad)	
`H`	an equivalent or effective dose (Sv, mSv, rem, mrem)	

output	meaning
`SI_value`	value in the SI unit (Bq, Gy or Sv)
`traditional_value`	value in the traditional unit (Ci, rad or rem)
`si_unit`	the SI unit
`traditional_unit`	the traditional unit
`value_mCi`	activity in mCi
`value_MBq`	activity in MBq
`value_mSv_or_mGy`	dose in mSv or mGy
`value_mrem_or_mrad`	dose in mrem or mrad

- **Method:** $Ci = A/3.7e10$; rad = 100 Gy-value; rem = 100 Sv-value
- **Assumes:** exactly one of A, D, H
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_radiation_unit_conversion', A=370000000.0)
# -> SI_value = 3.7e+08, traditional_value = 0.01, si_unit = Bq, traditional_unit = Ci, value_mCi =
10, value_MBq = 370   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_radiation_unit_conversion', H=0.05)
# -> SI_value = 0.05, traditional_value = 5, si_unit = Sv, traditional_unit = rem, value_mSv_or_mGy
= 50, value_mrem_or_mrad = 5000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radiation unit conversion: A = 10 mCi	SOLVED	SI_value = 3.7e+08, traditional_value = 0.01, si_unit = Bq, traditional_unit = Ci, value_mCi = 10, value_MBq = 370
Radiation unit conversion: H = 5 rem	SOLVED	SI_value = 0.05, traditional_value = 5, si_unit = Sv, traditional_unit = rem, value_mSv_or_mGy = 50, value_mrem_or_mrad = 5000
Radiation unit conversion: D = 250 mrad	SOLVED	SI_value = 0.0025, traditional_value = 0.25, si_unit = Gy, traditional_unit = rad, value_mSv_or_mGy = 2.5, value_mrem_or_mrad = 250
Radiation unit conversion: A = 1 Ci, H = 1 rem	INVALID_INPUT	Radiation unit conversion (Bq and Ci, Gy and rad, Sv and rem): give exactly one quantity: an activity A, an absorbed dos

U-235 depletion at constant power — `nuc_constant_power_depletion`

How much of an initial mass of $U-235$ remains after operation at a constant power (the flux rising as the fuel is consumed), the fraction burnt, the mass consumed and the factor by which the flux has risen.

$$N(t) = N_0 - \frac{P\,t}{E_f}\frac{\sigma_a}{\sigma_f}, \qquad \frac{\phi(t)}{\phi_0} = \frac{N_0}{N(t)}$$

input	meaning	condition
`m0`	initial mass of $U-235$	
`P`	constant thermal power	
`t`	operating time	
`E_f`	recoverable energy per fission (e.g. 200 MeV)	
`sigma_a`	$U-235$ absorption cross section (barns)	
`sigma_f`	$U-235$ fission cross section (barns)	

output	meaning
`m`	$U-235$ mass remaining
`burnt`	fraction of $U-235$ consumed
`consumed`	$U-235$ mass consumed
`flux_ratio`	flux at t over the initial flux (N_0/N)
`fissions`	number of fissions P t/E_f

- **Method:** fissions = P t/E_f; atoms absorbed = fissions sigma_a/sigma_f; $N = N_0 - P$ t sigma_a/(E_f sigma_f); $\phi/phi0 = N_0/N$
- **Assumes:** constant power, one group; $U-235$ only (no breeding); M(U-235) = 235.043928 g/mol
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
nuclear_engineering_ai999x.solve('nuc_constant_power_depletion', m0=100.0, P=30000000.0,
t=31557600.0, E_f=3.204353268e-11, sigma_a=6.809e-26, sigma_f=5.826e-26)
# -> m = 86.5229 kg, burnt = 0.134771, consumed = 13.4771 kg, flux_ratio = 1.15576, fissions =
2.95451e+25   (SOLVED)
```

```
nuclear_engineering_ai999x.solve('nuc_constant_power_depletion', m0=1.0, P=100000.0, t=2592000.0,
E_f=3.204353268e-11, sigma_a=6.809e-26, sigma_f=5.826e-26)
# -> m = 0.99631 kg, burnt = 0.00368982, consumed = 0.00368982 kg, flux_ratio = 1.0037, fissions =
8.089e+21   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Depletion at constant power: m0 = 100 kg, P = 30 MW, t = 1 y, E_f = 200 MeV, sigma_a = 680.9 b, sigma_f = 582.6 b	SOLVED	m = 86.5229 kg, burnt = 0.134771, consumed = 13.4771 kg, flux_ratio = 1.15576, fissions = 2.95451e+25
Depletion at constant power: m0 = 1 kg, P = 100 kW, t = 30 d, E_f = 200 MeV, sigma_a = 680.9 b, sigma_f = 582.6 b	SOLVED	m = 0.99631 kg, burnt = 0.00368982, consumed = 0.00368982 kg, flux_ratio = 1.0037, fissions = 8.089e+21
Depletion at constant power: P = 30 MW, t = 1 y, E_f = 200 MeV, sigma_a = 680.9 b, sigma_f = 582.6 b	NEEDS_INPUT	nuc_constant_power_depletion needs m0 (initial mass of U-235). Example: solve('nuc_constant_power_depletion', m0='100 kg
Depletion at constant power: m0 = 100 kg, P = 30 MW, t = 1 y, E_f = 200 MeV, sigma_a = 680.9 b, sigma_f = 582.6 b, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: nuclear_engineering_ai999x.templa
Depletion at constant power: m0 = 1 kg, P = 3000 MW, t = 1 y, E_f = 200 MeV, sigma_a = 680.9 b, sigma_f = 582.6 b	INVALID_INPUT	U-235 depletion at constant power: the fuel would be used up before t at this power.